

THERE’S MORE THAN ONE WAY: MODE COLLAPSE IN RLVR & MODEBENCH

Anonymous authors

Paper under double-blind review

ABSTRACT

Most questions we ask a language model have more than one right answer, and keeping several is worth something: the user gets a choice, and somewhere to go when the first turns out unusable. Training against a verifier spends them, because a correct response earns one point whether it is the thousandth copy of a familiar solution or one never produced before. Seeing this requires deciding when two right answers are the same right answer, so MODEBENCH takes that decision from the verifier itself: across five executable domains every accepted output is run and the verifier reports which answer it is, so breadth becomes how often two correct responses differ — which, unlike a count of distinct answers, cannot rise merely because the model got more questions right. Measured this way, training against a verifier narrows breadth while accuracy holds or climbs, and the frontier models we measured are already narrow. Re:Max keeps the alternatives: the same verifier that tells modes apart indexes a memory holding one verified example per mode, each rehearsed equally, so a solution found once is practiced as often as one found constantly. Across three model scales, two RL objectives and a harder construction of every task, this improves both how often a policy succeeds and how many ways it can, without trading one outcome for the other.

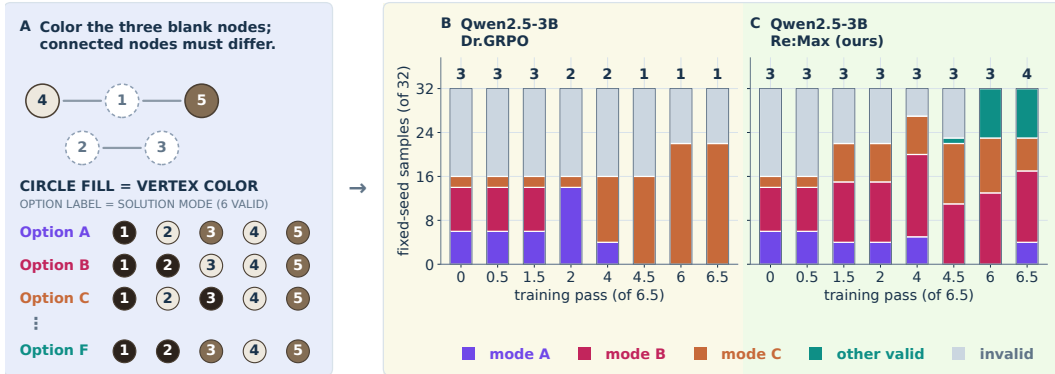


Figure 1: **Correctness can rise while sampled alternatives narrow.** One graph admits six valid colorings. Bars pool four eight-sample draws from matched Qwen2.5-3B runs. At pass 6.5, Dr.GRPO produces 22/32 correct outputs from one mode; Re:Max produces 32/32 across four. Circle fills mark vertex colors; dashed rings mark vertices left uncolored in the prompt. This illustrates the distinction; Fig. 8 separates the effects of the fresh objective and replay.

1 INTRODUCTION

A language model can usually satisfy the same request in more than one way: different explanations, different plans, different programs that all meet the requirements. Those alternatives are worth keeping. They give the user a choice, they are what test-time search (Brown et al., 2024; Snell et al., 2024) and reranking (Cobbe et al., 2021; Wang et al., 2023) consume, and



ModeBench dataset and models



Code repository Links anonymized for review.

they are what is left to fall back on when the first answer turns out to be unusable. What happens to them as a model is trained to succeed more often?

Reinforcement learning with verifiable rewards (RLVR) trains against an executable check (Guo et al., 2025), and that check cannot tell one success from another: a solution the model has produced a thousand times and a solution it has never produced before are both worth exactly one point. Nothing in the objective asks it to keep the second. Fig. 1 shows the cost on a graph-colouring problem with six valid answers: two policies trained under matched conditions end up with very different repertoires, and the more accurate one is the narrower.

This has been hard to establish because measuring it means saying when two correct answers are the same correct answer, and the usual proxies answer a different question. Token-level diversity (Li et al., 2016; Zhu et al., 2018) counts two spellings of one solution as variety. A judge that groups answers by meaning (Farquhar et al., 2024; Zheng et al., 2023) puts a second model between the data and the claim. MODEBENCH instead takes the decision from the verifier that already decides correctness: every accepted output is executed and identified by what it did, so presentation differences merge and genuinely different solutions stay apart. One definition then does double duty — it tells us what to count, and it tells a training run what to remember.

RLVR-only training pushes policies onto a single way of being right while their accuracy holds or rises, and every deployed frontier model we measured is concentrated in the same way even where it answers essentially every prompt correctly. Drawing hundreds of samples instead of eight does not come close to closing the gap: at 512 draws a frontier model still returns a small fraction of the answers its problems admit, and each doubling of the budget adds a fraction of one more. The fix we propose needs no extra sampling: index a memory by the same equivalence that measures breadth, store one verified example per mode, and rehearse every mode equally, so a solution found once is practiced as often as one found constantly.

- MODEBENCH: five executable domains in which the verifier that grades an answer also reports which answer it is, at four difficulty levels, four open model families and 7 frontier models (Section 2).
- RLVR concentrates correct outputs, and so do fixed deployments. A verifier that only certifies whether an answer is right is indifferent to which right answer it gets: the objective is satisfied by any distribution on correct outcomes, and it is the update, not the optimum, that picks one. Under an idealized categorical mean-flow model these binary-reward updates generically converge to a point mass on the initially dominant correct mode. Training follows that prediction: all twelve Dr.GRPO and GRPO before/after comparisons in the domains with measurable initial breadth move toward concentration, and no frontier model we measured reaches 1.5 effective verified modes, which a larger sampling budget does not close (Sections 5.1 and 2.3).
- Re:Max, which stores one verified exemplar per mode and rehearses modes equally. Added to Dr.GRPO it raises accuracy in all 14 complete model-domain blocks and lifts terminal PCMD at every scale we trained; added to MaxRL, a stronger fresh objective, it still adds both, and the gain survives a harder construction of every task (Sections 3 and 5.2–5.4).

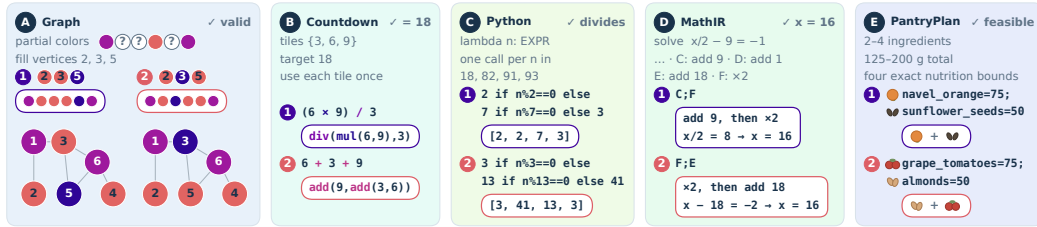


Figure 2: **One prompt can have different verified outputs.** Keys identify a color vector in Graph, a normalized computation tree in Countdown, an executed return vector in Python, an algebraic state trajectory in MathIR, or an ingredient support in PantryPlan. Cosmetic aliases merge; different accepted computations or configurations remain distinct under each domain’s execution rule.

2 MODEBENCH: MEASURING SUCCESSFUL ALTERNATIVES

Before we can say that a model has stopped producing alternatives, we need a rule for when two of its answers count as the same answer. The rule has to be strict enough that reprinting one solution with different whitespace is not recorded as a new idea, and generous enough that two genuinely different solutions are not merged into one. It also has to be cheap, because we want to apply it to every sample of every training run rather than only to a final evaluation — and the same rule will later decide what a training run keeps in memory, so it has to be something a learner can consult online. That last requirement is what makes MODEBENCH a microscope rather than a survey. Demanding an equivalence a learner can act on restricts us to tasks where behavioral sameness can be executed, which is what lets us watch a training property that is otherwise unobservable; it is not a claim to have captured semantic diversity in open-ended natural-language use.

MODEBENCH takes that rule from the verifier. Every domain already has to run a candidate answer to decide whether it is correct; we ask the same execution to also hand back a short canonical description of what the answer did. For a prompt x with specification s_x , the validator maps an output y either to failure $\perp$ or to a key $V(y, s_x) \in \mathcal{C}_x$, and two accepted outputs are the same mode exactly when their keys agree. The full set of valid keys $\mathcal{C}_x$ is never revealed — not its members and not its size. A policy, and the memory that stores its successes, only ever sees keys for answers the policy actually produced.

Fig. 2 shows the five domains, chosen so that “a different mode” means something different in each. In Countdown and MathIR it means the model reached the same answer by a different route. In Python it means the program it wrote behaves differently on the test inputs. In Graph and PantryPlan it means a different valid answer altogether: another coloring, another set of ingredients. Because a mode means something different in each domain, we report per-domain results beside every average. App. B and App. C give the per-domain rules and the prompts.

2.1 SUCCESS AND VERIFIED-MODE DIVERSITY

Two properties of a policy matter here and are easy to confuse: how often it is right, and — given that it is right — which of the available right answers it gives. A benchmark reporting only the first cannot see the failure we are after, and the obvious way to report the second mostly reports the first again.

That obvious way is to count distinct correct answers in K samples. Write P for the chance a sample is correct and q_c for how that correct probability is divided among the valid answers. Then

$$\underbrace{\Pr[D_K > 0]}_{\text{pass}@K} = 1 - (1 - P)^K, \quad \underbrace{\mathbb{E}[D_K]}_{\text{distinct}@K} = \sum_c [1 - (1 - Pq_c)^K], \quad (1)$$

where D_K counts the distinct correct answers. Both sides climb with P : a policy that is simply right more often returns more distinct answers without having broadened at all. Across the 375 untrained measurements of Fig. 3 that count correlates .936 with `pass@8` — close enough to be a second copy of the accuracy axis.

We want a question whose answer cannot move just because the model got more questions right. Of two independent correct responses to the same prompt, how often are they different answers? That is pairwise correct-mode diversity,

$$\text{PCMD} = 1 - \sum_c q_c^2 = \Pr[Z_1 \neq Z_2 \mid \text{both responses verified}]. \quad (2)$$

Correctness has cancelled: (2) depends on q alone, so a policy that doubles its success rate while allocating those successes the same way scores what it scored before. It is the fraction of correct pairs in a draw that disagree, unbiased at every $K \geq 2$ (App. A), and easy to read — 0 means every correct answer is the same one, an even spread over m modes gives $1 - 1/m$, and $1/(1 - \text{PCMD})$ converts back into an effective number of distinct answers.

A fall in PCMD is what we call *mode collapse*: a statement about where correct probability sits, not that any answer reached probability zero, which finite samples cannot establish. It also has a floor — two correct draws are needed for it to exist, so a policy that almost never succeeds supports no

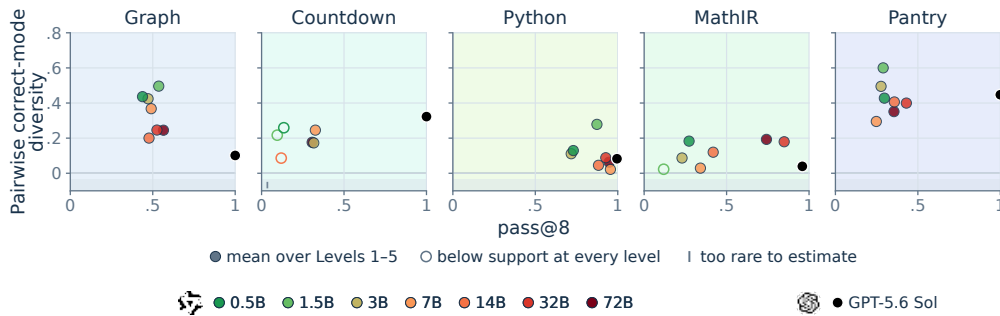


Figure 3: **Breadth narrows as models get larger, and success does not buy it back.** Frozen Qwen2.5-Instruct scales: colour is parameter count; black is the frontier model of Section 2.3, off the ramp. Each mark averages one scale over the five levels; Fig. 10 reads them apart. Hollow marks cleared the support bar at no level, and ticks are too rarely solved to place.

statement about how its successes are spread. Every PCMD we quote therefore carries the number of prompts defining it, and a measurement with too few is left blank rather than filled with noise.

2.2 HARDER TASKS WITH THE SAME NOTION OF A MODE

A benchmark is useful only for as long as models cannot saturate it, so every domain is built at five levels of difficulty. Fig. 3 measures the Qwen2.5-Instruct family across those levels at every scale we could run, which holds the training recipe fixed and lets size be the only thing that varies. Two things show up at once. Harder levels push success down within a scale, and accuracy and breadth move independently even before any training: across the untrained measurements PCMD correlates $-.332$ with $\text{pass}@8$. And as the models get bigger they get more accurate and answer in fewer ways — PCMD falls with scale in 13 of the 20 domain-level blocks that support the comparison, while $\text{pass}@8$ rises.

That second pattern is what the rest of the paper has to explain. If breadth simply disappeared with capability there would be little to be done about it, but size is not what settles it: at matched parameter count, different recipes land at visibly different PCMD, and a change of recipe moves the measure further at fixed scale than scale moves it at fixed recipe (App. B.5). The open question is therefore whether narrowing is something a model grows into or something a particular kind of training does to it. Section 3.1 gives a reason to expect the latter, and the matched comparisons in Section 5 test it directly.

Each level is a fresh construction, not a re-scored copy of the one before it. The validator, the mode equivalence and the distribution of how many modes a prompt admits stay fixed; what grows is the structure to be searched — more hidden graph vertices, more Countdown operands, longer factor case vectors. The fit targets only $\text{pass}@1$ and $\text{pass}@8$ of a reference model, never breadth (App. B.3), and the ladder behaves accordingly: averaged over all 5 domains and every Qwen scale, $\text{pass}@8$ falls at each of the 4 steps of the ladder, from .495 at Level 1 to .307 at Level 5.

The frontier models make the separation of the two axes starkest. They answer .991 of prompts correctly, far above every open checkpoint measured here, yet their mean PCMD is .229 — below the .570 of a 1.7B model that answers only .475 of the same prompts.

2.3 CONCENTRATION IN FRONTIER MODELS

The same measurement applies to models we did not train, and they are already concentrated. Asked a Level-3 Graph problem with six valid colorings, GPT-5.6 Sol is right in all 1,024 draws and almost always right the same way, offering barely one of the six. Correctness is saturated there, so PCMD isolates the collapse exactly; on MathIR it reaches zero (App. L).

Sampling harder does not recover what was lost. The obvious remedy is to draw more. Fig. 4 follows five domains and three levels out to 512 draws per problem, and the curves flatten far below what the problems admit: a model that has settled on two colorings does not find the other four by being asked five hundred times. Python is the extreme case. Its problems admit several hundred distinct return vectors on average, and five hundred draws surface between one and four of them — under two percent of what is there, and under one percent at Levels 2 and 3. The curves have not saturated: fourteen of the fifteen domain-level panels still gain between 256 and 512 draws. But each

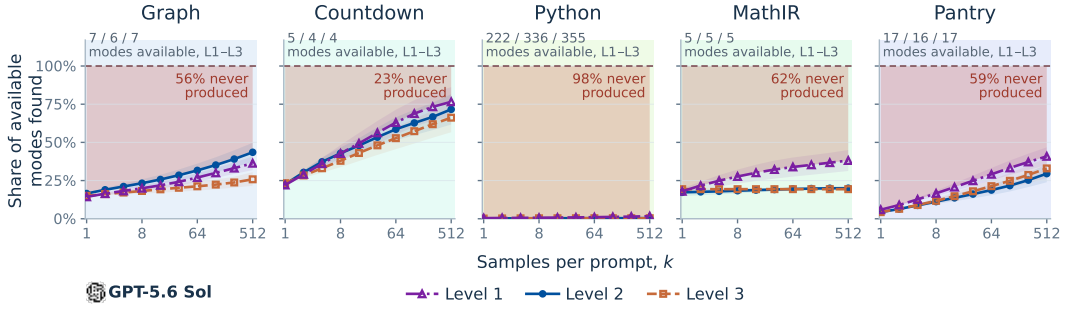


Figure 4: **More samples need not recover the available modes.** GPT-5.6 Sol, medium reasoning; 32 problems per domain and level, 512 responses each. Curves are distinct verified modes in random k -subsets as a share of the modes those problems admit, so the shaded band is what no level ever produced; shading is the pointwise 95% problem bootstrap. Printed counts are the enumerated support those problems admit, in Level 1/2/3 order, and every panel shares one scale (App. J.3).

doubling of the budget buys a fraction of one mode, against gaps of several modes everywhere and several hundred in Python, so sampling does not come remotely close to closing them. These are finite-budget measurements and do not establish a limit or rule out rare unseen modes (App. J.3).

The pattern is not one model’s quirk: no deployment we measured reaches even one and a half effective verified modes (App. L.1). Two interventions widen breadth without replacing it. Because different models collapse onto different answers, mixing responses from two deployments covers more verified modes than either alone (App. K.2); raising temperature widens breadth too, but stops paying early (Fig. 34). Nor is the concentration an artifact of how we look: quadrupling to $K = 32$ moves a trained policy by thousandths and MathIR at 64 draws is as concentrated as at eight (Apps. I and J), while disabling reasoning costs correctness and breadth together (Table 81, App. M.5, Fig. 35).

3 RE:MAX: LEARN FROM NEW SAMPLES AND REMEMBER SUCCESSES

3.1 WHY MEMORY ADDRESSES A DIFFERENT PRESSURE

Collapse follows from two properties of the objective, neither about model size: a binary verifier pays the same for every correct answer, so nothing asks a second way of being right to survive, and a fresh-sample update acts only on what the policy just sampled — which is what it already prefers. They close a loop, the mode that is ahead being drawn most often, reinforced most and drawn more often still, whose end Fig. 5 draws: under the independent-logit geometry of Theorem N.4 a single winner, a property of that geometry rather than of binary reward alone (App. N.6).

Breaking the loop does not depend on the geometry. A rare mode is sampled rarely, so the expected force any bounded fresh-sample score puts on it vanishes: no reweighting of the advantage reaches a mode the policy has stopped producing (App. N.3). A stored target does, whether or not the policy rediscovers it, so a fixed bank with a constant replay dose holds every banked mode above a probability floor (Theorem N.9) — the argument for adding a second path rather than sharpening the first.

3.2 REPLAYING DISTINCT VERIFIED SUCCESSES

The analysis asks for one specific thing: a target that survives its own rarity. Each training prompt therefore owns an initially empty, bounded bank $\mathcal{B}_x$. The first time a mode appears, the bank keeps one verified exemplar e_c that realizes it; later rediscoveries of the same mode add nothing. A deterministic schedule revisits nonempty banks throughout training. For an exemplar of token length $|e_c|$, the replay loss is

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (3)$$

The uniform weight is the whole point: a mode found once is rehearsed exactly as much as a mode found a hundred times, so popularity cannot compound. It is also what makes the bank a counterweight to the fresh update rather than another copy of it — App. A.3 decomposes this

loss into a term that rewards more banked probability and a term minimized by an even spread across the banked modes, which are the two things the fresh flow works against.

Adding this loss to the fresh update gives Re:Max or, with Dr.GRPO, Re:Dr. Exemplars stay outside the fresh reward group, so the two can be varied independently, and controls run the same bank and scoring code with the replay gradient set to zero. Banks hold only training-prompt responses, so held-out evaluation measures what training changed in the policy rather than what the bank can recite (App. D).

3.3 LEARNING FROM FRESH ROLLOUTS

Replay is added to a fresh objective, not substituted for one, so we vary that objective to check that memory is not merely covering for a weak choice. MaxRL combines success objectives across sampling budgets (Tajwar et al., 2026); its binary-reward group estimator weights a correct response more strongly when successes are scarce. Running replay under both MaxRL and Dr.GRPO (Liu et al., 2025) therefore asks whether memory still contributes once fresh learning is already doing better.

Neither objective avoids the starvation above. A rare mode has to be sampled before it can contribute a score term, and a group whose responses are all correct or all incorrect carries zero centered task advantage, so even a policy that is succeeding often can stop receiving any comparative signal. App. D.1 gives the exact advantages and the finite-group interpretation, and App. G.3 measures how often this actually happens during our runs.

Fig. 6 shows the two learning paths we combine: binary rewards learn from fresh samples, while replay returns to successes the policy has already found. Both run on the policy’s own outputs, and neither needs gold solutions.

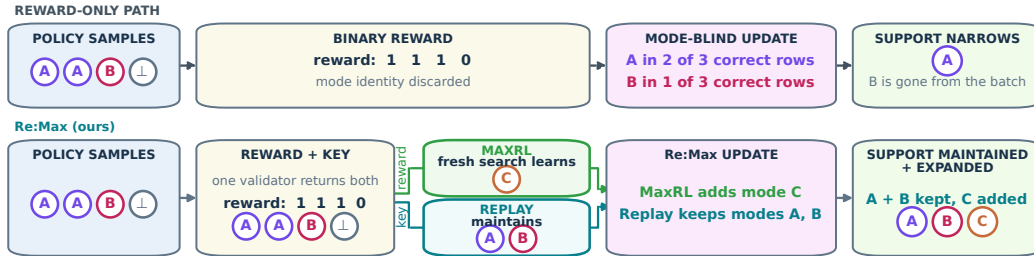


Figure 6: **Fresh learning and memory use the same verified samples.** MaxRL learns from their binary rewards. Replay stores one exemplar per mode and revisits retained modes with equal weight. In the schematic, fresh samples add C while replay revisits A and B. A stored exemplar can continue to receive a training signal after it becomes rare in fresh samples.

4 EXPERIMENTAL DESIGN

Every comparison below is paired: the two arms share initialization, prompt stream, seed, budget, and evaluation requests, and differ only in the intervention named. Cost is matched where it dominates: the same 3,072 updates and $G = 16$ fresh rollouts, with the control at a zero replay derivative, so replay buys no extra sampling; what it adds is one forward and one backward pass over at most $M = 16$ short banked exemplars per visited prompt (Table 6). We train Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B on all five Level-1 domains for eight passes with five seeds each, adding

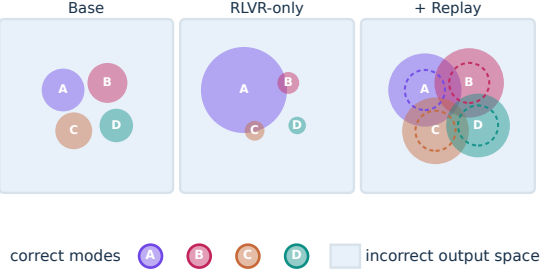


Figure 5: **What the bank preserves.** Verified modes of one prompt, each area the probability the policy produces that mode. RLVR-only training concentrates it on the mode found first; uniform replay keeps every banked mode and leaves room to find more.

replay to Dr.GRPO and crossing Dr.GRPO/MaxRL with replay off and on, then repeat the factorial on Level 2 at 0.5B. Frequency-weighted replay, UCPO, RLEP, and fixed Semantic-MaxEnt are controls for alternative explanations. Evaluation averages four eight-sample draws on 128 held-out prompts that never enter training or the bank (Table 6 gives every shared setting).

We separately evaluate five hosted models — GPT-5.6 Sol, GPT-5.4, Grok 4.3, Kimi K3, and Claude Opus 4.8 — on 480 fixed prompts spanning all five domains and Levels 1–3, eight stateless responses each, graded through a fixed formatting normalizer so that presentation differences are not counted as verified alternatives (App. M). These are inference-only: no replay is applied and no training effect is identified.

5 RESULTS

RLVR narrows a policy onto a single verified answer, and it does so across scales, domains and both objectives (Section 5.1). Adding a memory of distinct successes reverses that narrowing without costing correctness, and the active ingredient is the equal weighting rather than the reuse (Section 5.2). The gain is not an artifact of a weak baseline: it survives a stronger fresh objective (Section 5.3) and a harder construction of every task (Section 5.4).

5.1 RLVR CONCENTRATES VERIFIED OUTPUTS

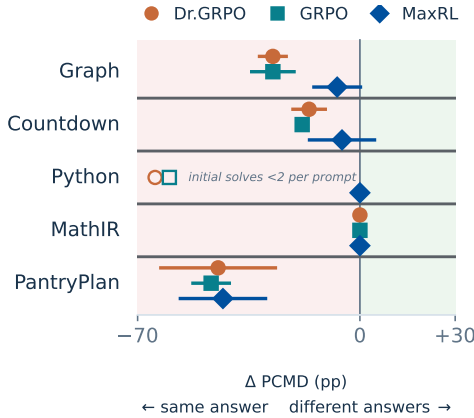


Figure 7: **Training concentrates verified outputs.** Final minus initial PCMD at Qwen2.5-3B under three objectives, five domains; means over five seeds with nominal 95% intervals. Nine of the thirteen measurable comparisons move left, four sit at zero, none moves right. Open marks name an objective PCMD cannot score: the initial Python checkpoint returns two correct responses to no prompt, so those are gaps in the measure, not measured zeros. Other scales: Fig. 11; counts: App. G.1.

are not a weaker version of the same thing. MathIR sits at zero because its correct answers were already concentrated before training, and Python is unscorable because its initial checkpoint never returns two correct responses to the same prompt — a domain can be past the point where narrowing is observable. Where there is breadth to lose, PantryPlan and Graph lose the most. Replay reverses the direction in every matched pair (Fig. 7; Apps. G.1 and G.2).

5.2 REPLAY IMPROVES BOTH OUTCOMES

Verified memory improves both outcomes across scales. Re:Dr raises correctness in every tested model–domain comparison (Fig. 8; per domain and scale in Table 9), and the breadth claim sur-

Training can replace varied attempts with one repeated answer. The starkest case is Python, where every Dr.GRPO seed starts out writing a different program almost every time. Within a few hundred steps all of that is gone: each prompt receives one byte-identical answer, on every test prompt, and it never recovers. The policy has stopped exploring in any observable sense, and the answer it settles on is not always a correct one.

This is also where the fresh objective runs out of signal. A binary-reward group teaches nothing unless it contains both a success and a failure, and by then almost none of them do: under the control, one update in seventy carries any task gradient at all. Replay keeps roughly one in five informative, because a banked exemplar does not have to be rediscovered to be trained on (App. G.3).

Concentration among correct outputs is separately measurable. Collapsing onto one answer is the extreme; the subtler and more general effect is a policy that still succeeds in several ways but increasingly prefers one of them. Across all five domains no comparison moves the other way: nine of the thirteen PCMD can score move toward repetition and four are flat. The flat ones

Table 1: **The alternatives at Qwen2.5-0.5B**. Each method minus matched terminal Dr.GRPO; Avg weights the five domains equally within seed. Bold marks a column’s best, over the trained arms only. A superscript marks a paired seed count below five, an em dash too few verified pairs on one side to difference. Per-seed values and paired intervals stay in the record.

	Δ pass@8						Δ PCMD					
	Grph	Cntd	Pyth	Math	Pntr	Avg	Grph	Cntd	Pyth	Math	Pntr	Avg
Re:Dr (ours)	+.646	+.191	+.356	+.285	+.207	+.337	+.559	+.480	+.082	+.017	+.334	+.294
Re:Max (ours)	+.621	+.185	+.509	+.277	+.199	+.358	+.525	+.504	+.249	+.005	+.317	+.320
Before training	+.132	-.383	-.172	-.292	+.380	-.067	+.108	—	—	+.027	+.591	+.242
MaxRL (no replay)	+.213	+.101	.000	-.066	+.009	+.051	+.086	+.015	.000	.000	.000	+.020
UCPO	-.003	+.133	.000	+.031	+.053	+.043	-.003²	+.036	.000²	-.001	.000²	+.014
RLEP-Dr	+.098	+.103	.000	+.126	.000	+.066	+.164	+.039	.000	-.001	.000	+.040
Semantic-MaxEnt	+.002	+.141	.000	-.121	+.201	+.045	+.011	+.039	.000	+.001	—	+.013
GRPO	+.037	+.159	.000	-.052	+.023	+.034	+.008	+.001	.000	+.001	.000	+.002

vives on the success-conditional axis: replay ends above its control in 14 of the 14 comparisons where both sides define PCMD. The gaps are not marginal — controls finishing at PCMD of zero, every success the same success, against replay arms with a third of their verified pairs differing. Python is where the two axes separate: correctness roughly triples at Qwen2.5-0.5B while PCMD moves $+.082$, and at Falcon3-1B the control never returns two verified solutions to one prompt. Succeeding more often is not succeeding in more ways (App. E.2).

It is the balancing, not merely the reuse. Replay could help simply by training on known-good outputs again. To separate that from the equal weighting, we change one thing: instead of giving every retained mode the same share, we weight modes by how often fresh rollouts rediscovered them. Reuse is identical; only the balance differs. Equal weighting wins on breadth — clearly in Graph, Countdown and PantryPlan, with no aggregate difference in correctness (App. E.6, Fig. 24). Refusing to let popularity decide how much a mode is rehearsed is doing real work.

5.3 REPLAY COMPLEMENTS A STRONGER FRESH OBJECTIVE

Memory adds value when fresh learning already uses MaxRL. The paired comparisons in Fig. 8 separate replay from the choice of fresh objective. At all three scales, replay improves domain-averaged correctness and PCMD under both Dr.GRPO and MaxRL. On breadth every unmodified arm ends below its untrained model and every replay arm above its control, five of the six above untrained as well. Replay’s benefit therefore persists beyond MaxRL’s sampling-aware fresh objective; replay and stronger on-policy learning address complementary parts of the training process.

Replay helps most where the policy has least to lose: tens of points of correctness at the smallest scale, and double digits at every scale. Domain differences survive that average and we report them — Falcon Graph moves the wrong way on raw support, and two of the 3B correctness intervals include zero. Adding the replay update is what carries the gain (Fig. 13, Apps. E.2 and E.4).

Existing objectives that act on the same pressure. Four controls — UCPO, RLEP, a fixed semantic-entropy bonus, and replay weighted by rediscovery frequency — give local rather than general gains (Table 1). The last is the most direct: same bank, same reuse, modes weighted by how often fresh rollouts rediscovered them instead of equally. Equal weighting adds $+.318$ extra modes per prompt over it ($+.272, +.358$) at no cost in correctness, so refusing to let popularity decide rehearsal is the active ingredient, not reuse itself (Apps. E.5 and E.6).

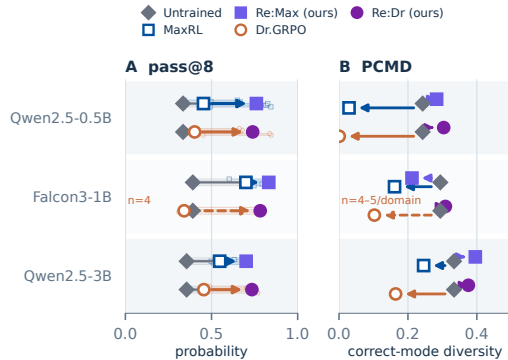


Figure 8: **Replay complements both fresh objectives at multiple scales.** Diamonds are initial models; open/filled markers are training without/with replay. (A) averages all five domains within seed; faint paths show seeds. (B) draws both arrows from the untrained model — one to the fresh objective alone, one to it with memory — over the domains where every arm clears the support bar. Solid arrows are complete five-seed blocks; thinner pairs print their counts. App. Fig. 14 gives each domain at every scale.

5.4 REPLAY

TRANSFERS TO A HARDER CONSTRUCTION

The benefit extends to harder task con-

structions. At Qwen2.5-0.5B, replay improves average correctness and verified-mode diversity under both fresh objectives within both levels, each comparison holding the level’s prompts and fresh objective fixed (Fig. 9).

The retained alternatives are worth having. Withdrawing an ingredient from a held-out Pantry problem after the policy has produced its eight plans, replay-trained policies keep more feasible and recover in fewer attempts, against both tuned-temperature and diversity-prompt controls (App. K.1). Their correctness is higher too: the retained alternatives are the ones a changed problem can use.

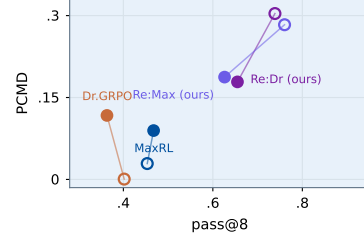


Figure 9: **Replay across difficulty levels at Qwen2.5-0.5B.** Open/filled markers denote Level 1/2; both levels and all four methods share five paired seeds. `pass@8` averages all five domains; `PCMD` averages the two — Graph and MathIR — where every method clears the support bar at both levels, so no arm rests on an easier subset (App. E.3).

6 RELATED WORK

Alignment narrows what a model produces. RLHF and reasoning RL reduce output diversity, entropy and high-budget coverage (Kirk et al., 2024; Dang et al., 2025; Cui et al., 2025; Yue et al., 2025). That literature establishes the cost; our question is what survives it, and whether training can act on it.

What counts as a different answer. Diversity can only be measured once sameness is defined. Semantic clustering infers it from language (Farquhar et al., 2024), which is general but depends on a judge. MODEBENCH takes identity from the same executable check that assigns reward, so two outputs differ exactly when they execute differently — narrower, cheaper, and shared by training and evaluation. Alternatives matter because sampling-based selection consumes proposals (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Lightman et al., 2024).

Broadening support. Outcome bonuses, set prediction, distribution matching and uniform-correct objectives all reshape the fresh update (Song et al., 2025; Chen et al., 2025; Li et al., 2025; 2026a;b; Lochab et al., 2026), and MaxRL combines objectives across sampling budgets (Tajwar et al., 2026); all act on what the policy samples now — the pressure our analysis identifies. Memory sidesteps it, but its weighting decides what survives: replay mitigates forgetting (Lin, 1992; Rolnick et al., 2019) and RLEP reuses successful reasoning trajectories (Zhang et al., 2025), yet both revisit successes in proportion to how they were collected. Balancing modes independently of discovery frequency separates our memory from those and connects it to behavioral archives in quality-diversity methods (Mouret & Clune, 2015; Ecoffet et al., 2021).

7 CONCLUSION AND LIMITATIONS

MODEBENCH makes reliable answering and verified breadth separately observable, and Re:Max keeps discovered successes in play after fresh sampling stops producing them. Four limits remain: replay retains only what sampling finds; mode counts say nothing about worth, and we test usefulness in one place only, a held-out Pantry adaptation; `PCMD` says least where breadth most needs establishing; and the equivalence is executable by construction, so whether these results carry where sameness cannot be executed is untested. Measure and intervention both point one way here, toward more spread; but `PCMD` reads the whole distribution over a known mode set, so the question it makes askable next is whether a *chosen* distribution over the admitted modes can be trained for.

REFERENCES

- Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98):1–76, 2021. URL <https://jmlr.org/papers/v22/19-736.html>.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models, 2025. URL <https://arxiv.org/abs/2508.10751>.
- C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404. URL <https://academic.oup.com/biomet/article-abstract/26/4/404/291538>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025. URL <https://arxiv.org/abs/2505.22617>.
- Xingyu Dang, Christina Baek, J. Zico Kolter, and Aditi Raghunathan. Assessing diversity collapse in reasoning. In *ICLR Workshop on Scaling Self-Improving Foundation Models without Human Supervision*, 2025. URL <https://openreview.net/forum?id=AMiKsHLjQh>.
- Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590:580–586, 2021. doi: 10.1038/s41586-020-03157-9.
- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024. doi: 10.1038/s41586-024-07421-0.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- Josef Hofbauer and Karl Sigmund. *Evolutionary Games and Population Dynamics*. Cambridge University Press, Cambridge, United Kingdom, 1998.
- Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=PXD3FAVHJT>.
- Chenyi Li, Yuan Zhang, Bo Wang, Guoqing Ma, Wei Tang, Haoyang Huang, and Nan Duan. SetPO: Set-level policy optimization for diversity-preserving LLM reasoning, 2026a. URL <https://arxiv.org/abs/2602.01062>.
- Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pp. 110–119, 2016. doi: 10.18653/v1/N16-1014. URL <https://aclanthology.org/N16-1014/>.
- Long Li, Zhijian Zhou, Jiaran Hao, Jason Klein Liu, Yanting Miao, Wei Pang, Xiaoyu Tan, Wei Chu, Zhe Wang, Shirui Pan, Chao Qu, and Yuan Qi. The choice of divergence: A neglected key to mitigating diversity collapse in reinforcement learning with verifiable reward, 2025. URL <https://arxiv.org/abs/2509.07430>.

- Xiaozhe Li, Yang Li, Xinyu Fang, Shengyuan Ding, Peiji Li, Yongkang Chen, Yichuan Ma, Tianyi Lyu, Linyang Li, Dahua Lin, Qipeng Guo, Qingwen Liu, and Kai Chen. Beyond mode collapse: Distribution matching for diverse reasoning, 2026b. URL <https://arxiv.org/abs/2605.19461>.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321, 1992. doi: 10.1007/BF00992699.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2503.20783>.
- Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking RLVR’s indifference to diversity, 2026. URL <https://arxiv.org/abs/2605.00365>.
- Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 6820–6829. PMLR, 2020. URL <https://proceedings.mlr.press/v119/mei20b.html>.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL <https://arxiv.org/abs/1504.04909>.
- Gabriel Pereyra, George Tucker, Jan Chorowski, Łukasz Kaiser, and Geoffrey Hinton. Regularizing neural networks by penalizing confident output distributions. *arXiv preprint arXiv:1701.06548*, 2017. URL <https://arxiv.org/abs/1701.06548>.
- David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://arxiv.org/abs/1811.11682>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Abhijeet Sinha, Sundari Elango, and Dianbo Liu. Expected return causes outcome-level mode collapse in reinforcement learning and how to fix it with inverse probability scaling, 2026. URL <https://arxiv.org/abs/2601.21669>.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Yuda Song, Julia Kempe, and Remi Munos. Outcome-based exploration for LLM reasoning, 2025. URL <https://arxiv.org/abs/2509.06941>.
- Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng, and Andrea Zanette. Maximum likelihood reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.02710v3>. Version 3.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?, 2025. URL <https://arxiv.org/abs/2504.13837>.

Hongzhi Zhang, Jia Fu, Jingyuan Zhang, Kai Fu, Qi Wang, Fuzheng Zhang, and Guorui Zhou. RLEP: Reinforcement learning with experience replay for LLM reasoning, 2025. URL <https://arxiv.org/abs/2507.07451>.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, volume 36, 2023. URL <https://arxiv.org/abs/2306.05685>.

Yaoming Zhu, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu. Texus: A benchmarking platform for text generation models. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pp. 1097–1100, 2018. doi: 10.1145/3209978.3210080. URL <https://arxiv.org/abs/1802.01886>.

APPENDIX ORGANIZATION

The supplement is organized in seven parts, mapped here to their pages.

Part I. Scope and measurement	
A Metric and Objective Details	15
A.1 Measuring success and diversity	15
A.2 Endpoints and the reported breadth axis	16
A.3 Why balance within the replay bank	17
Part II. Benchmark and method	
B Benchmark and Evaluation Protocol	19
B.1 Split identities and executable validation	19
B.2 Python execution contract	19
B.3 Matched Level-2 construction	20
B.4 Frozen base models across the admitted levels	21
B.5 Other model families at matched scale	21
B.6 Training and evaluation settings	21
C Exact Domain Prompts	24
C.1 Graph coloring	24
C.2 Countdown	25
C.3 Python factors	25
C.4 MathIR action menu	25
C.5 PantryPlan	25
D Replay Algorithm	26
D.1 Fresh task advantages	26
D.2 State and optimizer update	26
D.3 Admission, capacity, and deterministic scheduling	27
D.4 Matched controls and recoverable state	27
D.5 Fixed Semantic MaxEnt comparator	27
Part III. Training results	
E Empirical Results and Comparisons	28
E.1 Comparison registry	28
E.2 Terminal comparisons across scales	28
E.3 Terminal comparison on the harder level	31
E.4 Accuracy and verified modes throughout training	33
E.5 Supporting comparators	37
E.6 Uniform versus fresh-frequency replay	40
F Fixed Semantic-Entropy Factorials	42
G Collapse and Replay Mechanism Checks	43
G.1 Conditional concentration from saved training outputs	43
G.2 Breadth before and after RLVR-only training	45
G.3 Fresh-gradient degeneracy and observed output collapse	49
G.4 Fixed-bank exemplar score trajectories	50
Part IV. What breadth represents	
H Matched Prompt-Hint Ablation	52
H.1 Removing strategy hints from unchanged problems	52
I The Decoding Grid Behind the Objection	55
J Sampling-Budget Ablation	56
J.1 Protocol and estimands	56
J.2 Hosted discovery: guidance, formatting, and late modes	66
J.3 All five domains and all three levels at 512 draws	66
J.4 Observed sampling-budget effects	68
K Inference Follow-ups	69
K.1 Pantry portfolios under changed requirements	69
K.2 Complementarity across hosted models	71
K.3 Sensitivity to task-relevant equivalences	72
Part V. Fixed deployments	
L Inference-Only Concentration in a Hosted Model	74
L.1 Hosted concentration on the success-conditional axis	76
M Comparison across Hosted Deployments	78
M.1 A separate Python prompt sensitivity test	78
M.2 A matched requested-temperature sensitivity test	80
M.3 GPT-5.6 Sol: temperature, pass@8 and verified modes	80
M.4 Native provider outcomes and undefined concentration	84
M.5 Matched reasoning-control comparison	95
Part VI. Idealized analysis	
N Mode Collapse and Verified Replay	97
N.1 Categorical abstraction and mean updates	97
N.2 Winner-take-all collapse in the categorical mean flow	99
N.3 A replay signal when fresh groups supply none	100
N.4 Fixed-bank replay and qualitative retention	100
N.5 Complete responses and finite evaluation visibility	102
N.6 Scope relative to entropy, admission, and neural optimization	103
Part VII. Records	
O Source Integrity and Reproducibility	105
O.1 Admissible sources and missing checkpoints	105
O.2 Conflicted initial checkpoints, admitted by provenance	105
O.3 Data, models, and scientific records	106
O.4 Exact continuation and validation	106
P Disclosures	106

NOTATION

Symbols are used consistently across the benchmark, the algorithm and the idealized analysis. Where the same letter carries different meanings in the literature we follow this table.

Table 2: **Notation.** Symbols reused across the benchmark, the algorithm, and the idealized analysis.

Symbol	Meaning
<i>Benchmark and metrics</i>	
$V(y, s_x)$	Validator: a canonical execution key, or $\perp$ for failure
$\mathcal{C}_x$	Set of valid keys for prompt x (never revealed to the learner)
m	Number of correct modes a prompt admits, $m = \mathcal{C}_x $
P	Probability that a sampled response is verified correct
q_c	Conditional probability of key c given correctness
Z_i	Canonical key of the i th draw, or $\perp$
$C(q)$	Conditional collision $\sum_c q_c^2$; the concentration measure
PCMD	Success-conditional breadth $1 - C(q)$; the reported breadth axis
$N_{\text{eff}}^{(2)}$	Effective number of verified modes, $1/(1 - \text{PCMD})$
D_K, F_K	Distinct verified keys, and correct fraction, in K draws
$\bar{D}_K$	Expected distinct verified keys, $\mathbb{E}[D_K] = \text{distinct}@K$
B	Extra verified modes, $\text{distinct}@K - \text{pass}@K$
<i>Algorithm</i>	
$\mathcal{B}_x$	Replay bank for prompt x ; $k = \mathcal{B}_x $ its size, $M = 16$ its capacity
$P_{\mathcal{B}}$	Total probability mass of banked keys
e_c	Stored exemplar realizing key c ; $ e_c $ its token length
G	Fresh rollout group size ($G = 16$ in training)
R_i, R	Binary reward of response i , and group success count $\sum_i R_i$
A_i	Fresh task advantage of response i
σ	Serialized round-robin replay cursor
κ	Clip constant of the Semantic-MaxEnt comparator
<i>Idealized analysis</i>	
$\mathcal{A}$	Category set, correct modes $\mathcal{C}$ and incorrect $\mathcal{N}$
$c_G(P), \Psi$	Mean-flow correctness coefficient and its potential
ρ	Effective replay dose in the mean flow; λ_{replay} its implemented weight
τ	Effective optimization time in the mean flow
C_T	Energy constant in the retention bound

Part I. Scope and measurement

A METRIC AND OBJECTIVE DETAILS

Correctness and solution diversity are distinct properties of a policy. For a prompt x with reference specification s_x , the validator maps a response to either failure or a canonical key:

$$V(\cdot, s_x) : \mathcal{Y} \rightarrow \mathcal{C}_x \cup \{\perp\}.$$

Here $\mathcal{C}_x$ is the set of valid solution keys and $\perp$ denotes failure. The learner observes a key only for a response it generates; neither the full set $\mathcal{C}_x$ nor its size is revealed.

For $Y \sim \pi_\theta(\cdot \mid x)$, the policy induces both a total probability of verified success and a distribution over how that probability is allocated:

$$P_\theta^+(x) = \Pr[V(Y, s_x) \neq \perp], \quad p_\theta^+(c \mid x) = \Pr[V(Y, s_x) = c \mid V(Y, s_x) \neq \perp].$$

The main text writes these two quantities as P and q_c , dropping the policy and the prompt where they are clear. The conditional distribution is defined only when $P_\theta^+(x) > 0$; when $P_\theta^+(x) = 0$, all three sampling metrics below are zero, while PCMD is undefined rather than zero. That is why a measurement resting on too few verified pairs is left blank throughout this supplement rather than filled in (App. A.2). The first quantity measures correctness. The second describes the distribution of verified behaviors conditional on being correct. We call concentration of p_θ^+ onto fewer keys *mode collapse*, particularly when P_θ^+ remains stable or increases. Moving correct probability onto keys that already hold more of it can only lower PCMD = $1 - \sum_c q_c^2$ (Lemma A.1), so the fall in PCMD (2) reported in the main text is exactly the concentration defined here. Neither form claims that a key reached probability zero, which finite samples cannot establish.

A.1 MEASURING SUCCESS AND DIVERSITY

For a prompt whose $K \geq 2$ verified draws are allocated $n_1, \dots, n_m$ across modes, the estimator of (2) is the fraction of verified pairs that differ:

$$\widehat{\text{PCMD}} = 1 - \frac{\sum_m \binom{n_m}{2}}{\binom{K}{2}} = 1 - \frac{\sum_m n_m(n_m - 1)}{K(K - 1)}, \quad (4)$$

which is unbiased for (2) at every $K \geq 2$. Two equivalent readings are sometimes useful: RarefiedDistinct@2 = $1 + \text{PCMD}$ is the expected number of distinct modes in exactly two verified draws, which fixes sampling depth rather than success count; and $N_{\text{eff}}^{(2)} = 1/(1 - \text{PCMD})$ is an effective number of verified modes. Both are monotone transforms of PCMD and order policies identically.

Every printed PCMD is an average of that estimator. A prompt’s draws are pooled across its four groups before scoring — q is a property of the policy and prompt rather than of the draw budget, so pooling estimates the same quantity from more pairs and defines it far more often — and a reported value is the unweighted mean over the prompts where PCMD is defined, carried with its across-prompt standard error. Prompts are weighted equally on purpose: weighting a prompt by how many verified draws it contributed would reintroduce the accuracy dependence PCMD exists to remove.

For K independent samples $Y_{1:K}$, let

$$D_K = |\{V(Y_i, s_x) : i \leq K\} \setminus \{\perp\}|, \quad F_K = K^{-1} \sum_i \mathbf{1}[V(Y_i, s_x) \neq \perp].$$

MODEBENCH reports three sampling averages with different units. $\text{distinct}@K = \mathbb{E}[D_K]$ is an expected count in $[0, K]$ of unique verified keys; $\text{pass}@K = \Pr[D_K > 0]$ and $\text{mean}@K = \mathbb{E}[F_K]$ are probabilities in $[0, 1]$.

Lemma A.1 (Success, sampled breadth, and PCMD). *Fix a prompt, a finite or countable valid-key set, and $K \geq 1$ independent draws from the same fixed sampling law. Let $\mu_c = \Pr[V(Y, s_x) = c]$ and $P = \sum_c \mu_c$. Then*

$$S_K := \text{pass}@K = 1 - (1 - P)^K, \quad \bar{D}_K := \text{distinct}@K = \sum_c [1 - (1 - \mu_c)^K]. \quad (5)$$

For a finite valid-key set of size $m \geq 1$ and $P > 0$, write $q_c = \mu_c/P$. At fixed P , concentration lowers $\bar{D}_K$: if q majorizes q' (its decreasingly sorted partial sums are at least those of q'), then $\bar{D}_K(P, q) \leq \bar{D}_K(P, q')$. In particular,

$$1 - (1 - P)^K \leq \bar{D}_K(P, q) \leq m[1 - (1 - P/m)^K]. \quad (6)$$

For $K \geq 2$, the lower bound is attained only by concentration on one key, and the upper bound only by uniform q . For $K = 1$, $\bar{D}_1 = P$ for every q . At fixed q , increasing P cannot decrease $\bar{D}_K$. By contrast $\text{PCMD}(q) = 1 - \sum_c q_c^2$ does not involve P : it is unchanged when correctness rises or falls at a fixed allocation. Concentration lowers it strictly: if q majorizes q' then $\text{PCMD}(q) \leq \text{PCMD}(q')$, with equality only when q' is a permutation of q . It runs from 0 at a point mass to $1 - 1/m$ at the uniform allocation, so its ceiling is set by the number of valid keys rather than by correctness.

Proof. All K draws fail with probability $(1 - P)^K$. For each key c , its appearance indicator has expectation $1 - (1 - \mu_c)^K$; summing indicators gives the second identity, also for countably many keys by nonnegativity. For fixed $P > 0$ and $K \geq 2$, the function $f(v) = 1 - (1 - Pv)^K$ is strictly concave on $[0, 1]$. Transferring mass from a larger coordinate to a smaller one without reversing their order increases $\sum_c f(q_c)$; successive such transfers give the majorization assertion. The point mass and uniform vector are its extreme cases, giving the bounds and equality conditions. Finally, $\partial \bar{D}_K / \partial P = K \sum_c q_c (1 - Pq_c)^{K-1} \geq 0$, which proves monotonicity in correctness. For the final claim, $\sum_c q_c^2$ is symmetric and convex, so the same transfer strictly decreases it whenever the two coordinates differ; PCMD is one minus that sum and no term in it involves P . The point mass and the uniform vector again give its extremes, by the same two transfers that gave the bounds on $\bar{D}_K$. $\square$

These are promptwise identities, before averaging over prompts. In particular, applying the nonlinear formula to a mean correctness generally changes the result. They explain the shared measurement in the training and hosted studies: success depends on total correct mass; sampled breadth also depends on its allocation.

Raw `distinct@K` measures sampled coverage, not the total number of positive-probability modes. For an uninformed reference, suppose a finite space has N actions and n_c actions map to valid key c . Uniform independent action sampling then gives $\sum_c [1 - (1 - n_c/N)^K]$ expected verified keys. The simpler expression $m_x[1 - (1 - 1/N)^K]$ applies when the m_x valid actions have distinct keys, as in our reported finite-action references. Open-ended program and derivation spaces have no canonical nominal “uniform” generator.

A.2 ENDPOINTS AND THE REPORTED BREADTH AXIS

The terminal co-primary endpoints are `pass@K`, the probability of finding any correct output, and raw `distinct@K`, the number of verified keys found. Both are reported for every model–domain–level block, here and in the result tables of App. E.

A separate question is which statistic should carry the breadth axis in the figures and the prose. Raw `distinct@K` is coupled to the number of successful draws: by (1) it grows with P at fixed q (Lemma A.1), and empirically it correlates .936 with `pass@8` across the 375 untrained measurements, while PCMD correlates $-.332$ on the 225 of them with sufficient support. Reported as breadth it therefore largely restates correctness, and the comparison it invites between a high-`distinct@8` domain and a low-`distinct@8` one is mostly a comparison of accuracy. We consequently report PCMD (2) as the breadth axis, which is a functional of q alone (Lemma A.1). Table 3 shows what that separation buys. From 7B up, Python is answered correctly on at least .988 of held-out prompts at Levels 1–3 while its PCMD never exceeds .044: near-perfect accuracy reached by returning the same answer almost every time. Read on the `distinct@8` axis that same block would look like the broadest domain in the table.

Support and explicit gaps. PCMD is undefined for a prompt returning fewer than two verified responses, and definedness is itself accuracy-dependent: the fraction of prompts defining PCMD correlates .99 with `pass@8`. This is an information limit rather than an artifact, since a policy that almost never succeeds exhibits no distribution of successes to measure. We handle it explicitly rather than silently. Every reported PCMD carries the number of prompts that define it, and measurements resting on fewer than 30 such prompts sit below the support bar. Figures leave

those empty rather than plot a noise-dominated estimate. Table 3 instead prints them in parentheses, so a weak measurement is visible as a number rather than as absence: a parenthesised value is an estimate built on too few prompts that returned two verified responses, and it is not on the same footing as the rest of the table. Of the 375 measurements of frozen base models, 225 clear this bar, and the gaps concentrate where success is rarest: most are Countdown, whose frozen success rate is too low to support any breadth statement. A parenthesised or empty entry means breadth was not reliably measurable there; it does not mean breadth was low.

Table 3: **Success and breadth across 7 frozen Qwen2.5 base models.** `pass@8` left, PCMD right, on the same 128 held-out prompts. Each domain is washed from its own low (red) to its own peak (green), so a row that warms to the right on the left while cooling to the right on the right is a family growing more accurate while answering in fewer ways: PCMD falls with scale in 13 of 20 blocks while `pass@8` rises. A parenthesised PCMD rests on fewer than 30 defining prompts and is noise-dominated; the em dash has none. Rows are levels within a domain and columns are model scales within a panel, and each of the ten washes is scaled independently, one per domain per metric. Countdown is the sparsest domain here: at the smaller scales too few of its prompts return two verified responses for PCMD to be estimated. App. E reports raw `distinct@8` for every entry of this table, under the identical protocol and the same held-out prompt splits.

Qwen2.5

Domain	L	pass@8							PCMD						
		0.5B	1.5B	3B	7B	14B	32B	72B	0.5B	1.5B	3B	7B	14B	32B	72B
Graph	1	.539	.682	.564	.680	.643	.674	.689	.407	.491	.502	.344	.232	.286	.251
	2	.189	.336	.404	.344	.268	.385	.434	(.377)	.562	.390	.404	.173	.196	.263
	3	.361	.551	.545	.521	.488	.484	.562	.385	.465	.441	.375	.196	.220	.233
	4	.387	.492	.441	.490	.473	.537	.637	.489	.475	.402	.447	.184	.247	.306
Countdown	1	.135	.088	.316	.428	.145	.051	.307	(.259)(.133)	.173	.259	(.117)(.036)	.176		
	2	.074	.104	.111	.117	.145	.029	.143	(.271)(.042)(.038)(.048)(.014)(.000)(.005)						
	3	.049	.094	.109	.225	.133	.055	.195	(.567)(.476)(.296)	.232	(.110)(.111)(.190)				
	4	.035	.039	.027	.084	.080	.021	.062	(.500)(.000)(.000)(.107)(.055)(.000)(.000)						
Python	1	.734	.852	.824	1.000	1.000	.996	1.000	.132	.306	.068	.023	.044	.036	.035
	2	.668	.816	.803	.996	1.000	.992	.998	.150	.283	.143	.010	.032	.024	.041
	3	.789	.953	.752	1.000	1.000	.988	.996	.106	.246	.210	.013	.038	.042	.036
	4	.000	.064	.492	1.000	.707	.799	.859	—	(.667)	.023	.032	.066	.240	.114
MathIR	1	.297	.123	.213	.330	.393	.777	.627	.137	(.007)	.086	.020	.108	.147	.108
	2	.174	.113	.268	.414	.559	.973	.852	(.057)(.000)	.129	.043	.051	.179	.186	
	3	.248	.127	.209	.400	.445	.766	.668	.229	(.108)	.046	.028	.104	.090	.150
	4	.137	.148	.088	.221	.430	.904	.799	(.000)(.000)(.000)	.026	.228	.187	.254		
Pantry	1	.229	.262	.264	.225	.412	.443	.398	.389	.643	.482	.329	.465	.379	.361
	2	.273	.275	.254	.271	.506	.545	.568	.306	.624	.534	.260	.430	.425	.400
	3	.273	.264	.180	.191	.295	.203	.203	.448	.472	(.511)(.294)	.347	(.259)	.361	
	4	.412	.357	.312	.195	.365	.303	.250	.568	.661	.469	(.320)	.368	.395	.283

PCMD is not a new measurement of the saved responses. It is exactly one minus the conditional-collision U-statistic $\hat{C} = \sum_m n_m(n_m - 1)/(K(K - 1))$ used in the conditional-concentration analysis; we report the diversity orientation because a metric that increases with breadth reads more directly alongside `pass@8`. Every PCMD value is recomputed from canonical keys already stored with the saved responses, without resampling or regrading.

The derived contrast `distinct@K` - `pass@K` = $\mathbb{E}[D_K - 1[D_K > 0]]$ estimates the number of verified modes beyond the first. It is computable without knowing the full valid set $\mathcal{C}_x$ but remains coupled to the number of successful draws, so we keep it as an interpretive decomposition rather than as a diversity measure. Because every headline evaluation fixes $K = 8$, reporting `distinct@8/8` would add no information beyond a linear rescaling; we retain the raw count and label its units explicitly.

A.3 WHY BALANCE WITHIN THE REPLAY BANK

Uniform sampling over keys prevents discovery frequency from determining replay frequency. To explain its relationship to the evaluation metric we analyze an equal-weight categorical surrogate,

without length normalization (Fig. 5 draws what the two terms preserve); App. D states the objective as implemented. A stored response’s probability need not equal its key’s total probability. For $k = |\mathcal{B}_x|$ banked keys, write $p(c)$ for their probabilities and $P_{\mathcal{B}}$ for their total mass. Then

$$-\frac{1}{k} \sum_{c \in \mathcal{B}_x} \log p(c) = -\log P_{\mathcal{B}} + \log k + \text{KL}(U_k \parallel p(c \mid c \in \mathcal{B}_x)). \quad (7)$$

The objective rewards greater banked verified mass and a more balanced conditional distribution, aligning with `pass@K` and `distinct@K`. The combined update need not improve both at every step: banked-key coverage contributes $\sum_c [1 - (1 - p(c))^K]$ to expected `distinct@K` and, at fixed $P_{\mathcal{B}}$ and k , concavity makes it largest under uniform allocation, while $1 - (1 - P_{\mathcal{B}})^K$, the probability of any banked key, depends only on total mass. Uniformity also maximizes the smallest conditional banked-key mass, making equal retention explicit.

That alignment is deliberate, which is a reason for caution rather than a result: the bank objective raises `distinct@K` by construction, so `distinct@K` cannot also serve as the evidence that replay worked. Held-out `pass@K` is co-primary for that reason, and `PCMD`, which the objective does not target, carries the breadth axis. Banks contain only training-prompt responses; evaluation responses never enter training.

Part II. Benchmark and method

B BENCHMARK AND EVALUATION PROTOCOL

MODEBENCH defines correctness and identity through the same executable validator: invalid outputs receive reward zero and no key; valid outputs receive reward one and a canonical execution key. Neither the complete valid-key catalogue nor its size is supplied to training, replay, scheduling, or stopping. Table 4 specifies the five domains and which surface forms merge. The public dataset preserves each domain’s split roles and construction identity. The training comparisons reported here use Levels 1 and 2.

Table 4: **MODEBENCH**. Correctness and identity come from one executable path; training starts without the mode catalogue. *Valid modes* is the mean number of correct execution modes per evaluation prompt, exact over the released multi-answer split. Every domain trains on 384 prompts and evaluates on a fixed 128-prompt split; the exact prompt each policy receives is reproduced verbatim in App. C for a direct, byte-level audit of the evaluation interface.

Domain	Generated answer / check	Executed key	Aliases that merge	valid modes
Graph coloring	Complete assignment; check fixed colors and edges	Color vector	Spacing only	6.27
Countdown	Parse operands; execute exactly	Normalized AST	Commutative order	4.52
Python factors	Execute restricted lambda externally	Return vector	Source programs with same behavior	229.44
MathIR	Execute bounded action program	State trajectory	Programs with the same states	5.00
PantryPlan	Exact ingredient allocation; check all bounds	Ingredient support	Quantity and ordering aliases	18.19

B.1 SPLIT IDENTITIES AND EXECUTABLE VALIDATION

Every domain uses 384 training and 128 disjoint evaluation prompts. Python materialization independently executes at least two functions with different valid keys for every row; MathIR executes every certified route through the same interpreter used for model outputs.

Table 5: Deterministic materialization identities. Hashes bind the released rows to their executable checks; the full hashes and construction records accompany the data.

	Python factors	MathIR action menu
Training rows	384	384
Evaluation rows	128	128
Exact modes per prompt	16–3,600	5
Training-row SHA-256	bcfa9acc...37cf7	bfc98362...5d3835
Evaluation-row SHA-256	be0f621c...94d183	df5783dd...328b96
Gold catalogue supplied to policy	no	no
Training-bank initialization	empty	empty

PantryPlan. The deterministic v2 release has 384/64/128 training/development/evaluation prompts across four formulation families. Exact support ranges from 8 to 45; the 128 evaluation prompts contain 2,328 feasible supports, averaging 18.1875. Training and evaluation row hashes are a4883c5e...a757f and 5d81ba8d...5923. The validator checks exact inventory, serving steps, mass, nutrient bounds, and dietary tags; identity is ingredient support. A verified allocation is not a claim of palatability, cookability, or safety.

B.2 PYTHON EXECUTION CONTRACT

Candidates are stripped, limited to 240 characters and 64 AST nodes, and parsed in expression mode. The only accepted form is `lambda n: EXPR` with exactly one ordinary argument named `n`: no defaults, positional-only, keyword, or variadic arguments. Integer literals have absolute value at most 10,000. The expression grammar permits `n`, `+`, `-`, `*`, floor division, remainder, integer comparisons, Boolean `and/or`, unary `plus/minus/not`, and conditionals. Boolean and non-integer literals, calls, imports, attributes, comprehensions, containers, sub-

scripts, assignment, loops, and other identifiers are rejected. Task specifications contain two to eight distinct integers in $[4, 1000]$ and at least two valid behavior vectors.

External execution. The trainer never executes candidate code. A locked parent client launches an isolated Python `-I` process in a fresh session, with a persistent JSON-lines worker accepting only restricted source and trusted task JSON. The worker repeats the AST checks, compiles the expression with empty `__builtins__`, and executes the frozen inputs under a 0.25-second real-time alarm; the parent enforces a separate one-second response deadline. A valid response contains the integer vector and key. Timeouts, empty or malformed responses, broken pipes, non-integer outputs, and failed divisor checks yield $\perp$ and zero reward. Broken pipes, operating-system errors, and malformed responses permit one clean worker restart; a timed-out request remains failed. Worker stdout is never parsed as a candidate answer, and exceptions never produce a key.

Behavioral identity. For every frozen input n_j , the returned d_j must be an integer but not a Boolean, satisfy $1 < d_j < n_j$, and divide n_j exactly. All checks must pass. The key is the ordered vector `python_factor`: $d_1, \dots, d_m$; programs merge exactly when their vectors agree. The exact support count $\prod_j |\{d : 1 < d < n_j, d \mid n_j\}|$ is used only for dataset validation.

B.3 MATCHED LEVEL-2 CONSTRUCTION

Level 2 changes problem structure while preserving the verifier and canonicalizer. Its 384/128/128 training/development/evaluation rows have disjoint identities across the Level-1 and Level-2 splits. Training histograms match the native Level-1 training sets. Development and evaluation histograms match designated Level-1 development and confirmation reserves in Graph, Countdown, Python, and MathIR; Pantry uses native references, with its 64-row development histogram doubled to 128 rows.

These construction reserves are not the native Level-1 terminal test sets. The displayed terminal evaluations have different valid-key histograms between levels in Graph, Countdown, and Python. Cross-level changes therefore combine task structure and test population; they do not isolate difficulty at fixed test-set support. Replay comparisons remain paired within each level on the same prompts, seeds, fresh objective, and training budget.

Domain	Level 1	Level 2 structural change
Graph coloring	Three hidden vertices	Four hidden vertices
Countdown	Three operands	Four operands, with values up to 14
Python factors	Four executed cases, all at most 96	Four cases, including a value above 96
MathIR	One-sided linear families	Variable-on-both-sides families
PantryPlan	Base dietary/composition constraints	More active dietary exclusions and tighter composition

Before treatment training, frozen Qwen2.5-0.5B development evaluation admitted a Level-2 domain only when `pass@8` lay in $[.10, .90]$ and was strictly below its construction-reference Level-1 value under identical decoding. The matched-level identity manifest records support histograms, row hashes, construction seeds, disjointness checks, and these admission receipts.

Scale-calibrated Level 3. Level 3 uses frozen Qwen2.5-3B against the same fixed Qwen2.5-0.5B Level-1 reference. Recipes are selected on development problems before fresh held-out confirmation, requiring absolute `pass@1` and `pass@8` differences of at most .04 and .08 in every domain, with four independent groups of eight samples per prompt. `Distinct@8` is reported but is not a fitting objective. The interface uses native chat, temperature 1, top- p 1, and 192 generated tokens: boxed-direct answers for Graph and the hybrid solver with legal-syntax guidance for the other domains. This matches fixed measured targets empirically; it is not a two-sided equivalence test.

The Python Level-3 interface used in the hint ablation removes the strategy hints (App. H), so its difficulty is not covered by that match. All reported results use the original datasets and prompts.

B.4 FROZEN BASE MODELS ACROSS THE ADMITTED LEVELS

Fig. 3 reports the frozen Qwen2.5-Instruct scales across the admitted levels and five domains; Fig. 12 adds the other three families, for 375 completed measurements in all. Every model uses the same 128 held-out prompts within a domain and level, with four independent groups of eight draws per prompt, so each plotted point rests on 4,096 responses. Each group contributes an indicator of any verified output to $\text{pass}@8$ and the number of distinct verified canonical keys to the registered $\text{distinct}@8$; we average groups within prompts and then average prompts. PCMD instead pools a prompt’s four groups before scoring it, because the conditional mode distribution it estimates is a property of the policy and prompt rather than of the eight-draw group. The four groups remain separate $K = 8$ observations throughout this calculation.

These measurements use native chat, $T = 1$, top- $p = 1$, and a 192-token output limit, with boxed-direct Graph answers and the registered hybrid interface elsewhere. Level 3 retains its original admitted prompts. These held-out measurements are separate from development fitting and do not imply that success decreases monotonically with level for every model and domain.

B.5 OTHER MODEL FAMILIES AT MATCHED SCALE

The main text holds the training recipe fixed and reads the scale axis within Qwen2.5. Fig. 12 instead places SmoLM2, Falcon3 and OLMo-2 beside those scales on the same axes, under the identical protocol: the same 128 held-out prompts per domain and level, the same four groups of eight draws, the same verifier and the same support requirement. Parameter count sets colour, so two families at the same size take the same colour and are read against each other directly.

At matched parameter count the families do not agree on breadth. Near 7B the three recipes separate on PCMD while their $\text{pass}@8$ stays close, and the ordering persists into the 10–15B band. Size therefore does not fix how many ways a model answers: a recipe can move PCMD further at constant scale than scale moves it at constant recipe. We report this as a measured property of the released checkpoints, not as a claim about which pretraining or post-training choice is responsible, which these frozen measurements cannot separate.

B.6 TRAINING AND EVALUATION SETTINGS

Table 6: **Matched run contract.** Settings and systems changes are shared by methods within each model–domain–seed comparison, including the zero-weight replay control. Optimizer updates and fresh-rollout budget are therefore identical across arms, and replay buys no additional sampling. What it does add, and what is not matched away, is one forward and one backward pass over at most 16 short banked exemplars on each visited prompt.

Component	Setting
Data	384 training and 128 disjoint evaluation prompts per domain and level; no evaluation row enters training or replay.
Fresh rollouts	$G = 16$, temperature 1, top- $p = 1$; one prompt group per update.
Optimization	Eight passes (3,072 updates), one PPO epoch, $\beta_{KL} = 0$; paired initialization, prompt stream, optimizer, hardware class, and checkpoint cadence.
Evaluation	Passes 0, .5, . . . , 8; four fixed temperature-1, top- $p = 1$, $K = 8$ draws per prompt, plus a separate temperature-0 greedy trace. No best-checkpoint selection.
Replay	Capacity $M = 16$ per prompt; one round-robin bank per update; $\lambda_{\text{replay}} = .10$. Controls execute the same code with an exact-zero replay derivative; realized banks and workloads can differ.
Training seeds	Qwen2.5-0.5B: 43–47; Falcon3-1B: 55–59; Qwen2.5-3B: 70–74.
Learning rate	Qwen-0.5B and Falcon-1B: constant 2×10^{-7} , with optimizer-state offload for Falcon. Qwen-3B: cosine decay with 10^{-7} peak, 10^{-8} floor, and 10% warmup.
Response tokens	Graph and Countdown: 192; Python: 192/512/192 at 0.5B/1B/3B; MathIR: 64/128/64; PantryPlan: 8.
Evaluation seeds	Base 610100/610200/610300/610400/76299 for Graph/Countdown/Python/MathIR/PantryPlan; draw d uses base plus d .



Figure 10: Pairwise correct-mode diversity. Each row corresponds to a model size, and each column to a dataset.

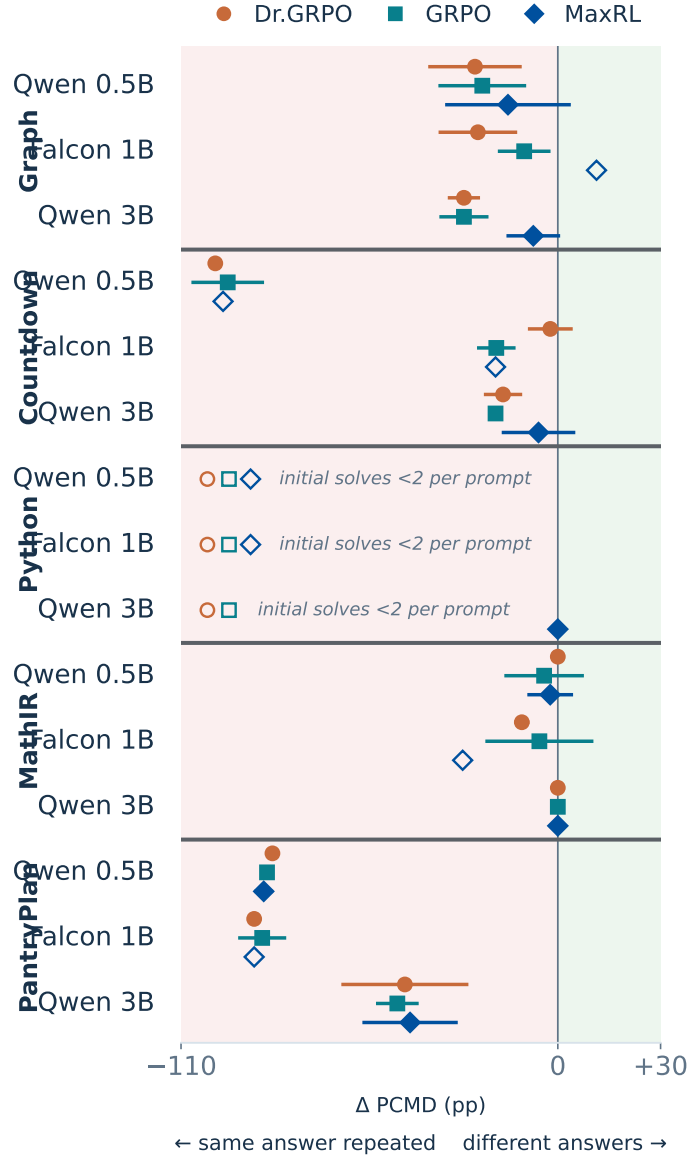


Figure 11: **Concentration at every scale.** Fig. 7 for all three scales rather than Qwen2.5-3B alone: final minus initial PCMD under Dr.GRPO, GRPO and MaxRL, five domains by three scales. Marks are means over five seeds with nominal 95% seed intervals; open marks carry fewer than five seeds and no interval, or name an objective PCMD cannot score because the initial checkpoint returns two correct responses to no prompt. The scale ordering is the same one Fig. 3 uses.

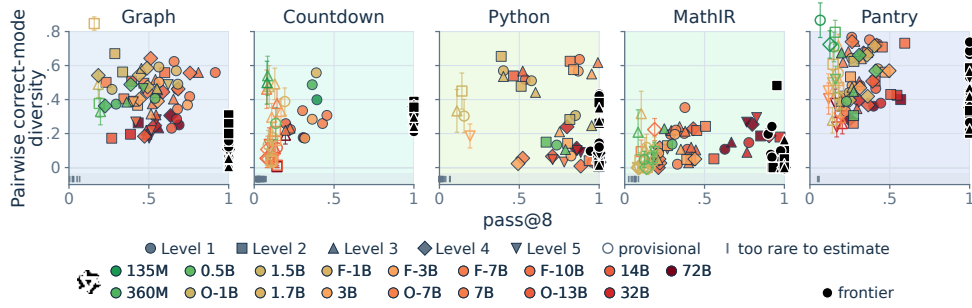


Figure 12: **Four model families on the axes of Fig. 3.** Frozen instruction-tuned models from SmolLM2, Qwen2.5, Falcon3 and OLMo-2, plotted as `pass@8` against PCMD; colour runs green to red with parameter count on a log scale, and marker shape identifies the level. This figure carries every model family and every level collected, where Fig. 3 shows Qwen2.5 alone. Grey markers are all 7 hosted deployments of Section 2.3, of which the main-body figure draws only GPT-5.6 Sol; they have no parameter count and so sit off that ramp. Fig. 10 separates the same data one scale per row. Ticks below each axis mark measurements whose success is too rare for PCMD to be estimated on enough prompts; those are given no height. Because colour encodes size rather than family, models of different families at the same scale share a colour and can be compared directly.

Base model	Pinned revision
Qwen2.5-0.5B-Instruct	7ae557604adf67be50417f59c2c2f167def9a775
Falcon3-1B-Instruct	28ba2251970a01dd1edc7ba7dad2eb71216ccfdcf
Qwen2.5-3B-Instruct	aa8e72537993ba99e69dfaafa59ed015b17504d1

Responses are independent within each evaluation draw. For each training seed, metrics average over all 128 prompts and four draws; raw responses and draw-level records are retained. The co-primary terminal outcomes are `pass@8` and raw `distinct@8`; `mean@8` is diagnostic. Evaluation never supplies training data, bank entries, checkpoint selection, or a stopping rule. Complete paired five-seed blocks receive nominal pointwise 95% Student-*t* intervals; partial primary intersections report exact counts and descriptive means without intervals. Intervals condition on the frozen prompts and evaluation requests. Domain averages are explicitly labeled, and no missing terminal result is replaced by a shallower checkpoint. Source admission and curve gaps follow App. O.1; comparison-specific intersections and the replay-weighting bootstrap are specified with their results.

C EXACT DOMAIN PROMPTS

Each domain uses one system message and one user message, with no few-shot examples. The same template renders training and evaluation for every method. These are the first rows of the frozen evaluation splits, including the exported chat-template wrapper. The checked generator `ops/check_paper_domain_prompts.py` reproduces them from the datasets. A two-space continuation indent marks display wrapping: joining it to the previous line with one space recovers the original string. The PantryPlan listing retains its real newlines.

C.1 GRAPH COLORING

Template `qwen_boxed`; 384 training and 128 evaluation rows in `var/data/graph_coloring_modebench_v2`.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Color this graph with vertices 1 through 6. The undirected edges are: (1,2), (1,3), (3,5),
(3,6), (4,6), (5,6). Assign each vertex one color from {1, 2, 3} so that no edge has the
same color on both endpoints. A partial coloring is 2??1?2, where ? means uncolored.
Keep every shown digit fixed. There are exactly 3 question marks. Return exactly one
final answer and no explanation: exactly 3 digits, each one 1, 2, or 3, for the missing
positions from left to right, inside \boxed{ }.<|im_end|>
<|im_start|>assistant
```

C.2 COUNTDOWN

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/exact_countdown_easy3_probe.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Using the numbers [3, 6, 9], create an arithmetic expression that equals 18. Use each
given number exactly once. You may use +, -, *, /, and parentheses. Return exactly one
final answer and no explanation: the expression inside \boxed{ }.<|im_end|>
<|im_start|>assistant
```

C.3 PYTHON FACTORS

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/python_factor_modebench_v1.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Write one pure Python function with the exact form lambda n: EXPR. An external Python tool
will call it once for each n in [18, 82, 91, 93]. For every call, return an integer d
satisfying 1 < d < n and n % d == 0. Different valid return vectors are different
solution modes. You may use integer literals, n, +, -, *, //, %, comparisons, Boolean
operators, and conditional expressions; calls, imports, attributes, containers, and
other names are forbidden. Return exactly the one-line lambda inside \boxed{} and no
explanation.<|im_end|>
<|im_start|>assistant
```

C.4 MATHIR ACTION MENU

Template qwen_boxed; 384 training and 128 evaluation
rows in var/data/mathir_action_menu_v1.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
MathIR linear-menu-v1. Solve for x by choosing an executable action sequence. Every
selected action is applied exactly to BOTH sides of the current equation, followed by
exact simplification. Bindings: a=2, b=-9, c=-1. Initial equation: x/a + b = c. Action
menu: A means div(a); B means add(b); C means sub(b); D means sub(c); E means
sub(mul(a,b)); F means mul(a). Choose 1-4 action IDs. Illegal operations, repeated
states, and paths that do not finish with x isolated are rejected. Return only the IDs
separated by semicolons inside the answer box; for example formatting only: \boxed{A;C}.
Do not return x's numeric value.<|im_end|>
<|im_start|>assistant
```

C.5 PANTRYPLAN

Template qwen_pantry_support_mask; 384 training and 128 evaluation
rows in var/data/pantry_plan_modebench_v2.

```
<|im_start|>system
Return only six binary digits and no other text. Each digit maps to the corresponding
Pantry row in printed order. A trusted environment will choose exact quantities on
precisely the selected rows.<|im_end|>
<|im_start|>user
Create one feasible high fiber snack from this frozen pantry.
Nutrition values are per 100 g. Quantities are exact integer grams.

Pantry:
- almonds: available=100g, step=25g, minimum-if-used=50g; energy=584kcal, protein=21.5g,
  fiber=10.8g, sodium=0.0mg; tags=tree_nut
- banana: available=125g, step=25g, minimum-if-used=50g; energy=97.0kcal, protein=0.74g,
  fiber=4.62g, sodium=0.0mg; tags=none
- carrots: available=150g, step=25g, minimum-if-used=50g; energy=45.0kcal, protein=0.941g,
  fiber=3.1g, sodium=86.6mg; tags=none
```

```

- grape_tomatoes: available=100g, step=25g, minimum-if-used=50g; energy=27.0kcal,
  protein=0.83g, fiber=2.1g, sodium=6.0mg; tags=none
- navel_orange: available=150g, step=25g, minimum-if-used=50g; energy=47.0kcal,
  protein=0.91g, fiber=2.0g, sodium=9.0mg; tags=none
- sunflower_seeds: available=125g, step=25g, minimum-if-used=50g; energy=571kcal,
  protein=18.9g, fiber=7.22g, sodium=0.0mg; tags=none

Use 2 to 4 ingredients.
Forbidden tags: none
Targets:
- mass_g: minimum 125, maximum 200
- energy_kcal: minimum 239.4, maximum 432.25
- protein_g: minimum 6.423
- fiber_g: minimum 3.478
- sodium_mg: maximum 37.6

Choose the ingredient support as a six-bit mask in the exact Pantry row order. Bit 1
includes that row and bit 0 excludes it.<|im_end|>
<|im_start|>assistant

```

D REPLAY ALGORITHM

Table 2 lists the symbols used here. For each prompt x , $\mathcal{B}_x$ stores at most M verified policy exemplars, one per canonical key; the serialized cursor σ selects banks for replay. The same update supports either fresh objective.

D.1 FRESH TASK ADVANTAGES

For a group of G fresh responses with verified binary rewards r_i and $R = \sum_i r_i$, Dr.GRPO uses $A_i = r_i - R/G$, while MaxRL uses

$$A_i^{\text{MaxRL}} = \begin{cases} Gr_i/R - 1, & R > 0, \\ 0, & R = 0. \end{cases}$$

The centered MaxRL estimator (Tajwar et al., 2026) gives failures advantage -1 in mixed groups and weights each success more strongly when successes are scarce. All-correct and all-failure groups have zero task advantage. Lemma N.3 gives its harmonic pass@ k mean objective. These advantages apply only to fresh responses; bank exemplars enter the separate verified-likelihood loss.

D.2 STATE AND OPTIMIZER UPDATE

Algorithm 1 One Re:Max optimizer update

Require: Policy π_θ ; prompt x and verifier specification s_x ; group size G ; fresh objective $o \in \{\text{MaxRL}, \text{Dr.GRPO}\}$; replay weight λ_{replay} ; states $(\mathcal{B}, \sigma)$

Ensure: Atomically updated $(\theta', \mathcal{B}', \sigma')$

- 1: Draw the on-policy group $\mathcal{G} = \{y_i \sim \pi_\theta(\cdot | x)\}_{i=1}^G$
- 2: **for** $i = 1, \dots, G$ **do**
- 3: Execute $\tilde{c}_i \leftarrow V(y_i, s_x)$ and observe task reward r_i
- 4: Set $c_i \leftarrow \tilde{c}_i$ iff $r_i > 0 \wedge \tilde{c}_i \neq \perp$; otherwise set $c_i \leftarrow \perp$
- 5: **end for**
- 6: Compute fresh task advantages A_i^o from $\{r_j\}_{j=1}^G$; for Re:Max use the formula in App. D.1
- 7: After the full group is validated, deterministically retain one policy exemplar for each new $c_i \neq \perp$ in $\mathcal{B}_x$, up to capacity M
- 8: $(z, \sigma') \leftarrow \text{NEXTBANK}(\{\mathcal{B}_u\}_u, \sigma)$ by a fixed round-robin cursor
- 9: **if** a replay bank $\mathcal{B}_z$ exists **then**
- 10: For each retained exemplar e_c , compute $s_c = |e_c|^{-1} \log \pi_\theta(e_c | z)$
- 11: $L_{\text{replay}} \leftarrow -|\mathcal{B}_z|^{-1} \sum_{c \in \mathcal{B}_z} s_c$
- 12: **else**
- 13: $L_{\text{replay}} \leftarrow 0$
- 14: **end if**
- 15: $w_{\text{rep}} \leftarrow (G - 1)/G^2$ for one replay pseudo-group per G on-policy rows
- 16: $\theta' \leftarrow \text{UPDATE}(\theta, L_{\text{PPO}}(A^o) + w_{\text{rep}} \lambda_{\text{replay}} L_{\text{replay}})$
- 17: Checkpoint $(\theta', \mathcal{B}', \sigma')$ as one atomic state

D.3 ADMISSION, CAPACITY, AND DETERMINISTIC SCHEDULING

A bank is keyed by the SHA-256 digest of the exact prompt-token sequence. Only loss-active rows with positive reward and a nonempty validator key qualify. The full group is validated against the pre-group bank before any new key is committed. For each new key, the lexicographically smallest normalized response-token tuple is retained; new keys are admitted in sorted-key order. Admission is therefore invariant to rollout-row order. The bank keeps one exemplar per key, up to $M = 16$, with no eviction or replacement. Later keys may enter diagnostics but not replay support when capacity is full. Prompt-hash collisions or changed tokens under an existing hash are hard errors.

At each update, the scheduler sorts nonempty prompt-bank hashes and selects one by a fixed round-robin cursor; singleton banks are eligible. The cursor is independent of reward, evaluation, desired support, or entropy, and is serialized with the bank. Stored prompt and response tokens are materialized unchanged into a right-padded teacher-forcing batch. Prompt and padding positions are masked. Each exemplar contributes its mean response-token log probability, averaged uniformly across keys in the selected bank before applying λ_{replay} and the exact $(G - 1)/G^2$ rollout scaling in Algorithm 1. Neither exhaustive support nor evaluation outputs enter this loss.

D.4 MATCHED CONTROLS AND RECOVERABLE STATE

Controls share validation, bank updates, scheduling, materialization, and the score pass, with the replay score-gradient vector replaced by exact zeros. Treatment activates only that derivative. Policies can subsequently differ in samples, bank contents, selected banks, exemplar lengths, and realized computation. Logs retain bank/key counts, replayed token counts, raw and applied losses, score-gradient summaries, cursor phase, and the exact-zero switch. Model and optimizer state, bank counts and exemplar tokens, replay and data cursors, and request-stream state are checkpointed atomically; the loader validates schema, capacity, scheduler and estimator constants, token types, prompt hashes, key/exemplar consistency, and cursor bounds before accepting a resume.

D.5 FIXED SEMANTIC MAXENT COMPARATOR

Fixed Semantic MaxEnt is an entropy-inspired fresh-rollout comparator. For validator-positive canonical outcomes, an empirical predictor combines successful policy history and successful leave-one-out peers. If K keys have total count N , unit pseudocounts and one reserved unseen bucket give

$$\hat{p}(a) = \frac{n(a) + 1}{N + K + 1}, \quad \hat{p}(\text{UNSEEN}) = \frac{1}{N + K + 1}. \quad (8)$$

The added detached advantage on an eligible successful row is

$$A_{i,t}^{\text{sem}} = \eta \frac{\min\{-\log \hat{p}(a_i), \kappa\} - \mathbb{E}_{\hat{p}}[\min\{-\log \hat{p}(a), \kappa\}]}{\kappa}, \quad \eta = .10, \kappa = 5. \quad (9)$$

Invalid and unsuccessful rows receive zero. The centered score is bounded by η in magnitude. It can favor an underrepresented verified outcome present in a fresh rollout, but supplies no direct reinforcement to an absent remembered exemplar. Its heterogeneous empirical effects therefore remain a comparison with exploration rather than a retention guarantee.

For the true conditional distribution $q_\theta = p_\theta(\cdot \mid \mathcal{C})$,

$$\nabla_\theta H(q_\theta) = \mathbb{E}_{a \sim q_\theta} [(-\log q_\theta(a)) \nabla_\theta \log q_\theta(a)]. \quad (10)$$

This motivates the direction of redistribution, not an unbiased-gradient claim for the empirical estimator. Hold a detached predictor score vector s and baseline b fixed independently of the scored action, for example conditional on its leave-one-out peers. Averaging successful rows under the unconditional policy then gives

$$\mathbb{E}_{Z \sim p}[\mathbf{1}\{Z \in \mathcal{C}\}(s_Z - b) \nabla_Z \log p_Z] = P \mathbb{E}_{c \sim q}[s_c \nabla_Z \log q_c] + (\mathbb{E}_q[s] - b) \nabla_Z P.$$

A historical baseline need not equal the current conditional mean, so this bonus can change correctness as well as diversity. Even with the exact unclipped score $s_c = -\log q_c$ and baseline $b = H(q)$, averaging over unconditional rollout rows gives $P \nabla H(q)$. For a single correct key, conditional entropy is zero, yet the implemented unseen bucket can give that key negative semantic advantage. The comparator thus carries neither an exact entropy-gradient interpretation nor an individual-mode preservation guarantee.

Part III. Training results

E EMPIRICAL RESULTS AND COMPARISONS

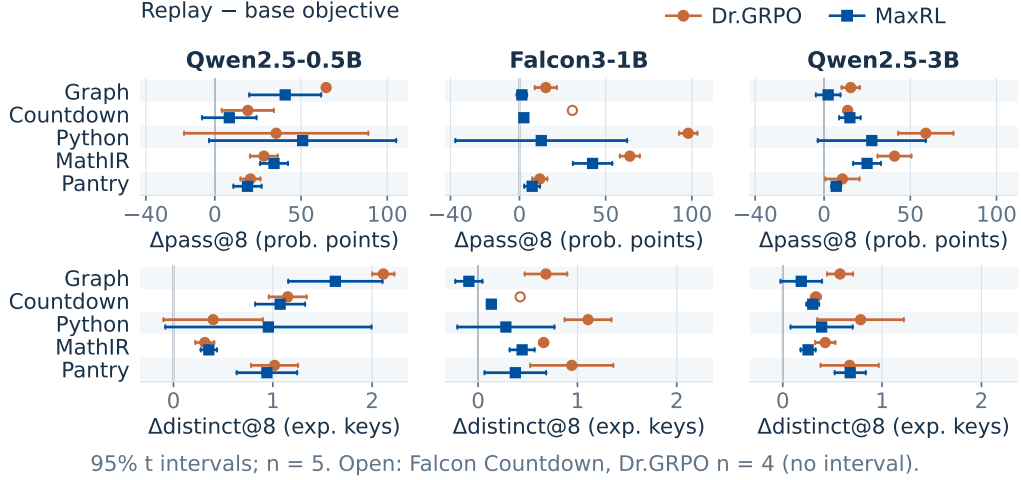


Figure 13: **Canonical replay complements both fresh objectives.** Each point compares replay with its matched no-replay control at pass eight, within domain and scale. The two endpoints have different units: success probability points and expected verified-key counts. All 75 MaxRL pairs are complete; Falcon Countdown uses four Dr.GRPO pairs and all other Dr.GRPO blocks use five. Nominal intervals accompany complete blocks only. Domain variation is retained; these comparisons identify the implemented replay addition rather than a universal benefit or factorial interaction.

E.1 COMPARISON REGISTRY

The primary comparisons cross fresh objective (Dr.GRPO or MaxRL) with verified replay, first across three model scales on Level 1 and then at Qwen2.5-0.5B on Level 2. Table 7 separates these comparisons from the supporting controls and replay-weighting ablation. Terminal results are fixed to a single analysis snapshot; the Qwen2.5-0.5B weighting aggregate uses the same seed population throughout.

Table 7: **Comparison scope and available terminal evidence.** A complete block has five registered paired seeds. Primary partial intersections retain their exact counts and receive no interval. UCPO and sparse RLEP-Dr are reported only at the two smaller model scales; their denominators cover all five benchmark domains and five registered seeds at each selected scale.

Comparison	Scope	Terminal coverage	Question
Dr.GRPO / Re:Dr	Level 1, all three scales	74/75 pairs; 14/15 complete blocks	Verified memory beyond fresh reward
MaxRL / Re:Max	Level 1, all three scales	75/75 pairs; 15/15 complete blocks	Memory beyond a stronger fresh objective
Four-method factorial	Level 2, Qwen-0.5B	5/5 complete domains; 100/100 endpoints	Within-level replay replication
GRPO	Level 1, all three scales	75/75 runs	Reward-standardization control
UCPO	Level 1, 0.5B and 1B	50/50 runs	On-policy advantage redistribution
Sparse RLEP-Dr	Level 1, 0.5B and 1B	47/50 runs	Success memory without uniform key balancing
Fixed Semantic MaxEnt	Level 1, registered scale/domain blocks	10/15 complete blocks	Fresh-rollout semantic exploration
Uniform / frequency replay	Level 1, nine registered blocks	45/45 treatment cells; 9/9 complete blocks	Contribution of replay-key balancing

E.2 TERMINAL COMPARISONS ACROSS SCALES

Within each model–domain–seed comparison, methods share initialization, data identities, the eight-pass horizon, evaluation requests, verifier, and replay scheduling code. The replay control runs the same bank and scoring path with zero replay derivative. Resulting policies can build different banks. The admissible intersections are defined in App. O.1.

Table 8: **Matched results after eight training passes.** Columns compare Dr.GRPO with Re:Dr after exactly eight training passes. Entries are means over the displayed paired-seed intersection; n @pass gives its size and training pass. P, M, and D denote `pass@8`, `mean@8`, and `distinct@8`. Falcon Countdown has $n = 4$ and other blocks $n = 5$ (App. O.1). Pale green marks a Re:Dr mean that is strictly higher than its matched Dr.GRPO mean in the same row.

Model	Domain	n @pass	Dr.GRPO			Re:Dr		
			P	M	D	P	M	D
🚩 Qwen2.5-0.5B	Graph coloring	5@8.0	.323	.323	.325	.969	.404	2.438
	Countdown	5@8.0	.480	.465	.487	.672	.597	1.637
	Python factors	5@8.0	.172	.172	.172	.528	.507	.571
	MathIR	5@8.0	.512	.464	.513	.796	.688	.829
	PantryPlan	5@8.0	.522	.522	.522	.729	.545	1.539
Falcon3-1B	Graph coloring	5@8.0	.656	.300	1.034	.809	.334	1.717
	Countdown	4@8.0	.325	.267	.415	.632	.343	.838
	Python factors	5@8.0	.000	.000	.000	.979	.722	1.107
	MathIR	5@8.0	.095	.068	.097	.736	.292	.756
	PantryPlan	5@8.0	.633	.633	.633	.751	.560	1.576
🚩 Qwen2.5-3B	Graph coloring	5@8.0	.682	.360	1.060	.836	.349	1.637
	Countdown	5@8.0	.609	.569	.628	.746	.505	.962
	Python factors	5@8.0	.069	.069	.069	.659	.603	.852
	MathIR	5@8.0	.278	.168	.279	.686	.580	.706
	PantryPlan	5@8.0	.638	.558	.841	.744	.546	1.514

Table 9: **Replay improves correctness and verified breadth at every scale.** Re:Dr minus matched Dr.GRPO at the terminal checkpoint, per domain and scale behind the cross-domain averages of Fig. 8; read as in Table 1.

	Δ pass@8						Δ PCMD					
	Grph	Cntd	Pyth	Math	Pntr	Avg	Grph	Cntd	Pyth	Math	Pntr	Avg
🚩 Qwen2.5-0.5B	+ .646	+ .191	+ .356	+ .285	+ .207	+ .337	+ .559	+ .480	+ .082	+ .017	+ .334	+ .294
Falcon3-1B	+ .153	+ .307 ⁴	+ .979	+ .640	+ .118	+ .442 ¹	+ .224	+ .032 ¹	—	+ .022 ¹	+ .317	+ .187 ⁴
🚩 Qwen2.5-3B	+ .154	+ .137	+ .590	+ .409	+ .106	+ .279	+ .176	+ .100	+ .046 ²	+ .013	+ .247	+ .126

Dr.GRPO retention contrasts. Both co-primary block means are positive in all 14 complete five-seed comparisons, and raw `distinct@8` increases in all 74 admissible pairs. At Qwen2.5-0.5B, Countdown `pass@8` rises from .480 to .672 and raw `distinct@8` from .487 to 1.637; the extra-mode effect is +.959 (paired 95% interval [+ .863, +1.054]). Python correctness rises from .172 to .528, with an extra-mode effect of +.043 ([−.077, +.164]); MathIR rises from .512 to .796, with extra modes +.030 ([+.001, +.060]). These intervals are nominal, unadjusted Student-*t* estimates over five paired training seeds. Extra modes remain coupled to correctness, so the same Countdown comparison is also read on the success-conditional axis, where Dr.GRPO ends at PCMD .007 and Re:Dr at .487: the replayed policy is not merely succeeding more often, its successes are spread across modes where the control’s are not. Countdown keys identify computation routes, but these held-out endpoints do not identify which previously discovered routes survived training.

Fig. 14 resolves the domain-specific results behind the main cross-domain display. All five 3B MaxRL domain comparisons are complete: both arms have seeds 70–74 at the registered terminal endpoint (50/50 runs). The Dr.GRPO pair uses five seeds except Falcon Countdown, which uses four. These populations remain separate: a factorial interaction requires the common four-arm intersection, not differences of aggregates with different seeds.

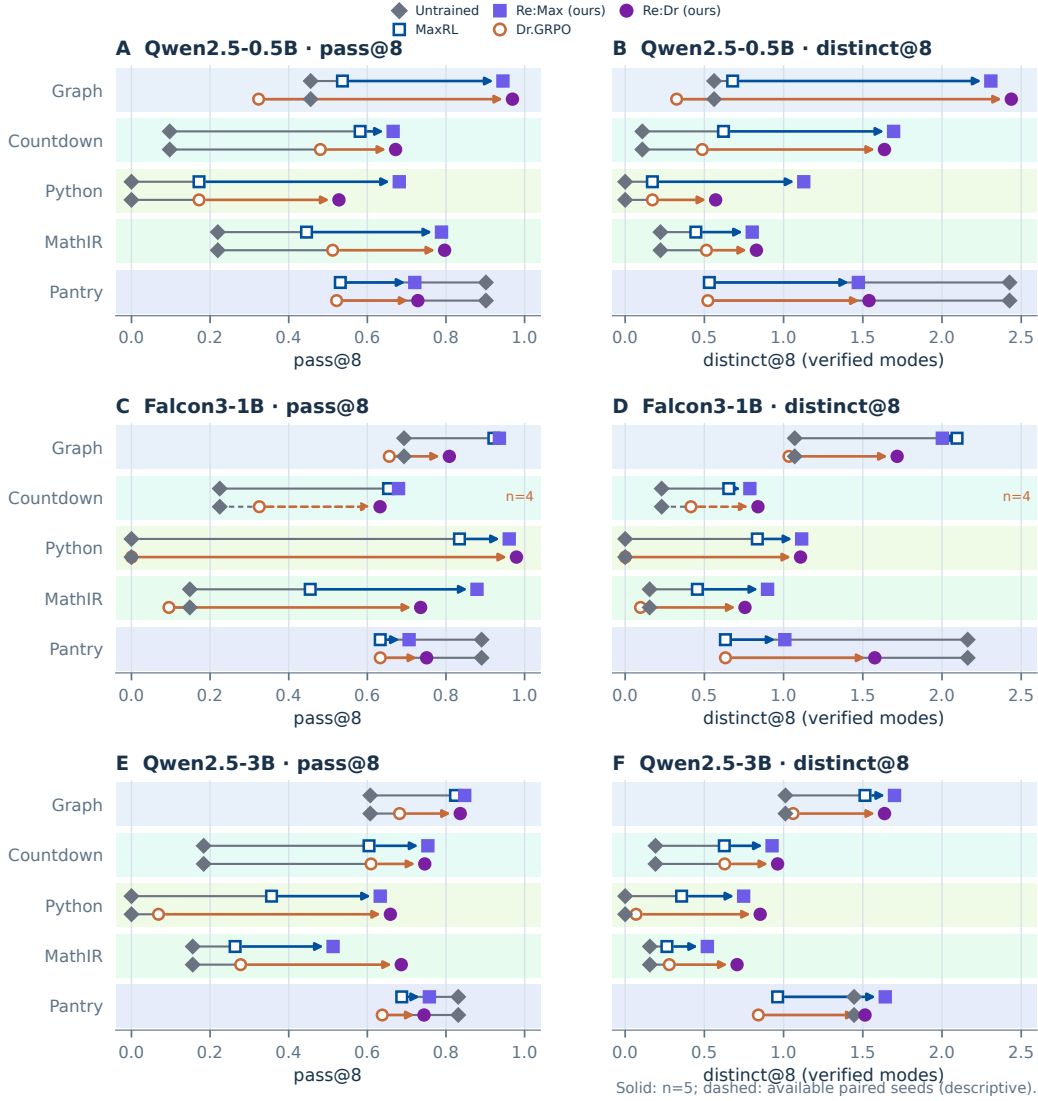


Figure 14: **Per-domain results behind the cross-domain averages in Fig. 8.** Rows show Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns show correctness and raw support. MaxRL–Re:Max pairs retain five terminal seeds throughout the two smaller models and Qwen2.5-3B: every MaxRL domain uses $n = 5$. Solid connectors identify these complete comparisons. Dr.GRPO–Re:Dr uses four admissible seeds for Falcon Countdown and five elsewhere. Domain tints match Fig. 2. Replay arrows, trained-arm markers, and untrained references use the same encoding as Fig. 8. The E118 model index groups the archived exports by model and domain.

Write $P = \text{pass}@8$, $D = \text{distinct}@8$, and $B = D - P$ (Table 2). The derived B counts verified modes beyond the first correct outcome in each sampled set and remains coupled to correctness. The endpoint tables report exact paired terminal effects. A dash denotes no observed pair; partial means retain $n/5$ without an interval. Complete-block intervals are nominal 95% estimates, without multiple-comparison or repeated-look adjustment. All underlying arm means and seed effects remain in the adjacent machine-readable records.

Table 10: **Re:Max effects across model scales at pass 8.** All effects are Re:Max minus MaxRL on the displayed paired seeds. Brackets give paired Student- t intervals for ΔB in complete blocks. The machine-readable report also retains both arm means, all per-seed effects, and intervals for P and D .

Model	Domain	Replay arm	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	Re:Max	5/5	+0.408	+1.630	+1.221 [+0.926, +1.517]
Qwen-0.5B	Countdown	Re:Max	5/5	+0.084	+1.074	+0.990 [+0.859, +1.121]
Qwen-0.5B	Python	Re:Max	5/5	+0.509	+0.955	+0.446 [-0.156, +1.048]
Qwen-0.5B	MathIR	Re:Max	5/5	+0.343	+0.356	+0.013 [+0.008, +0.017]
Qwen-0.5B	PantryPlan	Re:Max	5/5	+0.189	+0.941	+0.751 [+0.520, +0.982]
Falcon-1B	Graph	Re:Max	5/5	+0.014	-0.093	-0.107 [-0.220, +0.005]
Falcon-1B	Countdown	Re:Max	5/5	+0.025	+0.133	+0.108 [+0.082, +0.134]
Falcon-1B	Python	Re:Max	5/5	+0.127	+0.280	+0.153 [-0.272, +0.578]
Falcon-1B	MathIR	Re:Max	5/5	+0.424	+0.443	+0.019 [+0.006, +0.031]
Falcon-1B	PantryPlan	Re:Max	5/5	+0.073	+0.375	+0.302 [+0.034, +0.569]
Qwen-3B	Graph	Re:Max	5/5	+0.023	+0.186	+0.163 [+0.021, +0.305]
Qwen-3B	Countdown	Re:Max	5/5	+0.150	+0.302	+0.152 [+0.119, +0.185]
Qwen-3B	Python	Re:Max	5/5	+0.277	+0.391	+0.115 [+0.080, +0.149]
Qwen-3B	MathIR	Re:Max	5/5	+0.249	+0.255	+0.006 [+0.001, +0.011]
Qwen-3B	PantryPlan	Re:Max	5/5	+0.070	+0.679	+0.609 [+0.467, +0.751]

At Qwen2.5-0.5B, Re:Max adds 1.221, .990, and .751 extra modes on Graph, Countdown, and PantryPlan, respectively; each five-seed interval excludes zero. Falcon Graph instead has a negative mean raw-support effect (-0.093, interval [-0.229, +0.043]), and its correctness effect is inconclusive (+0.014, [-0.016, +0.044]). Positive cross-domain effects therefore do not imply a breadth gain in every domain. All five Qwen2.5-3B blocks are complete. Their extra-mode effects are +0.163, +0.152, +0.115, +0.0059, and +0.609 for Graph, Countdown, Python, MathIR, and PantryPlan; every nominal interval excludes zero, although MathIR’s breadth gain is small. Correctness intervals exclude zero for Countdown, MathIR, and PantryPlan; Graph and Python remain inconclusive. Extra modes are a function of correctness as well as of allocation, so the direction of these effects is confirmed separately on PCMD, where Re:Dr ends above Dr:GRPO in 13 of the 14 comparisons that clear the support bar (App. A.2).

Cross-domain factorial summaries. Averaging five domains equally within each paired seed, Re:Max minus MaxRL gives +0.307 on pass@8 and +0.991 on raw distinct@8 at Qwen2.5-0.5B (unadjusted 95% intervals [+0.208, +0.406] and [+0.815, +1.167]), and +0.133/+0.228 at Falcon3-1B ([+0.043, +0.223] and [+0.094, +0.361]). Both use five paired seeds. The Dr:GRPO replay effects on these two endpoints are +0.337/+0.999 at Qwen2.5-0.5B, +0.442/+0.774 at Falcon3-1B, and +0.279/+0.559 at Qwen2.5-3B. The Falcon Dr:GRPO average uses four common admissible seeds without an interval; the other two use five. These are post-hoc descriptive cross-domain summaries. At Qwen2.5-3B, MaxRL/Re:Max arm means are .547/.701 on correctness and .744/1.107 on raw support. Equal domain averages within each of seeds 70–74 give replay effects of +0.154 ([+0.086, +0.222]) and +0.363 ([+0.285, +0.441]), respectively. The common four-arm intersection contains fourteen complete five-seed blocks and Falcon Countdown at four seeds. Factorial interactions require that intersection rather than differences between aggregates using different seeds.

E.3 TERMINAL COMPARISON ON THE HARDER LEVEL

Fig. 9 uses all five domains, including PantryPlan. Each has all five seeds at step 3,072 for both levels and all four methods. Means average domains equally within each seed, then the five seed means. All 100 Level-2 endpoints are admitted; each within-domain contrast uses seeds 43–47, including all four arms. The levels have separate held-out prompts, with population differences in App. B.3; their comparison does not isolate the effect of task difficulty.

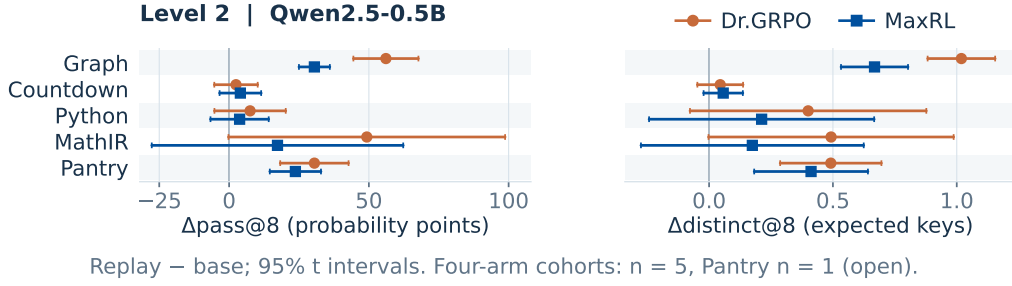


Figure 15: **Replay effects within Level 2.** Replay and control are compared on the same prompts and training seeds, with separate Dr.GRPO and MaxRL effects. All five domains have complete five-seed factorials, including the twenty PantryPlan endpoints. Construction and admission are described in Section 2.2; terminal absolute values appear in Fig. 9. Differences between levels combine task, population, and protocol changes, so they do not identify a causal effect of difficulty alone.

Table 11: **Replay effects on the harder Level-2 tasks at pass 8.** MaxRL and Dr.GRPO identify the fresh objective; each effect is its replay variant minus the corresponding objective without replay. Each row uses its own pairwise intersection, which can exceed the four-method intersection. Brackets give paired Student- t intervals for ΔB only when all five registered pairs are available.

Model	Domain	Objective	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	Dr.GRPO	5/5	+0.561	+1.019	+0.458 [+0.394, +0.522]
Qwen-0.5B	Graph	MaxRL	5/5	+0.305	+0.668	+0.363 [+0.280, +0.446]
Qwen-0.5B	Countdown	Dr.GRPO	5/5	+0.025	+0.045	+0.020 [-0.014, +0.055]
Qwen-0.5B	Countdown	MaxRL	5/5	+0.040	+0.057	+0.017 [+0.001, +0.032]
Qwen-0.5B	Python	Dr.GRPO	5/5	+0.075	+0.400	+0.325 [-0.228, +0.878]
Qwen-0.5B	Python	MaxRL	5/5	+0.037	+0.212	+0.175 [-0.295, +0.645]
Qwen-0.5B	MathIR	Dr.GRPO	5/5	+0.493	+0.493	+0.000 [+0.000, +0.000]
Qwen-0.5B	MathIR	MaxRL	5/5	+0.173	+0.175	+0.002 [-0.003, +0.006]
Qwen-0.5B	PantryPlan	Dr.GRPO	5/5	+0.305	+0.491	+0.186 [+0.103, +0.270]
Qwen-0.5B	PantryPlan	MaxRL	5/5	+0.237	+0.412	+0.175 [+0.028, +0.321]

Level-2 Graph provides the clearest replay effects on correctness and extra modes under both objectives. Countdown’s Re:Max extra-mode effect is small (+0.017, interval [+0.001, +0.032]), while its correctness and raw-support intervals include zero. Python and MathIR have positive mean correctness and raw-support effects with intervals including zero; MathIR’s extra-mode effects are near zero. Pantry’s complete five-seed Dr.GRPO replay contrast is +.305 on P , +.491 on D , and +.186 on B ; its MaxRL replay contrast is +.237, +.412, and +.175, respectively. All six nominal intervals exclude zero. The B intervals are [+0.103, +0.270] and [+0.028, +0.321]; correctness also improves, so these are unconditional breadth effects rather than effects at fixed correctness.

For each common seed and endpoint Y , the replay interaction is

$$I = (Y^{\text{Re:Max}} - Y^{\text{MaxRL}}) - (Y^{\text{Re:Dr}} - Y^{\text{Dr.GRPO}}).$$

Replay can add value under both fresh objectives without a positive interaction. On Level-2 Graph, MaxRL alone raises P relative to Dr.GRPO by +.233 ([+.140, +.325]), while the replay interaction on P is −.256 ([−.329, −.183]). This is an additional replay benefit after stronger fresh optimization, without a superadditive interaction.

Table 12: **Fresh-objective effects and replay interactions on Level 2.** All three contrasts within a domain use the same four-arm seed intersection. The interaction is (Re : Max – MaxRL) – (Re : Dr – Dr.GRPO). Each entry gives its mean and, only for a complete five-seed block, a paired unadjusted 95% Student- t interval. Archived four-arm exports: E119 model index.

Domain	n	Contrast		ΔP	ΔD	ΔB		
Graph	5	MaxRL – Dr.GRPO	+ .233	[+.140, +.325]	+ .322	[+.162, +.482]	+ .089	[+.019, +.159]
Graph	5	Re:Max – Re:Dr	– .023	[–.062, +.016]	– .029	[–.198, +.140]	– .006	[–.142, +.130]
Graph	5	Replay $\times$ MaxRL interaction	– .256	[–.329, –.183]	– .351	[–.446, –.255]	– .095	[–.186, –.003]
Countdown	5	MaxRL – Dr.GRPO	– .014	[–.073, +.045]	– .014	[–.073, +.045]	+ .000	[+ .000, +.000]
Countdown	5	Re:Max – Re:Dr	+ .002	[–.008, +.012]	– .002	[–.047, +.044]	– .004	[–.042, +.035]
Countdown	5	Replay $\times$ MaxRL interaction	+ .016	[–.047, +.078]	+ .012	[–.068, +.092]	– .004	[–.042, +.035]
Python	5	MaxRL – Dr.GRPO	+ .000	[+ .000, +.000]	+ .000	[+ .000, +.000]	+ .000	[+ .000, +.000]
Python	5	Re:Max – Re:Dr	– .037	[–.232, +.157]	– .188	[–.988, +.613]	– .150	[–1.014, +.713]
Python	5	Replay $\times$ MaxRL interaction	– .037	[–.232, +.157]	– .188	[–.988, +.613]	– .150	[–1.014, +.713]
MathIR	5	MaxRL – Dr.GRPO	+ .283	[–.193, +.759]	+ .283	[–.193, +.759]	+ .000	[+ .000, +.000]
MathIR	5	Re:Max – Re:Dr	– .036	[–.158, +.085]	– .035	[–.158, +.088]	+ .002	[–.003, +.006]
MathIR	5	Replay $\times$ MaxRL interaction	– .320	[–.864, +.225]	– .318	[–.860, +.224]	+ .002	[–.003, +.006]
PantryPlan	5	MaxRL – Dr.GRPO	+ .019	[–.067, +.105]	+ .019	[–.081, +.119]	+ .000	[–.021, +.021]
PantryPlan	5	Re:Max – Re:Dr	– .049	[–.113, +.014]	– .061	[–.249, +.128]	– .012	[–.145, +.122]
PantryPlan	5	Replay $\times$ MaxRL interaction	– .068	[–.168, +.032]	– .080	[–.279, +.119]	– .012	[–.134, +.110]

Level-2 Graph endpoint detail. The development admission comparison lowers frozen-model pass@8 from .438 to .148. At pass 8, Dr.GRPO/MaxRL reach .202/.434, versus .763/.739 with their replay variants. The paired correctness gains are +.561 and +.305 (unadjusted 95% Student- t intervals [+ .445, +.677] and [+ .250, +.360], five seeds each). The weakest replay run (.711) exceeds the strongest non-replay run (.477). Replay adds 1.019/.668 raw distinct@8, yielding 1.243/1.214 under the two fresh objectives. Correctness rises steeply here, so that raw gain is partly a correctness gain; PCMD separates the two. On the same block Dr.GRPO ends at .111 and MaxRL at .112, against .346 and .325 for their replay variants — roughly a threefold difference in the chance that two verified colorings differ, at matched conditioning on being correct. These observations show improved correctness and sampled support on harder Graph; they do not establish that broader support causes the correctness gain. Cross-level prompts are distinct, and the Level-2 aggregate covers Graph, Countdown, Python and MathIR.

E.4 ACCURACY AND VERIFIED MODES THROUGHOUT TRAINING

The following curves include every primary method and model scale used in the main paper. Each objective pair uses its domain-specific terminal seed intersection as one fixed cohort throughout training. A checkpoint is plotted only when all members have complete, valid evaluations for both methods. Missing or conflicting checkpoints leave gaps; seed counts never shrink to extend a mean. Four fixed-seed draws of eight samples on the same 128 held-out prompts define each evaluation. Accuracy and mode curves use identical seeds and checkpoints, without smoothing or outcome-based selection.

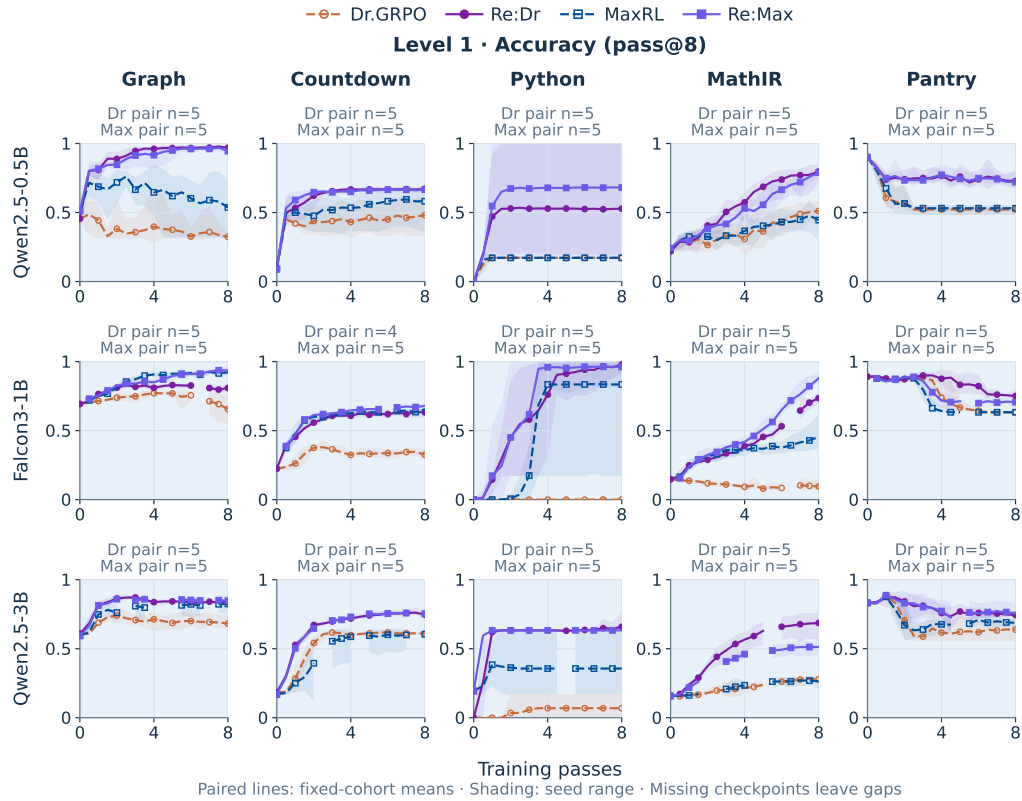


Figure 16: **Accuracy throughout training at all three model scales.** Rows are Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B; columns are the five domains. Dr:GRPO, Re:Dr, MaxRL, and Re:Max use the same method colors as Fig. 8. Lines are fixed-cohort means of pass@8; bands are seed ranges, not confidence intervals. Counts identify each objective’s terminal paired seeds. Qwen2.5-3B MaxRL uses all five seeds in every domain; Falcon Countdown Dr:GRPO/replay uses four. Gaps indicate checkpoints that are missing or whose provenance conflicts, and are left unjoined rather than interpolated across, so a break in a line is an absence of admissible evidence at that step rather than a measured value of zero for that arm at that checkpoint.

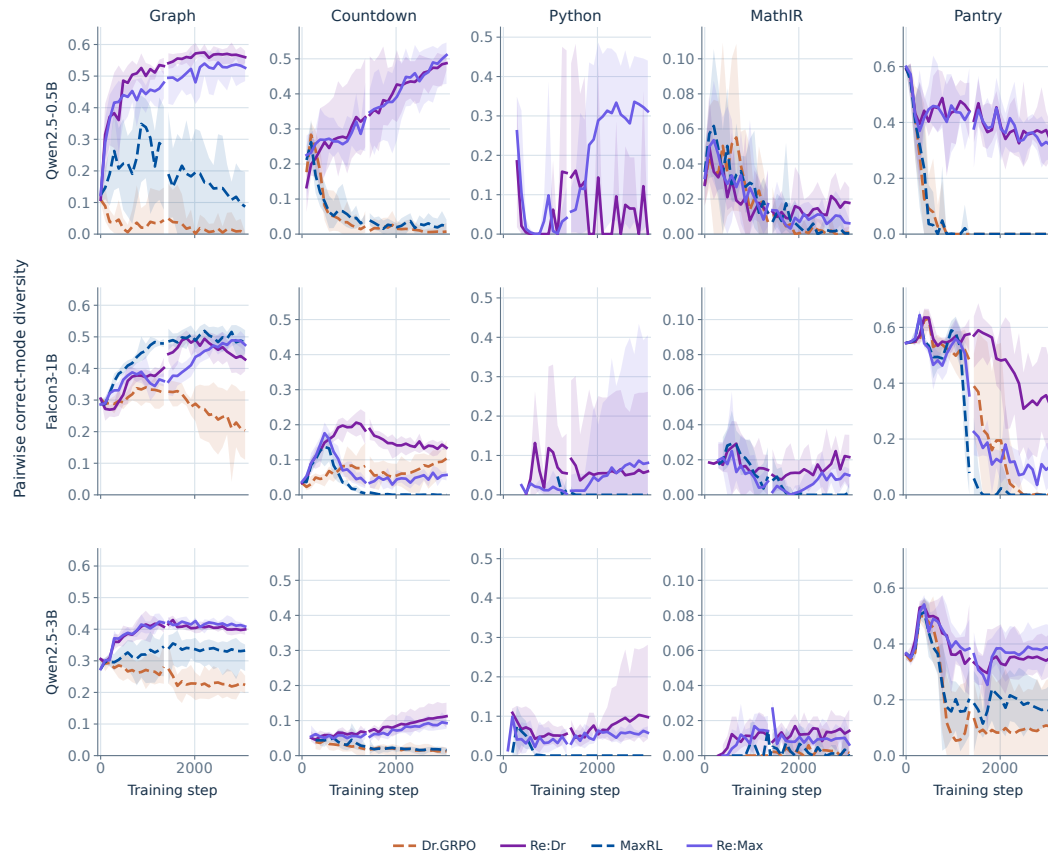


Figure 17: **Verified-mode diversity throughout training at all three model scales.** The same model rows, methods, seeds, and checkpoints as Fig. 16, now measuring PCMD. Because the metric conditions on the verified draws, a falling line reports that successes concentrated, not that they became rarer — the distinction the accuracy curves cannot draw. The two fresh objectives fall in Graph, Countdown, and Pantry at every scale, reaching PCMD .000 in Pantry at 0.5B and 1B, while the replay arms rise or hold. Lines are descriptive fixed-cohort means and bands show seed ranges. A count of one has no band. Domain-specific vertical scales are shared across model rows; absolute heights across different domains should be read against their labeled axes. A step is plotted only where at least 30 prompts return two verified responses, and segments are never joined across a suppressed step, so a gap in a line marks a step whose breadth could not be measured on enough prompts rather than a step at which breadth was measured and found to be zero.

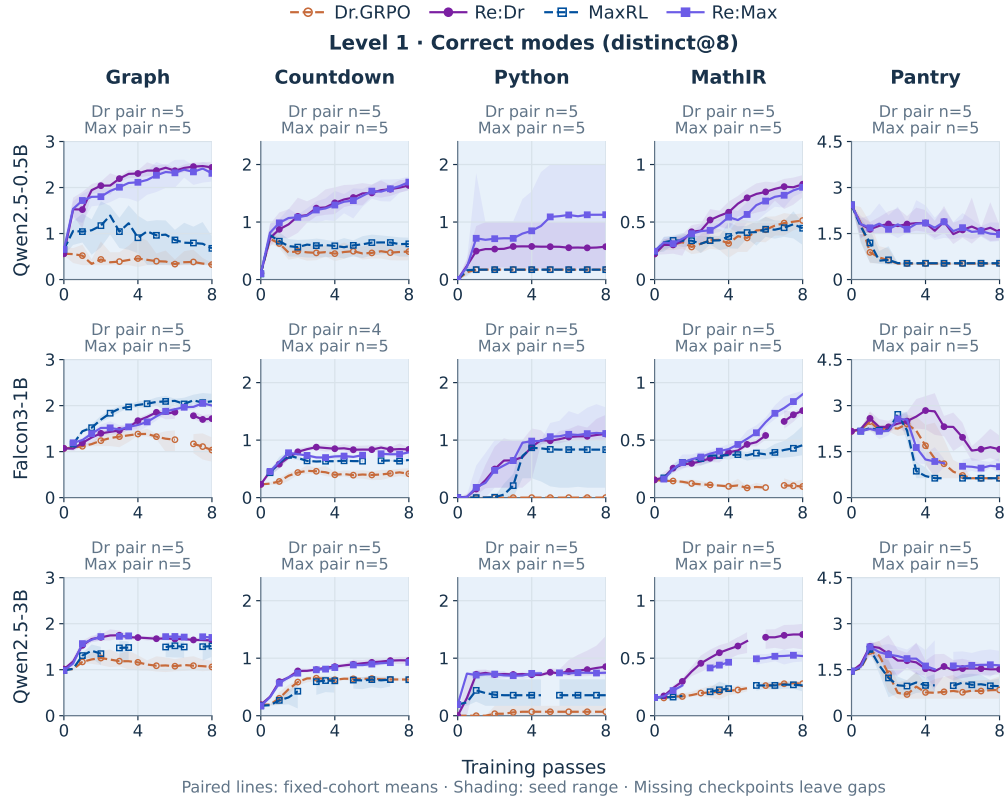


Figure 18: **Registered verified-mode counts throughout training.** The same rows, methods, seeds, and checkpoints as Fig. 17, measuring the registered distinct@8 rather than PCMD. It is retained because distinct@8 is a registered co-primary endpoint. Read against the panel above, it also shows why the success-conditional axis is reported alongside it: curves that rise here can do so because correctness rose, and the two figures separate in exactly the cells where they do. Lines are descriptive fixed-cohort means and bands show seed ranges. A count of one has no band. Missing evaluations are never carried forward or joined across a gap.

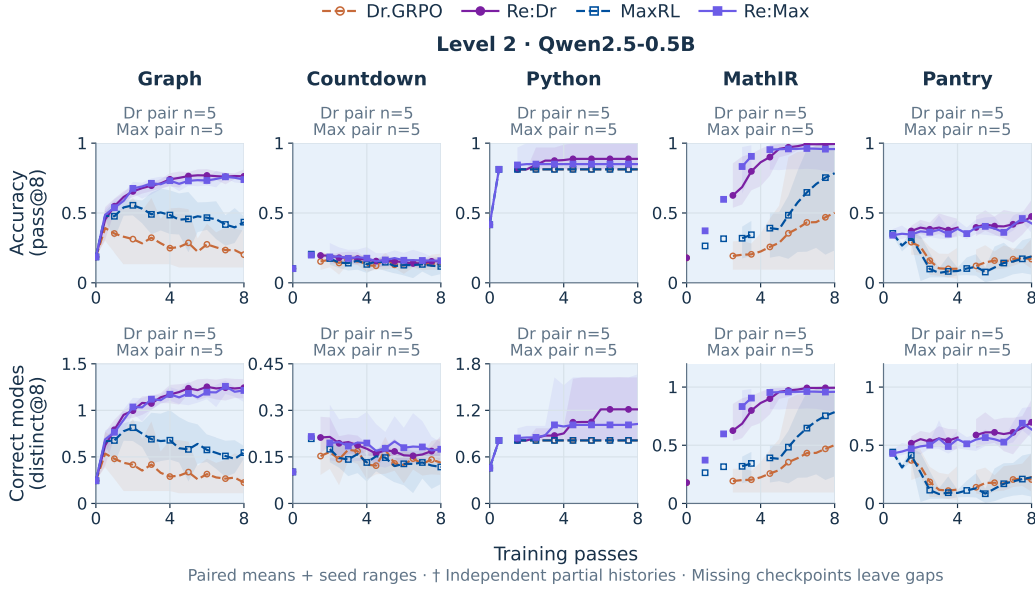


Figure 19: **Accuracy and registered verified-mode counts during Level-2 training.** Qwen2.5-0.5B is the model tested at this level. Rows show `pass@8` and the registered `distinct@8` for all four primary methods; Fig. 20 gives the success-conditional reading. Graph, Countdown, Python, and MathIR contain five terminal paired seeds per objective. PantryPlan uses two terminal Dr.GRPO/replay pairs and one MaxRL/replay pair. Additional unpaired histories are marked separately; they do not contribute to paired means. One-seed curves have no band. Both metrics follow the same evaluation rules, with gaps at missing checkpoints and no extrapolation.

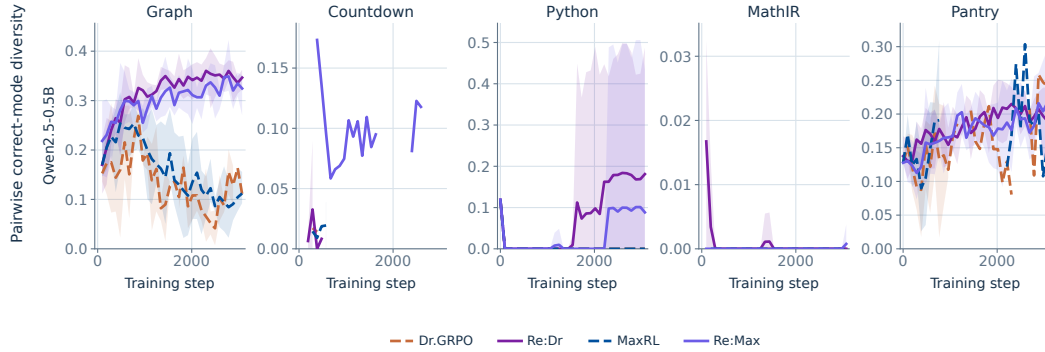


Figure 20: **Verified-mode diversity during Level-2 training.** The same model, methods, seeds, and checkpoints as Fig. 19, measuring PCMD. Graph separates the arms most clearly: both replay arms rise to roughly .34 while Dr.GRPO and MaxRL fall to near .11, so the replayed policies end with about three times the chance that two verified colorings differ. MathIR collapses to PCMD .000 under every arm at this level. Level-2 support is thin in Countdown and Python, where only the steps clearing 30 defined prompts are drawn and segments are not joined across the rest.

E.5 SUPPORTING COMPARATORS

We ran four objectives that act on this same pressure as controls: UCPO’s on-policy advantage redistribution, RLEP’s reuse of successful trajectories, a fixed semantic-entropy bonus, and replay weighted by how often fresh rollouts rediscovered each mode. Their gains are local rather than general — UCPO helps Countdown correctness and extra modes while its MathIR effects are inconclusive, and RLEP raises MathIR correctness with extra modes flat at $-.002$. The weighting control is the most direct of the four, because it changes one thing: keep the same bank and the same reuse, and weight

modes by rediscovery frequency instead of equally. Equal weighting adds $+.318$ extra modes per prompt over it ($+.272, +.358$) with no aggregate correctness difference (App. E.6). Reusing successes is not the active ingredient; refusing to let popularity decide how often each one is rehearsed is.

GRPO changes group-reward normalization; UCPO redistributes fresh on-policy advantage; sparse RLEP-Dr replays verified successes without uniform canonical-key weighting. Fixed Semantic MaxEnt supplies a fresh-rollout exploration incentive (App. D.5). These are controls for alternative explanations of the retention results.

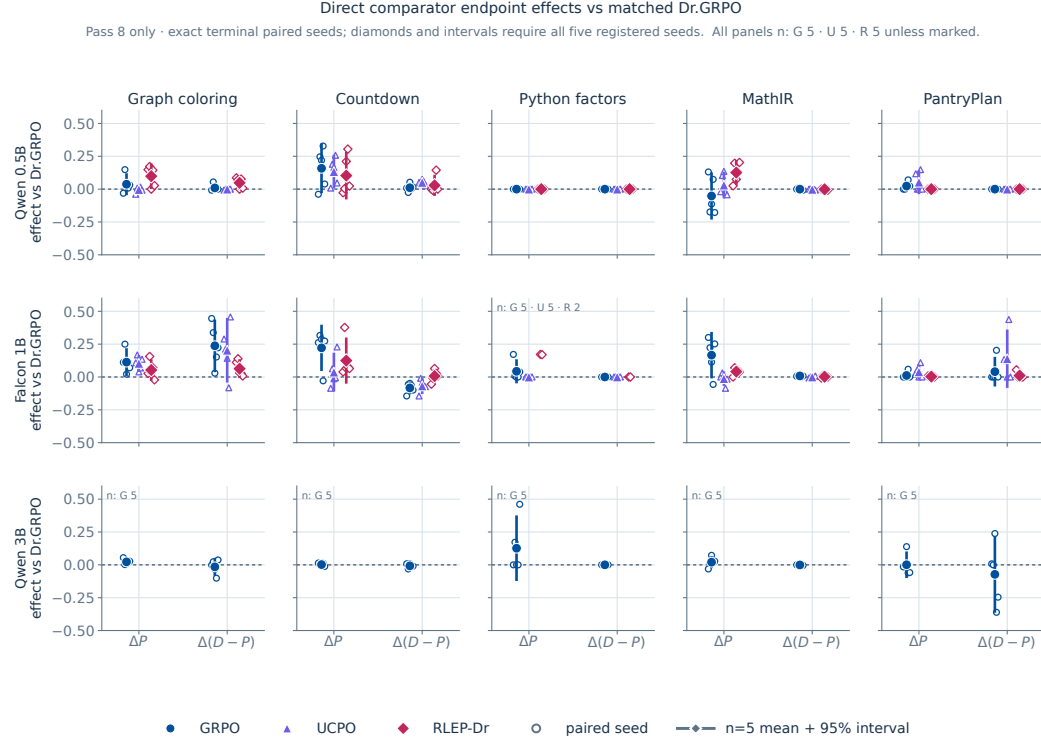


Figure 21: Direct alternatives produce local rather than uniform final gains. Each effect subtracts the same-seed Dr.GRPO control from the comparator after eight passes. Effects are shown for $\text{pass}@8$ and extra verified modes beyond the first success. Circles are paired seeds; diamonds and 95% Student- t intervals appear only for complete five-seed blocks. Exact n is printed, and unsupported comparisons remain blank rather than being extrapolated or imputed at any seed.

The alternatives give local rather than uniform benefits. Qwen-0.5B Countdown UCPO raises P by $+.133$ ($+.005, +.262$) and B by $+.051$ ($+.031, +.071$), whereas its MathIR effects are inconclusive. MathIR RLEP-Dr raises P by $+.126$ ($+.030, +.222$) with neutral extra modes ($-.002, [-.006, +.003]$); its Countdown effects and Falcon Pantry effects are inconclusive. The full five-seed 3B plain-GRPO blocks likewise have intervals including zero on both correctness and extra modes. RLEP’s frozen offline pool was collected at temperature $.7$, top- $p = .95$; all reported evaluations use the common temperature-1, top- $p = 1$ contract. An offline pool versus an online bank remains an algorithmic difference.

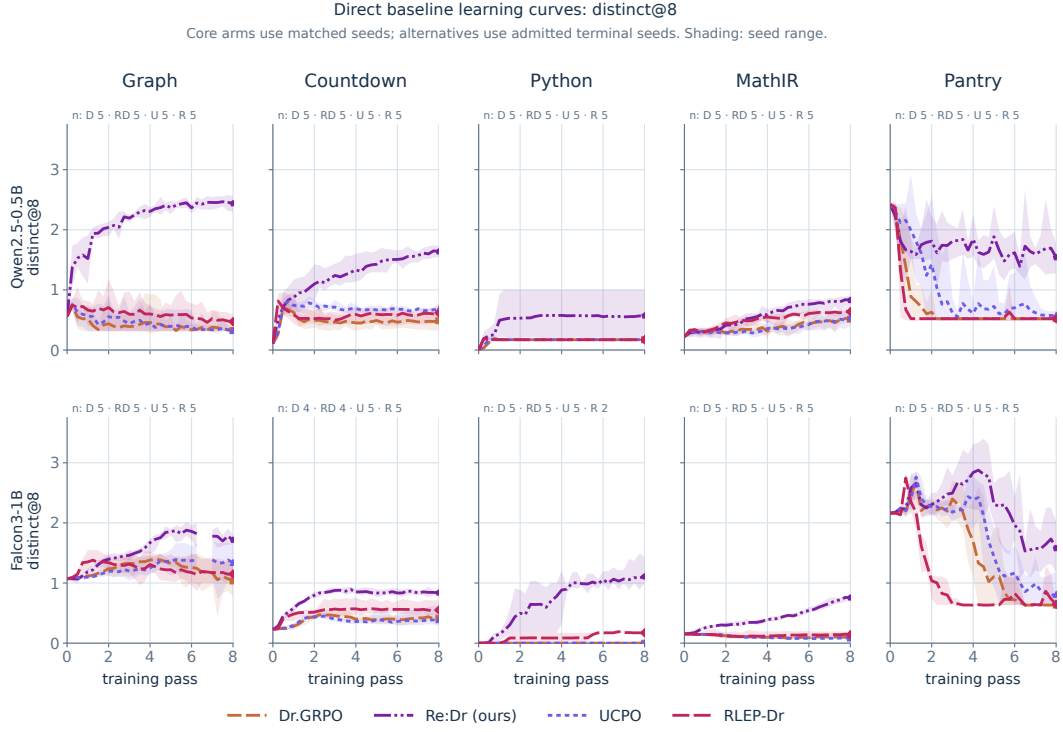


Figure 22: **Verified-mode trajectories for the supporting comparators.** Rows show Qwen2.5-0.5B and Falcon3-1B; columns are the five domains. UCPO and sparse RLEP-Dr are compared with Dr.GRPO and Re:Dr at these two smaller scales. Labels give exact method-specific seed counts. Lines are descriptive registered distinct@8 means and bands are seed ranges; Fig. 17 gives the PCMD reading for the primary methods, and Fig. 23 gives the matching accuracy curves, drawn on exactly the same seed populations and the same registered evaluation checkpoints.

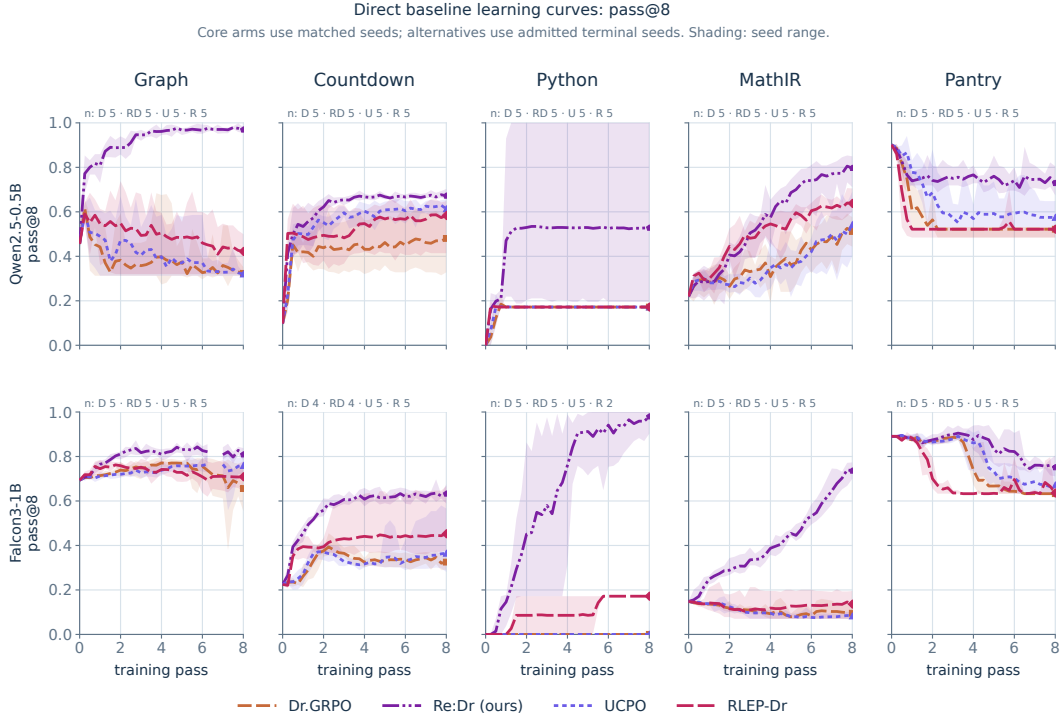


Figure 23: **Accuracy trajectories for the supporting comparators.** The same model rows, domains, methods, terminal seeds, and registered checkpoints as Fig. 22, now measuring `pass@8`. Bands show the seed range and the vertical axes span $[0, 1]$ throughout. MaxRL and Re:Max at all three scales are compared on both metrics in Figs. 16 and 17.

E.6 UNIFORM VERSUS FRESH-FREQUENCY REPLAY

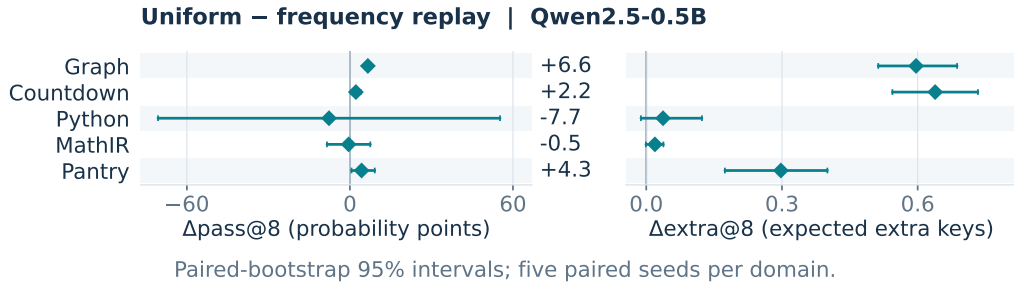


Figure 24: **Equal key weights can improve breadth beyond the first success.** Uniform minus fresh-frequency weighting of verified replay. Success and extra modes are separate endpoints; a positive extra-mode effect does not hold correctness fixed. The displayed model, domains, seeds, and uncertainty use the frozen weighting comparison. The domain variation and inconclusive success contrast limit the claim to the demonstrated breadth contribution.

The weighting ablation retains the same bank construction and update code, changing only the within-bank target: uniform weights over retained keys versus weights proportional to validator-positive fresh-rollout counts. Policy-dependent bank contents can subsequently differ. Positive contrasts below favor uniform weighting. The registered co-primary contrasts are ΔB and ΔP ; the five-domain Qwen2.5-0.5B aggregate first averages domain effects within each matched seed, giving equal domain weight.

Complete-block intervals enumerate all $5^5 = 3,125$ ordered bootstrap resamples of the five paired training seeds and use linearly interpolated 2.5th and 97.5th percentiles. A resampled seed carries its complete domain vector when estimating the five-domain aggregate. The endpoints and aggregate were preregistered; the exact seed-resampling and percentile conventions are analysis

choices. These nominal intervals quantify seed variability within the fixed benchmark, without resampling prompts or domains or adjusting across comparisons.

Table 13: **Uniform versus fresh-frequency replay at pass 8.** Effects are uniform minus frequency weighting. Brackets give the paired percentile bootstrap interval for ΔB over all 5^5 ordered resamples in complete blocks. All nine blocks contain five seed pairs (paired model exports).

Model	Domain	Weighting	$n/5$	ΔP	ΔD	ΔB [95% interval]
Qwen-0.5B	Graph	Uniform	5/5	+0.066	+0.662	+0.596 [+0.513, +0.687]
Qwen-0.5B	Countdown	Uniform	5/5	+0.022	+0.661	+0.639 [+0.544, +0.733]
Qwen-0.5B	Python	Uniform	5/5	-0.077	-0.040	+0.037 [-0.012, +0.123]
Qwen-0.5B	MathIR	Uniform	5/5	-0.005	+0.014	+0.019 [-0.001, +0.038]
Qwen-0.5B	PantryPlan	Uniform	5/5	+0.043	+0.341	+0.298 [+0.174, +0.400]
Falcon-1B	Graph	Uniform	5/5	+0.045	+0.278	+0.233 [+0.182, +0.320]
Falcon-1B	PantryPlan	Uniform	5/5	-0.021	-0.167	-0.146 [-0.564, +0.414]
Qwen-3B	Graph	Uniform	5/5	-0.006	+0.073	+0.079 [+0.011, +0.135]
Qwen-3B	PantryPlan	Uniform	5/5	+0.011	+0.219	+0.207 [+0.038, +0.377]

The registered Qwen2.5-0.5B five-domain effect is +.318 extra modes per prompt ([+.272, +.358]), with positive within-seed domain averages in all five seeds. Graph, Countdown, and PantryPlan drive this result; Python and MathIR remain inconclusive. The correctness effect is +.010 ([-.119, +.139]), establishing neither a gain nor equivalence. Python’s correctness effects range from -.781 to +.828 across seeds despite small extra-mode effects in four seeds.

Falcon Graph also favors uniform weighting on extra modes (+.233, [+0.182, +.320]), but its mean sampled correctness decreases by -.0152 ([-.0214, -.0090]); its P interval includes zero. Falcon Pantry is inconclusive: $\Delta B = -.146$ ([-.564, +.414]) and $\Delta P = -.021$ ([-.063, +.026]). Qwen2.5-3B Graph is complete at five pairs, with an extra-mode effect of +.079 ([+.011, +.135]); its correctness interval includes zero. Pantry also has five pairs, with $\Delta B = +.207$ ([+.038, +.377]) and inconclusive correctness (+.011, [-.020, +.043]). The evidence supports a contribution from key balancing in the stated domains, not a uniform accuracy or breadth advantage across every model-domain block.

Evidence boundary. The primary effects are within-domain, within-scale paired comparisons. They do not establish a scaling law, a positive factorial interaction, or preservation of every previously discovered mode. Every block in the scale factorial, the Level-2 factorial, and the weighting ablation is complete at five paired seeds; the separate Falcon Countdown Dr.GRPO comparison has four seeds and receives no interval.

F FIXED SEMANTIC-ENTROPY FACTORIALS

A fixed semantic-entropy bonus can be applied with or without verified replay. Pantry semantic arms use the registered E85 repairs: original E81/E82/E83 Pantry runs received zero semantic advantage because of an outcome-key defect and remain excluded. The historical Falcon factorial display had not applied that repair; no value from its superseded Pantry column is reused here. The matched four-arm studies compare its effect on Dr.GRPO, its effect on Re:Dr, and the difference of these effects. This restores the full factorial detail beyond the main semantic-only comparator. We reconstruct exact terminal four-draw endpoints under the current source-admission policy: identical repeated metrics are deduplicated, unapproved conflicting retries abort, and the previously excluded Falcon Countdown replay seed 59 remains excluded. All three contrasts in that cell therefore use the common four seeds 55–58; other 0.5B/1B cells use five. No old five-seed Falcon Countdown interval is reused. Tables report paired seed means and pointwise Student-*t* 95% intervals, without pooling models or domains.

The Qwen2.5-3B extension has only fixed seed 70 and no semantic-only arm. Its on-replay comparison is descriptive, has no interval and cannot identify a factorial interaction. These are fixed-coefficient experiments, not a sweep over entropy strengths. Their endpoints do not establish that semantic entropy reproduces the mechanism of a bank that revisits verified keys.

Table 14: Qwen2.5-0.5B: fixed semantic-entropy effects. Without replay is semantic-only minus Dr.GRPO; on replay is semantic-plus-replay minus Re:Dr; interaction subtracts the first effect from the second. Extra modes equal distinct@8 minus pass@8.

Domain	<i>n</i>	Contrast	Δ pass@8	Δ distinct@8	Δ extra modes
Graph	5	Without replay	0.002 [−0.028, 0.031]	0.004 [−0.034, 0.041]	0.002 [−0.010, 0.014]
Graph	5	On replay	−0.002 [−0.027, 0.023]	−0.022 [−0.179, 0.135]	−0.020 [−0.161, 0.120]
Graph	5	Interaction	−0.004 [−0.035, 0.028]	−0.026 [−0.193, 0.141]	−0.022 [−0.161, 0.116]
Countdown	5	Without replay	0.141 [−0.047, 0.330]	0.184 [0.011, 0.358]	0.043 [0.001, 0.085]
Countdown	5	On replay	−0.003 [−0.033, 0.028]	0.115 [0.022, 0.209]	0.118 [0.004, 0.232]
Countdown	5	Interaction	−0.144 [−0.326, 0.037]	−0.069 [−0.233, 0.095]	0.075 [−0.051, 0.201]
Python	5	Without replay	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
Python	5	On replay	−0.002 [−0.717, 0.714]	0.016 [−0.747, 0.780]	0.018 [−0.164, 0.200]
Python	5	Interaction	−0.002 [−0.717, 0.714]	0.016 [−0.747, 0.780]	0.018 [−0.164, 0.200]
MathIR	5	Without replay	−0.121 [−0.349, 0.106]	−0.121 [−0.347, 0.105]	0.001 [−0.006, 0.008]
MathIR	5	On replay	0.031 [−0.035, 0.097]	0.035 [−0.044, 0.114]	0.004 [−0.047, 0.055]
MathIR	5	Interaction	0.153 [−0.018, 0.324]	0.156 [−0.050, 0.362]	0.003 [−0.052, 0.058]
Pantry	5	Without replay	0.201 [0.119, 0.283]	1.120 [0.888, 1.352]	0.919 [0.713, 1.124]
Pantry	5	On replay	0.141 [0.044, 0.238]	1.095 [0.368, 1.823]	0.955 [0.321, 1.589]
Pantry	5	Interaction	−0.061 [−0.161, 0.040]	−0.025 [−0.826, 0.777]	0.036 [−0.682, 0.754]

Table 15: Falcon3-1B: fixed semantic-entropy effects. Without replay is semantic-only minus Dr.GRPO; on replay is semantic-plus-replay minus Re:Dr; interaction subtracts the first effect from the second. Extra modes equal distinct@8 minus pass@8.

Domain	<i>n</i>	Contrast	Δ pass@8	Δ distinct@8	Δ extra modes
Graph	5	Without replay	0.036 [−0.192, 0.264]	0.205 [−0.483, 0.893]	0.169 [−0.292, 0.630]
Graph	5	On replay	0.001 [−0.040, 0.043]	−0.006 [−0.113, 0.101]	−0.007 [−0.091, 0.077]
Graph	5	Interaction	−0.035 [−0.237, 0.168]	−0.211 [−0.886, 0.464]	−0.176 [−0.649, 0.297]
Countdown	4	Without replay	0.004 [−0.092, 0.100]	−0.042 [−0.258, 0.173]	−0.047 [−0.168, 0.074]
Countdown	4	On replay	0.036 [−0.028, 0.101]	0.047 [−0.082, 0.177]	0.011 [−0.064, 0.087]
Countdown	4	Interaction	0.032 [−0.098, 0.161]	0.090 [−0.188, 0.367]	0.058 [−0.099, 0.216]
Python	5	Without replay	0.002 [−0.003, 0.008]	0.002 [−0.003, 0.008]	0.000 [0.000, 0.000]
Python	5	On replay	−0.133 [−0.308, 0.043]	−0.091 [−0.503, 0.321]	0.042 [−0.296, 0.380]
Python	5	Interaction	−0.135 [−0.310, 0.040]	−0.093 [−0.507, 0.321]	0.042 [−0.296, 0.380]
MathIR	5	Without replay	−0.002 [−0.063, 0.060]	−0.002 [−0.063, 0.060]	0.000 [0.000, 0.000]
MathIR	5	On replay	0.086 [0.004, 0.168]	0.108 [0.003, 0.214]	0.022 [−0.009, 0.054]
MathIR	5	Interaction	0.087 [0.029, 0.146]	0.110 [0.043, 0.176]	0.022 [−0.009, 0.054]
Pantry	5	Without replay	0.011 [−0.019, 0.041]	0.091 [−0.161, 0.342]	0.080 [−0.142, 0.301]
Pantry	5	On replay	0.025 [−0.044, 0.094]	0.382 [−0.702, 1.465]	0.356 [−0.670, 1.382]
Pantry	5	Interaction	0.014 [−0.052, 0.081]	0.291 [−0.646, 1.228]	0.277 [−0.615, 1.168]

Table 16: Qwen2.5-3B: fixed semantic-entropy effects. Without replay is semantic-only minus Dr.GRPO; on replay is semantic-plus-replay minus Re:Dr; interaction subtracts the first effect from the second. Extra modes equal distinct@8 minus pass@8.

Domain	<i>n</i>	Contrast	Δ pass@8	Δ distinct@8	Δ extra modes
Graph	1	On replay	−0.016	−0.010	0.006
Countdown	1	On replay	0.039	0.146	0.107
Python	1	On replay	−0.590	−1.215	−0.625
MathIR	1	On replay	0.033	0.010	−0.023
Pantry	1	On replay	0.035	0.393	0.357

results/semantic_current_summary_20260912.json retains per-seed endpoints, source hashes, exclusion evidence and all three metrics. ops/build_paper_semantic_current_summary.py reconstructs these contrasts with the same strict endpoint reader used by the current core comparison, including all documented exclusions.

G COLLAPSE AND REPLAY MECHANISM CHECKS

G.1 CONDITIONAL CONCENTRATION FROM SAVED TRAINING OUTPUTS

This retrospective analysis distinguishes the distribution of correct keys from the probability of producing a correct output. If $P = 0$, C is undefined and $B_2 = 0$. For a fixed prompt with correct mass $P > 0$, write $q_c = \Pr[V(Y, s_x) = c \mid V(Y, s_x) \neq \perp]$ and define

$$C(q) = \sum_c q_c^2, \quad B_2 = 2P - P^2 C(q). \quad (11)$$

The first quantity is the probability that two independent correct outputs share a key. The second identity follows by expanding $B_2 = \sum_c [1 - (1 - Pq_c)^2]$. It makes explicit why sampled breadth can change through correctness or conditional concentration. A decrease in C alone does not imply majorization, larger total support, or recovery of any particular mode. In particular, observed collisions cannot establish that an unobserved key has exactly zero probability under the policy.

Connection to the conditional theory. Under the assumptions of Theorem N.4, the time change in its proof gives $dq_c/d\tau = q_c(q_c - C)$. Consequently,

$$\frac{dC}{d\tau} = 2 \left(\sum_c q_c^3 - C^2 \right) = 2 \text{Var}_{c \sim q}(q_c) \geq 0. \quad (12)$$

The variance vanishes exactly when q is uniform on its positive support. Thus concentration is monotone along this ideal categorical flow, with strict increase whenever the positive coordinates are unequal. The identity provides a theoretical basis for the diagnostic; it does not establish monotonicity for the implemented neural optimizer or a corresponding monotonic decrease under finite-bank replay.

Estimator and identifiable population. For $R \geq 2$ correct samples with per-key counts n_c , use the pair statistic

$$\hat{C} = \frac{\sum_c n_c(n_c - 1)}{R(R - 1)}. \quad (13)$$

For independent samples from one fixed prompt and policy, conditioning on $R = r$ leaves r independent correct labels with distribution q . Every ordered pair therefore agrees with probability $C(q)$, proving $\mathbb{E}[\hat{C} \mid R = r] = C(q)$ for $r \geq 2$. This is a promptwise identity; it does not prove unbiasedness after joint selection using other conditions or dependent prompts. We leave $R < 2$ undefined, rather than assigning a concentration value to insufficient correct observations.

For each paired training seed s , let E_s contain exactly the prompts with a defined estimate in both conditions. The reported seed contrast is $|E_s|^{-1} \sum_{x \in E_s} (\hat{C}_{B,x} - \hat{C}_{A,x})$ when E_s is nonempty. We then weight defined training seeds equally, preserving all registered blocks and their missingness. Before/after contrasts use each method’s own initial checkpoint and step 3,072; replay contrasts compare matched terminal conditions. No other method substitutes for a missing initial evaluation. Five defined seeds receive nominal Student- t intervals; partial blocks are descriptive. These intervals describe the measured seed means and are not simultaneous guarantees or uncertainty over unseen prompts.

The hosted analysis uses the same promptwise collision concept but pools correct pairs. At a common sampling budget, the ratio of expected colliding and correct pair counts is $\sum_x P_x^2 C(q_x) / \sum_x P_x^2$. Its finite-sample ratio is not generally unbiased for that target. The training contrast here weights eligible prompts equally. Changing correctness can change either eligibility or hosted pair weights, so the two aggregate values are not directly interchangeable.

Retrospective protocol and source coverage. The estimator and comparisons were recorded before the initial concentration contrasts, with the sampling-stream amendment also preceding those effects. Two retrospective extensions then followed the paper’s completed source census. The first included thirteen newly available terminal checkpoints and withdrew one conflicted initial checkpoint. The September 12 extension adds all ten remaining Level-2 Pantry terminal checkpoints, reusing 752 unchanged checkpoint identities and changing no estimator or eligibility rule. All prior results, caches, and source snapshots remain preserved. The final collection contains 475 registered runs and 902 available checkpoint evaluations, with 74 Dr.GRPO and 75 MaxRL Level-1 replay pairs and 25 pairs per objective at Level 2 before conditional eligibility. Complete terminal coverage neither supplies missing initial outputs nor makes every conditional contrast identifiable. Exact source hashes and both retrospective extensions’ timing remain bound to the result artifact.

Sampling-stream audit. Four saved eight-output groups do not provide 32 independently seeded responses. The recorded parent seeds increment by one; the audited vLLM V0 sampler increments each child’s seed by its output index. The resulting nominal stream ranges overlap, giving eleven distinct identifiers. For each prompt and checkpoint we retain the earliest draw/output position for every identifier, independently of correctness or key identity. Repeated positions sometimes disagree, so identical seed integers are not treated as a proof of identical outputs. Archived actor source is available for 150 of the 400 factorial cells and none of the 75 additional GRPO cells; historical dependency identity is not cryptographically established for every run. The stream mapping rests on the audited implementation and recorded neutral request contract, with the historical-runtime assumption stated explicitly.

The primary eleven-stream comparisons reuse nominal streams across conditions and remain descriptive after common eligibility. Two additional orientations compare the lower five identifiers in condition A with the upper six in B, then reverse the assignment. Each orientation retains its own eligible prompts and seed counts. Shared identifiers also recur across prompts; neither their separation across conditions nor these selected-population means establishes independent prompt observations. Independent fixed-law sampling is an explicit assumption of the estimator identity, not a conclusion of the source audit.

Original `mean@8`, `pass@8`, and `distinct@8` remain averages over the four intact groups, reconstructed from all saved verified keys. Cross-group overlap affects dependence, not linearity of those marginal averages. The complete result artifact also retains each intact-group collision comparison, a fixed-first-group check, fixed-across-seed prompt intersections, and four-condition change differences. A naive concatenated 32-position collision is retained solely as a reuse diagnostic. These views are not treated as independent replications or substituted for missing cells.

Findings and their limits. In Graph and PantryPlan, all twelve Dr.GRPO/GRPO before/after comparisons across the three scales show increased conditional concentration, with nominal intervals above zero. Correctness also increases on those same eligible prompts. Graph’s increases range from .098 to .274; PantryPlan’s range from .447 to .887. The two disjoint orientations agree in direction in eleven of these twelve blocks; Falcon Graph with GRPO is the exception. The evidence therefore supports observed training-induced concentration in these tasks, with a visible sampling sensitivity, rather than a universal claim about every RLVR run or exact disappearance of modes.

Re:Dr lowers concentration relative to Dr.GRPO in all six Graph and PantryPlan comparisons: changes range from $-.756$ to $-.273$ in Graph and $-.433$ to $-.300$ in PantryPlan. Both disjoint orientations agree in direction in all six, and their primary nominal intervals lie below zero. These comparisons also have lower per-sample correctness on their jointly eligible prompts. They establish a change in the conditional correct-key distribution on that observable population; they do not hold correctness fixed or imply that every accuracy measure improves on the same prompts. Full-test `pass@8` and sampled breadth remain the separately reported primary outcomes of the replay experiment.

The other domains constrain the interpretation. Dr.GRPO/GRPO Python initial outputs provide no jointly eligible prompts at this budget, so their longitudinal collision contrast is unidentified. Some Countdown initial comparisons use only one or two eligible prompts per seed. Several MathIR conditions already have observed collision one, leaving little detectable change. MaxRL comparisons, incomplete saved-output cohorts, and the two sampling orientations are reported explicitly in the following tables; we do not infer a universal concentration benefit from raw breadth gains.

In the completed 3B MaxRL replay cohort, Countdown, Python, and PantryPlan show lower concentration with nominal intervals below zero; MathIR’s measured contrast is zero. Graph’s primary contrast is negative, but its two disjoint orientations disagree in direction, as they do for Falcon Graph with MaxRL. At Level 2, both Graph replay contrasts are negative with agreement across orientations. Complete PantryPlan training still leaves sparse conditional populations: Re:Dr has a defined collision contrast for three seeds (2–16 eligible prompts), with mean $+0.495$; Re:Max has four defined seeds (1–19 eligible prompts), with mean $+0.439$. Their disjoint-orientation means are also positive but retain fewer defined seeds. Neither contrast receives a five-seed interval. Thus the full endpoint breadth gains coexist with increased concentration on these small jointly correct subsets; they do not establish uniformly reduced conditional concentration.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.242	[+0.110, +0.373]	17–40	+0.257 (5)	+0.284 (5)
Graph	GRPO	5	+0.221	[+0.097, +0.344]	15–28	+0.268 (5)	+0.287 (5)
Graph	MaxRL	5	+0.145	[−0.033, +0.324]	18–40	+0.115 (5)	+0.174 (5)
Graph	Re:Dr	5	−0.507	[−0.558, −0.456]	45–49	−0.519 (5)	−0.465 (5)
Graph	Re:Max	5	−0.399	[−0.458, −0.339]	40–51	−0.436 (5)	−0.348 (5)
Countdown	Dr.GRPO	5	+1.000	[+1.000, +1.000]	1–2	− (0)	+1.000 (5)
Countdown	GRPO	5	+0.964	[+0.863, +1.065]	1–2	− (0)	+0.920 (5)
Countdown	MaxRL	4	+0.977	—	0–2	− (0)	+1.000 (4)
Countdown	Re:Dr	5	+0.467	[+0.117, +0.816]	1–3	− (0)	+0.713 (4)
Countdown	Re:Max	5	+0.420	[+0.359, +0.480]	1–2	− (0)	+0.493 (5)
Python	Dr.GRPO	0	—	—	0	− (0)	− (0)
Python	GRPO	0	—	—	0	− (0)	− (0)
Python	MaxRL	0	—	—	0	− (0)	− (0)
Python	Re:Dr	0	—	—	0	− (0)	− (0)
Python	Re:Max	0	—	—	0	− (0)	− (0)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	5–8	+0.000 (5)	+0.000 (1)
MathIR	GRPO	5	+0.040	[−0.071, +0.151]	4–10	+0.000 (5)	+0.250 (2)
MathIR	MaxRL	5	+0.022	[−0.039, +0.084]	5–9	+0.000 (5)	+0.067 (5)
MathIR	Re:Dr	5	+0.059	[−0.046, +0.163]	8–11	+0.000 (5)	+0.367 (2)
MathIR	Re:Max	5	+0.063	[−0.009, +0.134]	9–11	+0.000 (5)	+0.250 (5)
Pantry	Dr.GRPO	5	+0.833	[+0.819, +0.847]	54–64	+1.000 (5)	+1.000 (5)
Pantry	GRPO	5	+0.849	[+0.845, +0.853]	64–65	+1.000 (5)	+1.000 (5)
Pantry	MaxRL	5	+0.859	[+0.836, +0.881]	54–65	+1.000 (5)	+1.000 (5)
Pantry	Re:Dr	5	+0.413	[+0.345, +0.481]	78–89	+0.543 (5)	+0.538 (5)
Pantry	Re:Max	5	+0.446	[+0.324, +0.567]	80–88	+0.593 (5)	+0.550 (5)

Table 17: **Initial to final: Qwen2.5-0.5B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

G.2 BREADTH BEFORE AND AFTER RLVR-ONLY TRAINING

The problem precheck evaluates unmodified Dr.GRPO and GRPO at passes 0 and 8 on all three models and five domains: 150 complete runs, each with four fixed $K = 8$ draws. The five-domain macro-average of extra verified modes falls by 98.2%–99.4% at Qwen2.5-0.5B and 41.5%–71.6% at larger scales. Correctness falls in 10 of the 30 domain–method–scale comparisons, while macro raw distinct@8 ends above its initial value at Qwen2.5-3B. This distinguishes loss of multiple correct outcomes from the broader question of whether any correct outcome is sampled. These are measured changes in the registered runs: they do not establish zero probability for the lost keys, nor a universal decrease in raw breadth across domains.

The same decomposition on the success-conditional axis gives close to the same numbers, which is the check that matters here: extra verified modes could in principle have fallen only because correctness fell. Macro PCMD declines by 98.5% and 99.8% at Qwen2.5-0.5B under GRPO and Dr.GRPO, and by 50.8%–65.1% at the larger scales, against the 98.2%–99.4% and 41.5%–71.6% measured on extra modes. Terminal macro PCMD reaches .000 for Dr.GRPO at Qwen2.5-0.5B. The collapse is therefore a property of how verified probability is allocated, not an artifact of how much of it there is. Falcon3-1B Countdown is the one arm where PCMD rises under Dr.GRPO, from a near-degenerate .033; it does not lift the macro average above zero decline.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.233	[+0.124, +0.343]	41–54	+0.059 (5)	+0.428 (5)
Graph	GRPO	5	+0.098	[+0.026, +0.170]	48–56	-0.186 (5)	+0.273 (5)
Graph	MaxRL	3	-0.113	–	48–52	-0.083 (3)	+0.039 (3)
Graph	Re:Dr	5	-0.007	[-0.065, +0.050]	37–45	+0.048 (5)	+0.045 (5)
Graph	Re:Max	2	-0.062	–	51–52	-0.300 (2)	-0.015 (2)
Countdown	Dr.GRPO	5	+0.022	[-0.038, +0.082]	6–8	+0.000 (5)	– (0)
Countdown	GRPO	5	+0.180	[+0.128, +0.231]	6–11	+0.126 (5)	– (0)
Countdown	MaxRL	2	+0.182	–	11	+0.143 (2)	– (0)
Countdown	Re:Dr	4	+0.035	–	10–11	+0.158 (4)	– (0)
Countdown	Re:Max	1	+0.135	–	11	+0.143 (1)	– (0)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	0	–	–	0	– (0)	– (0)
Python	Re:Dr	0	–	–	0	– (0)	– (0)
Python	Re:Max	0	–	–	–	– (0)	– (0)
MathIR	Dr.GRPO	5	+0.105	[+0.088, +0.122]	6	– (0)	+0.098 (5)
MathIR	GRPO	5	+0.054	[-0.099, +0.206]	3–7	– (0)	+0.233 (5)
MathIR	MaxRL	1	+0.278	–	6	+1.000 (1)	+0.133 (1)
MathIR	Re:Dr	5	+0.086	[+0.026, +0.146]	6–7	– (0)	+0.111 (5)
MathIR	Re:Max	2	+0.167	–	5–6	+1.000 (2)	+0.000 (2)
Pantry	Dr.GRPO	5	+0.886	[+0.886, +0.886]	76	+1.000 (5)	+1.000 (5)
Pantry	GRPO	5	+0.863	[+0.798, +0.928]	76	+0.945 (5)	+1.000 (5)
Pantry	MaxRL	1	+0.886	–	76	+1.000 (1)	+1.000 (1)
Pantry	Re:Dr	4	+0.437	–	75–83	+0.451 (4)	+0.618 (4)
Pantry	Re:Max	2	+0.746	–	76–78	+0.740 (2)	+0.978 (2)

Table 18: **Initial to final: Falcon3-1B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.274	[+0.232, +0.316]	44–47	+0.030 (5)	+0.339 (5)
Graph	GRPO	5	+0.274	[+0.207, +0.341]	46–49	+0.046 (5)	+0.364 (5)
Graph	MaxRL	5	+0.072	[-0.002, +0.145]	45–55	-0.208 (5)	+0.215 (5)
Graph	Re:Dr	5	-0.038	[-0.060, -0.016]	48–53	-0.227 (5)	+0.121 (5)
Graph	Re:Max	5	-0.145	[-0.198, -0.093]	46–52	-0.220 (5)	+0.165 (5)
Countdown	Dr.GRPO	5	+0.160	[+0.109, +0.211]	11–12	+0.200 (5)	-0.016 (5)
Countdown	GRPO	5	+0.182	[+0.182, +0.182]	11	+0.200 (5)	+0.000 (5)
Countdown	MaxRL	5	+0.056	[-0.046, +0.159]	9–11	+0.050 (5)	+0.000 (5)
Countdown	Re:Dr	5	+0.020	[-0.060, +0.099]	11–15	+0.169 (5)	-0.071 (5)
Countdown	Re:Max	5	-0.100	[-0.145, -0.055]	10–12	+0.000 (5)	-0.050 (5)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	5	+0.000	[+0.000, +0.000]	2–9	+0.000 (2)	+0.000 (5)
Python	Re:Dr	0	–	–	0	– (0)	– (0)
Python	Re:Max	5	-0.023	[-0.050, +0.003]	9	+0.000 (5)	-0.040 (5)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	11–14	+0.000 (5)	+0.000 (5)
MathIR	GRPO	5	+0.000	[+0.000, +0.000]	9–11	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	+0.000	[+0.000, +0.000]	7–14	+0.000 (5)	+0.000 (5)
MathIR	Re:Dr	4	-0.012	–	14	+0.000 (4)	+0.000 (4)
MathIR	Re:Max	5	+0.000	[+0.000, +0.000]	12–15	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	+0.447	[+0.266, +0.627]	49–67	+0.478 (5)	+0.245 (5)
Pantry	GRPO	5	+0.469	[+0.411, +0.526]	49–62	+0.514 (5)	+0.271 (5)
Pantry	MaxRL	5	+0.431	[+0.297, +0.565]	59–69	+0.394 (5)	+0.246 (5)
Pantry	Re:Dr	5	+0.184	[+0.081, +0.288]	64–71	+0.087 (5)	+0.037 (5)
Pantry	Re:Max	5	+0.142	[+0.018, +0.266]	68–75	+0.046 (5)	-0.015 (5)

Table 19: **Initial to final: Qwen2.5-3B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	+0.625	[+0.295, +0.955]	2–10	+0.633 (2)	+0.860 (5)
Graph	MaxRL	5	+0.452	[+0.387, +0.518]	6–10	+0.767 (4)	+0.484 (5)
Graph	Re:Dr	5	-0.204	[-0.282, -0.126]	9–11	+0.033 (5)	-0.372 (5)
Graph	Re:Max	5	-0.128	[-0.262, +0.006]	9–11	+0.333 (5)	-0.267 (5)
Countdown	Dr.GRPO	1	+0.000	–	0–1	– (0)	– (0)
Countdown	MaxRL	1	+0.000	–	0–1	– (0)	– (0)
Countdown	Re:Dr	5	+0.000	[+0.000, +0.000]	1	– (0)	– (0)
Countdown	Re:Max	3	+0.000	–	0–1	– (0)	– (0)
Python	Dr.GRPO	5	+0.302	[+0.302, +0.302]	32	+0.000 (5)	+0.378 (5)
Python	MaxRL	5	+0.302	[+0.302, +0.302]	32	+0.000 (5)	+0.378 (5)
Python	Re:Dr	5	+0.094	[-0.259, +0.448]	32	-0.224 (5)	+0.146 (5)
Python	Re:Max	5	+0.199	[-0.088, +0.485]	32	-0.070 (5)	+0.258 (5)
MathIR	Dr.GRPO	3	+0.000	–	0–5	– (0)	+0.000 (3)
MathIR	MaxRL	4	+0.000	–	0–6	– (0)	+0.000 (4)
MathIR	Re:Dr	5	+0.000	[+0.000, +0.000]	6	– (0)	+0.000 (5)
MathIR	Re:Max	4	+0.000	–	6	– (0)	+0.000 (4)
Pantry	Dr.GRPO	3	-0.578	–	0–3	-0.500 (1)	-1.000 (1)
Pantry	MaxRL	3	-0.238	–	1–5	-1.000 (1)	-1.000 (1)
Pantry	Re:Dr	2	-0.049	–	15–16	-0.075 (2)	-0.050 (2)
Pantry	Re:Max	3	+0.020	–	18–20	+0.092 (3)	-0.174 (3)

Table 20: **Initial to final: Qwen2.5-0.5B, Level 2.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.756	[-0.831, -0.680]	37–40	-0.853 (5)	-0.807 (5)
Graph	MaxRL	5	-0.587	[-0.759, -0.414]	41–73	-0.625 (5)	-0.605 (5)
Countdown	Dr.GRPO	5	-0.529	[-0.605, -0.452]	42–78	-0.529 (5)	-0.535 (5)
Countdown	MaxRL	5	-0.547	[-0.635, -0.459]	45–84	-0.578 (5)	-0.530 (5)
Python	Dr.GRPO	5	-0.090	[-0.341, +0.160]	22	-0.115 (5)	-0.062 (5)
Python	MaxRL	5	-0.266	[-0.577, +0.044]	22	-0.298 (5)	-0.272 (5)
MathIR	Dr.GRPO	5	-0.008	[-0.017, +0.001]	44–68	-0.006 (5)	-0.008 (5)
MathIR	MaxRL	5	-0.005	[-0.013, +0.003]	32–56	-0.007 (5)	-0.002 (5)
Pantry	Dr.GRPO	5	-0.433	[-0.518, -0.347]	54–66	-0.442 (5)	-0.494 (5)
Pantry	MaxRL	5	-0.417	[-0.547, -0.288]	53–71	-0.363 (5)	-0.508 (5)

Table 21: **Replay minus control at the final checkpoint: Qwen2.5-0.5B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.309	[-0.523, -0.094]	31–64	-0.129 (5)	-0.452 (5)
Graph	MaxRL	5	-0.021	[-0.040, -0.002]	95–100	+0.100 (5)	-0.013 (5)
Countdown	Dr.GRPO	4	-0.168	–	33–42	-0.139 (4)	-0.233 (4)
Countdown	MaxRL	5	-0.063	[-0.079, -0.047]	78–81	-0.062 (5)	-0.063 (5)
Python	Dr.GRPO	0	–	–	0	– (0)	– (0)
Python	MaxRL	5	-0.093	[-0.350, +0.165]	22–128	-0.039 (5)	-0.090 (5)
MathIR	Dr.GRPO	5	+0.004	[-0.007, +0.015]	9–14	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	-0.001	[-0.010, +0.009]	40–72	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	-0.404	[-0.600, -0.209]	74–81	-0.435 (5)	-0.352 (5)
Pantry	MaxRL	5	-0.129	[-0.223, -0.035]	81	-0.203 (5)	-0.043 (5)

Table 22: **Replay minus control at the final checkpoint: Falcon3-1B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.273	[-0.299, -0.247]	54–71	-0.363 (5)	-0.151 (5)
Graph	MaxRL	5	-0.162	[-0.261, -0.062]	68–83	-0.169 (5)	+0.059 (5)
Countdown	Dr.GRPO	5	-0.137	[-0.183, -0.090]	70–75	-0.153 (5)	-0.139 (5)
Countdown	MaxRL	5	-0.105	[-0.126, -0.084]	56–76	-0.113 (5)	-0.095 (5)
Python	Dr.GRPO	2	-0.190	–	0–22	-0.221 (2)	-0.141 (2)
Python	MaxRL	5	-0.153	[-0.265, -0.041]	22–81	-0.203 (5)	-0.083 (5)
MathIR	Dr.GRPO	5	-0.010	[-0.022, +0.003]	23–34	-0.017 (5)	+0.003 (5)
MathIR	MaxRL	5	+0.000	[+0.000, +0.000]	21–27	+0.000 (5)	+0.000 (5)
Pantry	Dr.GRPO	5	-0.300	[-0.420, -0.179]	71–83	-0.337 (5)	-0.305 (5)
Pantry	MaxRL	5	-0.292	[-0.319, -0.265]	81–89	-0.355 (5)	-0.237 (5)

Table 23: **Replay minus control at the final checkpoint: Qwen2.5-3B, Level 1.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Domain	Objective	n	$\Delta\hat{C}$	nominal 95% interval	$ E_s $	split 0 (n)	split 1 (n)
Graph	Dr.GRPO	5	-0.577	[-0.736, -0.418]	9–25	-0.599 (5)	-0.544 (5)
Graph	MaxRL	5	-0.405	[-0.472, -0.339]	28–42	-0.262 (5)	-0.416 (5)
Countdown	Dr.GRPO	4	-0.230	–	0–9	-0.281 (4)	-0.292 (4)
Countdown	MaxRL	4	-0.073	–	0–10	+0.000 (3)	-0.118 (4)
Python	Dr.GRPO	5	-0.209	[-0.563, +0.146]	104	-0.219 (5)	-0.234 (5)
Python	MaxRL	5	-0.106	[-0.397, +0.184]	104	-0.076 (5)	-0.121 (5)
MathIR	Dr.GRPO	5	+0.000	[+0.000, +0.000]	12–122	+0.000 (5)	+0.000 (5)
MathIR	MaxRL	5	-0.001	[-0.004, +0.002]	29–125	+0.000 (5)	-0.001 (5)
Pantry	Dr.GRPO	3	+0.495	–	0–16	+0.656 (1)	+0.554 (2)
Pantry	MaxRL	4	+0.439	–	0–19	+0.267 (3)	+0.208 (2)

Table 24: **Replay minus control at the final checkpoint: Qwen2.5-0.5B, Level 2.** Positive values indicate greater concentration. Equal prompt weights use $R \geq 2$ correct representatives in both conditions, separately for each contrast; $|E_s|$ is the range of eligible prompt counts out of 128 across available seed pairs. n counts seeds with a defined estimate, from five registered seeds. Each disjoint orientation has its own eligibility and displayed n . Intervals describe measured seed variability, are unadjusted and untruncated, and are omitted unless all five seeds are defined. Dashes retain unavailable or unidentified comparisons. Every registered comparison remains represented.

Initial evaluations are measured within each method. Qwen2.5-3B MathIR and Pantry E80-R1 seeds 71–74 carry slightly different initial requests from plain GRPO (.154 versus .162, and .832 versus .828 on P), so those cells have separate initial references.

Table 25: **Problem precheck, five-domain macro-averages.** Both RLVR-only objectives at pass 0 and pass 8, all three scales, five seeds and four fixed $K = 8$ draws per model–domain–method comparison. “Extra modes” is `distinct@8–pass@8`; “removed” is the fraction of that quantity lost between pass 0 and pass 8. Breadth loss is present at every scale and is far from uniform across scales, and at Falcon3-1B Dr.GRPO macro `pass@8` falls as well. Because extra modes move with correctness, the same declines are also reported on PCMD in the text above, where they are reproduced to within a few percentage points of the values printed in this table.

Model	Dr.GRPO			GRPO		
	pass@8	extra modes	removed	pass@8	extra modes	removed
Qwen2.5-0.5B	0.335 → 0.402	0.329 → 0.002	99.4%	0.335 → 0.435	0.329 → 0.006	98.2%
Falcon3-1B	0.391 → 0.342	0.332 → 0.094	71.6%	0.391 → 0.454	0.332 → 0.135	59.4%
Qwen2.5-3B	0.356 → 0.455	0.205 → 0.120	41.5%	0.356 → 0.490	0.199 → 0.101	49.4%

Graph and PantryPlan contain substantial initial breadth at every scale; the other three domains begin with at most .011 extra modes per prompt. A retained-breadth percentage is therefore reported only when the initial extra-mode count exceeds .05. Raw `distinct@8` ends below its initial value in six of the 15 Dr.GRPO domains. Re:Dr does so in two available domain summaries, both PantryPlan; Falcon Countdown uses four admissible seeds. On the success-conditional axis the contrast is sharper: averaged over the domains that remain measurable at the terminal step, Dr.GRPO ends at PCMD .017, .103, and .086 at 0.5B, 1B, and 3B against Re:Dr’s .233, .201, and .194. These observations concern sampled held-out support, not the survival histories of particular stored modes.

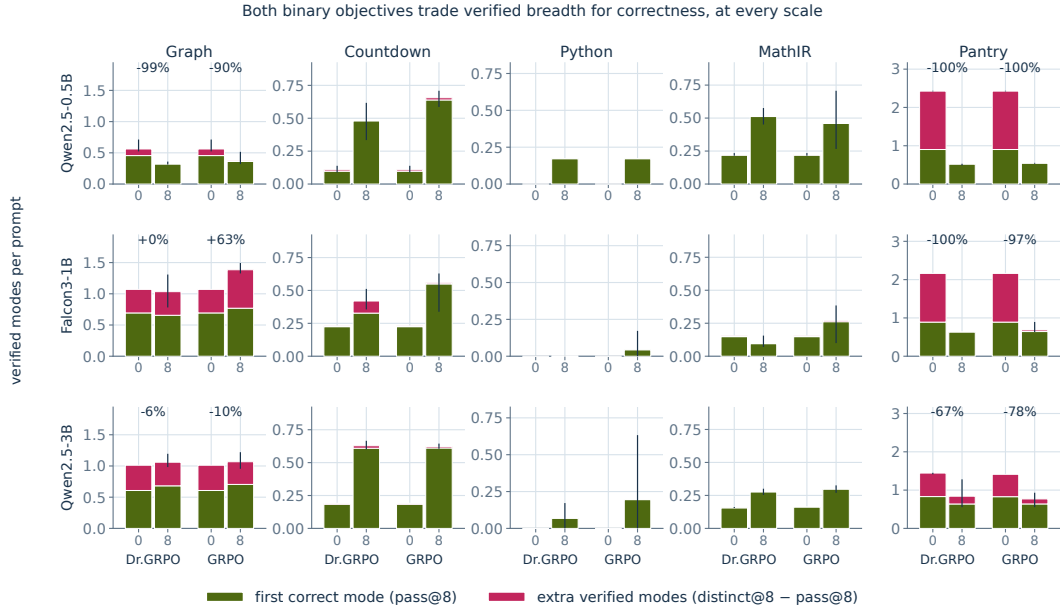


Figure 25: **RLVR-only training reduces average extra verified modes at each tested scale.** Bars split the registered `distinct@8` into the first correct mode and every verified mode beyond it, at training pass 0 and pass 8, for unmodified Dr.GRPO then GRPO in each panel. All 150 evaluations are complete at five seeds and four fixed $K = 8$ draws; whiskers are the seed range and pass 0 is measured per method. Badges give the change in extra verified modes, printed only where pass-0 breadth exceeds .05 of them; the other three domains start at .011 or less.

Table 26: **Absolute references for the registered raw `distinct@8`.** *Uniform* samples the exposed action representation uniformly and is defined only for Graph (27 color strings) and PantryPlan (64 bit masks). Frozen is the matched pass-0 model; the final two columns are pass-8 means. Dashes avoid pretending that an open-ended program or derivation space has a canonical uniform distribution.

Model	Domain	Uniform	Frozen	Dr.GRPO	Re:Dr
Qwen2.5-0.5B	Graph coloring	1.633	.561	.325	2.438
	PantryPlan	2.153	2.426	.522	1.539
Falcon3-1B	Graph coloring	1.633	1.070	1.034	1.717
	PantryPlan	2.153	2.162	.633	1.576
Qwen2.5-3B	Graph coloring	1.633	1.012	1.060	1.637
	PantryPlan	2.153	1.445	.841	1.514

Uniform-action references are meaningful for the enumerable Graph and Pantry representations. They are not defined for open-ended programs or derivations. Pantry’s bit-uniform reference exceeds every final model, so this domain is a support-concentration stress test rather than evidence of complete support coverage.

G.3 FRESH-GRADIENT DEGENERACY AND OBSERVED OUTPUT COLLAPSE

A Dr.GRPO group with identical rewards has exactly zero fresh task gradient. Only mixed groups containing both correct and incorrect responses can supply fresh reward information; optimizer momentum can still move a policy when that task gradient is zero. The fractions of mixed groups over all 3,072 Qwen2.5-0.5B updates, averaged over five seeds, are:

Domain	Dr.GRPO	Re:Dr
Graph coloring	.144	.954
Countdown	.140	.456
Python factors	.015	.185
MathIR	.364	.453
PantryPlan	.123	.789

Thus 98.5% of Python control updates carry zero fresh task gradient. Replay supplies a verified-likelihood gradient directly and can also change the policy so that later fresh groups become mixed.

A raw-response audit of all five Qwen2.5-0.5B Python control seeds establishes observed output collapse directly. At pass 0, a draw has 7.99 unique raw completions per prompt on average. By steps 288–480, every seed produces 32 byte-identical temperature-1 samples per prompt, also identical to the logged greedy completion, on all 128 prompts. This persists through step 3,072. Terminal draws contain 1.00 unique completion per prompt in the control and 1.97 under replay. A collapsed completion can be incorrect: this is one observed completion per prompt, not one correct key everywhere. Aggregate endpoint equality or zero draw spread alone would not establish that conclusion, nor does this finite audit rule out low-probability tails.

These checks explain a vulnerability of the registered sparse binary-reward recipe. They do not establish dominance over methods that keep fresh on-policy signal active. Dynamic mixed-group sampling, larger groups, reference KL, token-entropy regularization, and wider-decoding evaluations remain relevant controls beyond the registered factorial.

G.4 FIXED-BANK EXEMPLAR SCORE TRAJECTORIES

The fixed-bank study tracks Qwen2.5-0.5B Re:Dr on Graph for seeds 43–47. Before step 384’s fresh group, bank membership and fresh counts are frozen; admission then stops while uniform replay continues through 3,072 updates under the same training recipe. Every frozen exemplar is identified by prompt and canonical-key fingerprints, without selection by its score trajectory. Each scheduled replay visit records mean token log probability s and sequence log probability ℓ . Changes compare an identity’s first and final post-freeze visits; observation steps can differ across identities.

All 3,406 frozen identities across 1,575 seed–prompt pairs have finite, aligned scores at every scheduled visit: 8–9 visits per identity and 29,041 observations in total. The pooled median change is $+0.038$ nat/token (95% interval $[+.025, +.048]$), but the 10th percentile is $-.256$ and 2.61% of identities lose more than $.5$ nat/token ($[1.54, 3.77]\%$). The worst intermediate change is -1.328 nat/token. On the sequence scale, the median is $+1.54$ nat/sequence ($[+.104, +.198]$), while 21.99% lose more than $.5$ nat/sequence. Lengths range from 4 to 100 tokens, so the two threshold percentages have different meanings.

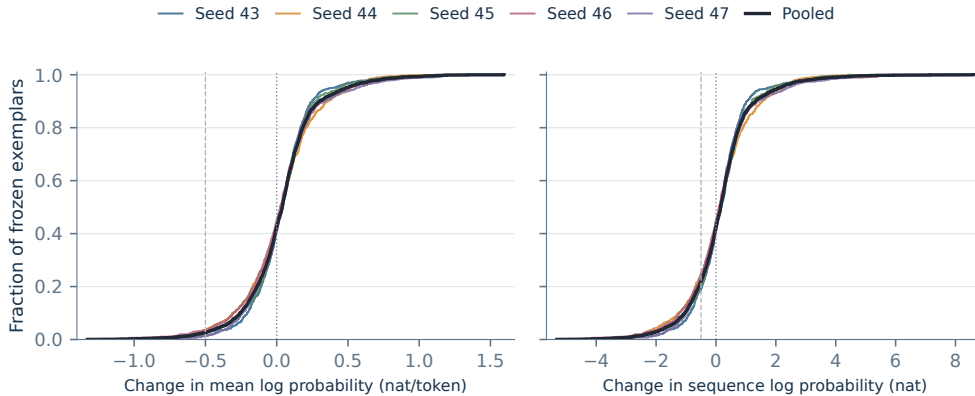


Figure 26: **Fixed-exemplar scores improve centrally but retain a declining tail.** Empirical cumulative distributions show first-to-final post-freeze score changes for every frozen identity, with one curve per seed and the pooled identity distribution. Mean token and sequence scores are separate panels; positive values indicate increased teacher-forced likelihood. Neither panel measures the exact probability of generating a canonical mode during policy sampling.

Pooled summaries weight identities equally. Descriptive 95% intervals use 10,000 hierarchical bootstrap draws (random seed 121), resampling prompts within each original seed and then five seeds with replacement; repeated seeds reuse their within-draw prompt sample. Linear empirical quantiles and percentile intervals are analysis choices. The artifact paper/results/e121_fixed_bank_survival.json retains all seed summaries, identity trajectories, source hashes, and audit diagnostics.

2700 These are teacher-forced score surrogates, not exact probabilities of sampling canonical modes,
2701 and visits do not cover every intervening update; the declining tail excludes a claim that every
2702 exemplar improves. Without a matched no-replay fixed-bank arm, the study does not identify a
2703 causal replay effect, and it does not numerically validate Theorem N.9's floor. Its contribution
2704 is longitudinal measurement of fixed exemplars alongside the held-out evidence.
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753

Part IV. What breadth represents

H MATCHED PROMPT-HINT ABLATION

H.1 REMOVING STRATEGY HINTS FROM UNCHANGED PROBLEMS

We freeze a paired follow-up on Python factors, MathIR, and Pantry at Levels 2 and 3. Within each of these six cells, the 32 smallest SHA-256 hashes of (20260911, level, domain, row index) select problems from the 128-row evaluation split, without consulting outcomes. Both arms generate eight fresh draws per identical problem. The original arm retains the published prompt. The neutral arm removes the Python suggestions to test small divisors with nested conditionals or dispatch on listed values, the MathIR ordered algebraic-isolation suggestion, and the Pantry preference for high-energy/protein, low-sodium ingredients such as seeds or oats. MathIR changes “those operations” to “the operations” to preserve the menu-ID instruction. The user problem, answer specification, executable verifier, and remaining format requirements are unchanged.

We report the paired neutral-minus-original effects on empirical $P_8 = \text{pass@8}$ and $D_8 = \text{distinct@8}$, with $B_8 = D_8 - P_8$ as additional verified breadth. All returned answers, including incorrect, empty, and truncated outputs, remain in their eight-draw groups; only verifier-accepted outputs create a verified key. Strict verification is primary; one unchanged, previously frozen formatting normalizer is a sensitivity analysis and preserves every strict success and canonical key. Python outputs in both arms undergo the same serial, warmed-verifier recheck, with original grades and any corrections retained in separate receipts.

Intervals use 20,000 whole-prompt bootstrap resamples, paired across arms and stratified by domain and level (seed 20260911). Cells contribute equally to reported macros. These are point-wise exploratory intervals for 32 problems per cell; crossing zero does not establish equivalence or absence of a prompt effect. A zero-width bootstrap interval records zero observed resampled variation, including all-tied P_8 outcomes; it is not precise evidence of population equivalence. Correct-pair collision and the certified-support uniform reference are secondary and retain their correct-pair denominators; the machine-readable report includes joint eligibility counts and undefined cells. Eight sampled draws do not identify total unseen support.

This appendix reports the complete local panel only. The separately registered frontier panel is not included here, and the overall local-plus-frontier experiment is incomplete.

For local models, we evaluate the initial Qwen2.5-0.5B model and fixed Level-2-trained DrGRPO and Re:Dr checkpoints. Python and MathIR use matched training seeds 43–47; Pantry uses seeds 43 and 46. Level 3 is transfer evaluation. Seed means retain the same selected problem set across checkpoints. Five-seed intervals resample whole training seeds and paired prompts; Pantry reports both seed effects and intervals conditional on the two checkpoints. The native local syntax constraints and 192-token budget differ from hosted inference, so effects are interpreted within each model and interface.

Two-seed Pantry sensitivity. Two checkpoints cannot support a stable estimate of training-seed variability. The following lists both strict prompt effects; the intervals in the local table condition on these two checkpoints. DrGRPO L2 (seed 43: $\Delta P_8 = +0.000$, $\Delta D_8 = +0.03$; seed 46: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.09$); DrGRPO L3 (seed 43: $\Delta P_8 = +0.031$, $\Delta D_8 = +0.06$; seed 46: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.09$); Re:Dr L2 (seed 43: $\Delta P_8 = -0.031$, $\Delta D_8 = -0.06$; seed 46: $\Delta P_8 = +0.000$, $\Delta D_8 = +0.03$); Re:Dr L3 (seed 43: $\Delta P_8 = +0.031$, $\Delta D_8 = +0.12$; seed 46: $\Delta P_8 = +0.062$, $\Delta D_8 = +0.09$).

Interpreting the intervention. The Python intervention removes an executable nested-conditional example along with divisor-testing and dispatch guidance; it also shortens the prompt. Its paired effect concerns this combined removal and does not isolate a preference cue from example or length effects. A reduction in distinct@8 accompanied by lower pass@8 is not by itself evidence of greater concentration among correct outputs: both endpoints fall when the model simply succeeds less often. This is precisely the ambiguity PCMD removes, since it is estimated from the verified draws alone, and it is the reading to consult when the two registered endpoints move together. Here the initial model is the single fixed Qwen2.5-0.5B-Instruct checkpoint before the reported RL training; it is already instruction-tuned and has no fine-tuning-seed replication in this ablation.

Table 27: Local Qwen2.5-0.5B checkpoints under strict verification. Each arrow gives original $\rightarrow$ neutral. P_8 is empirical pass@8, D_8 is verified distinct@8, and $B_8 = D_8 - P_8$. Effects are neutral minus original; brackets are pointwise 95% intervals. Local rows average the displayed number of training seeds. Five-seed intervals include seed and paired-prompt resampling; two-seed Pantry intervals condition on the two fixed checkpoints.

Model	Cell	P_8 : O $\rightarrow$ N	D_8 : O $\rightarrow$ N	ΔP_8 [95%]	ΔD_8 [95%]	ΔB_8
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.656 $\rightarrow$ 0.125	0.69 $\rightarrow$ 0.12	-0.531 [-0.719, -0.344]	-0.56 [-0.75, -0.38]	-0.03
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.250 $\rightarrow$ 0.281	0.25 $\rightarrow$ 0.28	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.156 $\rightarrow$ 0.188	0.19 $\rightarrow$ 0.22	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.719 $\rightarrow$ 0.031	0.81 $\rightarrow$ 0.03	-0.688 [-0.844, -0.531]	-0.78 [-0.97, -0.59]	-0.09
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.219 $\rightarrow$ 0.219	0.22 $\rightarrow$ 0.22	+0.000 [-0.125, +0.125]	+0.00 [-0.12, +0.12]	+0.00
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.281 $\rightarrow$ 0.312	0.66 $\rightarrow$ 0.69	+0.031 [+0.000, +0.094]	+0.03 [-0.06, +0.12]	+0.00
DrGRPO ($s = 5$)	MathIR L2	0.531 $\rightarrow$ 0.506	0.53 $\rightarrow$ 0.51	-0.025 [-0.100, +0.031]	-0.03 [-0.10, +0.03]	+0.00
DrGRPO ($s = 5$)	MathIR L3	0.194 $\rightarrow$ 0.156	0.19 $\rightarrow$ 0.16	-0.037 [-0.113, +0.019]	-0.04 [-0.11, +0.02]	+0.00
DrGRPO ($s = 2$)	Pantry L2	0.219 $\rightarrow$ 0.203	0.28 $\rightarrow$ 0.25	-0.016 [-0.156, +0.125]	-0.03 [-0.19, +0.12]	-0.02
DrGRPO ($s = 2$)	Pantry L3	0.172 $\rightarrow$ 0.172	0.27 $\rightarrow$ 0.25	+0.000 [-0.078, +0.078]	-0.02 [-0.16, +0.11]	-0.02
DrGRPO ($s = 5$)	Python factors L2	0.812 $\rightarrow$ 0.000	0.81 $\rightarrow$ 0.00	-0.812 [-0.938, -0.656]	-0.81 [-0.94, -0.66]	+0.00
DrGRPO ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.000	1.00 $\rightarrow$ 0.00	-1.000 [-1.000, -1.000]	-1.00 [-1.00, -1.00]	+0.00
Re:Dr ($s = 5$)	MathIR L2	0.994 $\rightarrow$ 0.975	0.99 $\rightarrow$ 0.97	-0.019 [-0.075, +0.000]	-0.02 [-0.07, +0.00]	+0.00
Re:Dr ($s = 5$)	MathIR L3	0.312 $\rightarrow$ 0.275	0.31 $\rightarrow$ 0.28	-0.037 [-0.138, +0.044]	-0.04 [-0.14, +0.04]	+0.00
Re:Dr ($s = 2$)	Pantry L2	0.516 $\rightarrow$ 0.500	0.75 $\rightarrow$ 0.73	-0.016 [-0.109, +0.078]	-0.02 [-0.20, +0.17]	+0.00
Re:Dr ($s = 2$)	Pantry L3	0.234 $\rightarrow$ 0.281	0.39 $\rightarrow$ 0.50	+0.047 [+0.000, +0.109]	+0.11 [+0.05, +0.19]	+0.06
Re:Dr ($s = 5$)	Python factors L2	0.887 $\rightarrow$ 0.556	1.24 $\rightarrow$ 0.68	-0.331 [-0.525, -0.150]	-0.57 [-0.85, -0.32]	-0.24
Re:Dr ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.494	1.42 $\rightarrow$ 0.66	-0.506 [-0.825, -0.194]	-0.76 [-1.07, -0.38]	-0.26

Table 28: Local Qwen2.5-0.5B checkpoints under frozen formatting normalization. Each arrow gives original $\rightarrow$ neutral. P_8 is empirical pass@8, D_8 is verified distinct@8, and $B_8 = D_8 - P_8$. Effects are neutral minus original; brackets are pointwise 95% intervals. Local rows average the displayed number of training seeds. Five-seed intervals include seed and paired-prompt resampling; two-seed Pantry intervals condition on the two fixed checkpoints.

Model	Cell	P_8 : O $\rightarrow$ N	D_8 : O $\rightarrow$ N	ΔP_8 [95%]	ΔD_8 [95%]	ΔB_8
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.656 $\rightarrow$ 0.125	0.69 $\rightarrow$ 0.12	-0.531 [-0.719, -0.344]	-0.56 [-0.75, -0.38]	-0.03
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.250 $\rightarrow$ 0.281	0.25 $\rightarrow$ 0.28	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.156 $\rightarrow$ 0.188	0.19 $\rightarrow$ 0.22	+0.031 [+0.000, +0.094]	+0.03 [+0.00, +0.09]	+0.00
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.719 $\rightarrow$ 0.031	0.81 $\rightarrow$ 0.03	-0.688 [-0.844, -0.531]	-0.78 [-0.97, -0.59]	-0.09
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.219 $\rightarrow$ 0.219	0.22 $\rightarrow$ 0.22	+0.000 [-0.125, +0.125]	+0.00 [-0.12, +0.12]	+0.00
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.281 $\rightarrow$ 0.312	0.66 $\rightarrow$ 0.69	+0.031 [+0.000, +0.094]	+0.03 [-0.06, +0.12]	+0.00
DrGRPO ($s = 5$)	MathIR L2	0.531 $\rightarrow$ 0.506	0.53 $\rightarrow$ 0.51	-0.025 [-0.100, +0.031]	-0.03 [-0.10, +0.03]	+0.00
DrGRPO ($s = 5$)	MathIR L3	0.194 $\rightarrow$ 0.156	0.19 $\rightarrow$ 0.16	-0.037 [-0.113, +0.019]	-0.04 [-0.11, +0.02]	+0.00
DrGRPO ($s = 2$)	Pantry L2	0.219 $\rightarrow$ 0.203	0.28 $\rightarrow$ 0.25	-0.016 [-0.156, +0.125]	-0.03 [-0.19, +0.12]	-0.02
DrGRPO ($s = 2$)	Pantry L3	0.172 $\rightarrow$ 0.172	0.27 $\rightarrow$ 0.25	+0.000 [-0.078, +0.078]	-0.02 [-0.16, +0.11]	-0.02
DrGRPO ($s = 5$)	Python factors L2	0.812 $\rightarrow$ 0.000	0.81 $\rightarrow$ 0.00	-0.812 [-0.938, -0.656]	-0.81 [-0.94, -0.66]	+0.00
DrGRPO ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.000	1.00 $\rightarrow$ 0.00	-1.000 [-1.000, -1.000]	-1.00 [-1.00, -1.00]	+0.00
Re:Dr ($s = 5$)	MathIR L2	0.994 $\rightarrow$ 0.975	0.99 $\rightarrow$ 0.97	-0.019 [-0.075, +0.000]	-0.02 [-0.07, +0.00]	+0.00
Re:Dr ($s = 5$)	MathIR L3	0.312 $\rightarrow$ 0.275	0.31 $\rightarrow$ 0.28	-0.037 [-0.138, +0.044]	-0.04 [-0.14, +0.04]	+0.00
Re:Dr ($s = 2$)	Pantry L2	0.516 $\rightarrow$ 0.500	0.75 $\rightarrow$ 0.73	-0.016 [-0.109, +0.078]	-0.02 [-0.20, +0.17]	+0.00
Re:Dr ($s = 2$)	Pantry L3	0.234 $\rightarrow$ 0.281	0.39 $\rightarrow$ 0.50	+0.047 [+0.000, +0.109]	+0.11 [+0.05, +0.19]	+0.06
Re:Dr ($s = 5$)	Python factors L2	0.887 $\rightarrow$ 0.556	1.24 $\rightarrow$ 0.68	-0.331 [-0.525, -0.150]	-0.57 [-0.85, -0.32]	-0.24
Re:Dr ($s = 5$)	Python factors L3	1.000 $\rightarrow$ 0.494	1.42 $\rightarrow$ 0.66	-0.506 [-0.825, -0.194]	-0.76 [-1.07, -0.38]	-0.26

Observed prompt dependence. Python shows a large loss of correctness after guidance removal. Across the five DrGRPO seeds, mean P_8 falls from .8125 to zero at Level 2 and from 1.0 to zero at Level 3; D_8 falls identically and $B_8 = 0$ in both arms. With no correct neutral DrGRPO outputs, its conditional-correct concentration is undefined. Re:Dr retains neutral $P_8 = .55625$ and .49375 and $D_8 = .675$ and .65625 at Levels 2 and 3, respectively, while losing additional breadth: $\Delta B_8 = -.2375$ and $-.25625$. The initial checkpoint also loses Python correctness (Table 27). These observations identify substantial dependence on the combined Python wording intervention; a raw mode-count decline cannot be interpreted solely as a redistribution among correct answers.

Post-hoc Python failure audit. A descriptive audit of all 5,120 saved DrGRPO Python draws (five seeds, two levels, both wordings) finds that all 2,560 original outputs extract the same lambda, whose body matches the conditional example in the prompt:

```
lambda n: 2 if n % 2 == 0 else 3 if n % 3 == 0 else 5
```

Of the 2,560 neutral draws, 1,816 (70.94%) are invalid Python expressions, 59 violate other restricted-language rules, 376 return non-integers, 276 fail the proper-divisor condition, and 33 lack an extracted candidate after reaching the token limit. Thus most neutral failures involve syntax; truncation accounts for only 1.29%. The unchanged extractor, parser, and executor reproduce every stored grade and successful canonical key. This taxonomy was developed after the paired results were known and does not identify an internal training mechanism. Counts, deter-

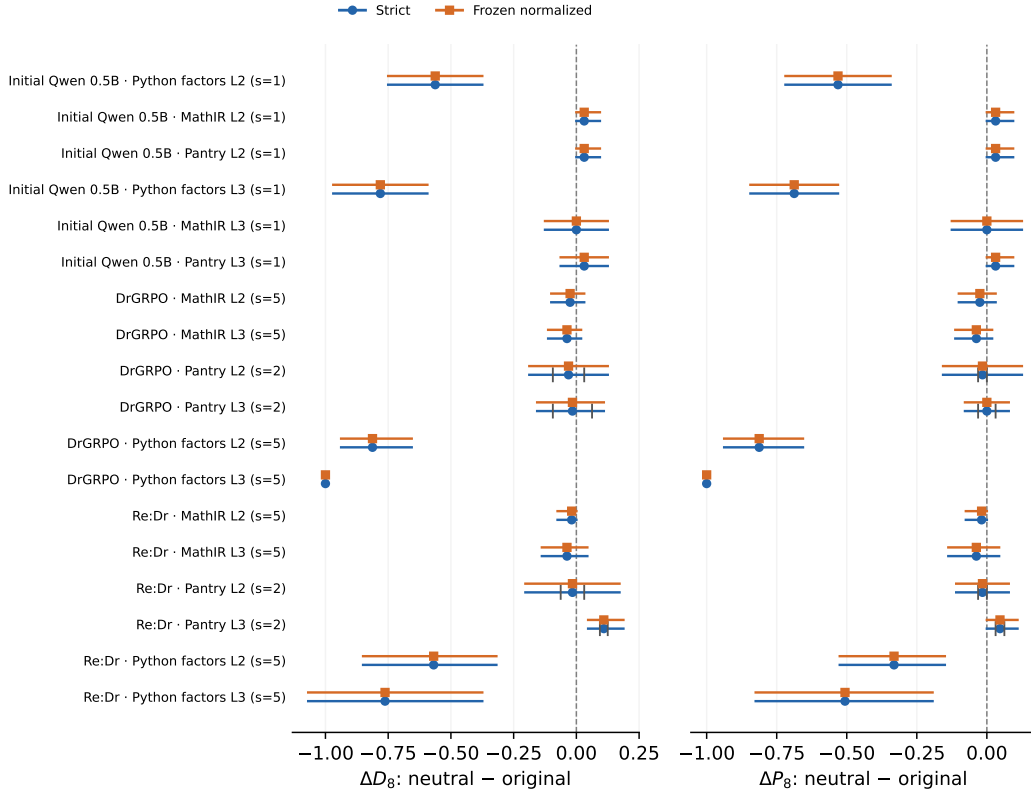


Figure 27: Prompt-hint removal effects for local checkpoints, preserving every registered domain-level cell. Blue circles use strict verification; orange squares use the same frozen formatting normalizer. Horizontal bars show the intervals described in the tables; zero is the vertical reference. Gray endpoint marks show the two individual Pantry seed effects.

ministic paired examples, and source hashes are retained in `results/modebench_prompt_ablation_python_failure_diagnostic_20260911.json`.

Concentration after hint removal. Every trained MathIR checkpoint has $B_8 = 0$ under both wordings at both levels. Within each problem’s eight-draw group, every observed pair of correct outputs shares a canonical key, in every trained checkpoint-level cell: correct-pair collision is 1.0, versus the certified-support uniform reference of .2. For example, at Level 2, Replay DrGRPO retains mean $P_8 = .975$ without the order hint, compared with .99375 originally. Across its five checkpoints, the neutral arm has 4,179 colliding pairs out of 4,179 correct pairs, from 154 eligible checkpoint-problem groups; the original arm has 4,382 out of 4,382 pairs from 157 such groups. Both arms evaluate the same 32 problems at each checkpoint. This preserves the observed concentration despite removing the specified order hint; it does not measure all unseen modes or remove the other interface constraints. Pantry effects are mixed. Re:Dr at Level 3 gains .109375 in D_8 , comprising .046875 in P_8 and .0625 in B_8 , but this comparison has only two training seeds. The separate completed 64-draw hosted follow-up tests both wordings on sixteen problems per cell in App. J; its population and draw budget remain distinct from this local eight-draw experiment.

I THE DECODING GRID BEHIND THE OBJECTION

Does a different decoding setting reopen a collapsed policy? The cheapest remedy would be to turn the temperature up. We put every frozen terminal checkpoint back through the ordinary evaluation path at six temperatures from 0.5 to 2.0, at four times the sample budget, and with the nucleus truncated to top- p .95 — eleven decoding settings per checkpoint, across five domains and five seeds (App. I). A RLVR-only policy stays where it collapsed at every one of them (Table 29). In Graph its PCMD is .000 across the entire grid: two correct colourings are never two different colourings, at any setting we tried, while a replay arm of the same cohort sits at .566 at ordinary settings. The largest value any decoding change buys the control in any domain is PantryPlan’s .069. Decoding is not where the missing modes went.

Table 29: **No decoding setting reopens a collapsed policy.** Qwen2.5-0.5B terminal checkpoints, five seeds, evaluated at eleven settings: temperature 0.5–2.0, $K \in \{8, 32\}$, and top- $p \in \{.95, 1\}$. The middle column gives the RLVR-only control’s PCMD range over every setting that clears the support bar; the last gives a replay arm of the same cohort at ordinary settings ($T = 1$, top- p 1, $K = 8$) for scale. Python never clears the bar under the control, which is itself the collapse reported in Section 5.1. This cohort trains for twelve passes and its replay arm predates Re:Dr, so the control column, not the contrast, is what answers the objection (App. I).

Domain	Settings reportable	Control PCMD over the grid	Replay at $T = 1$
Graph	11/11	.000	.566
Countdown	11/11	.006–.031	.522
Python	0/11	—	.440
MathIR	11/11	.000–.003	.028
PantryPlan	11/11	.037–.069	.446

This appendix answers the decoding objection with a completed grid that varies only how a frozen checkpoint is sampled. Each cell loads one terminal checkpoint, evaluates it once through the ordinary training evaluation path at a chosen $(T, \text{top-}p, K)$, and exits; no optimizer step, rollout or export occurs, so a cell is a pure function of the checkpoint and its decoding settings. Every non-decoding argument — prompt template, split, response length, draw seeds — is inherited from the source run’s own recorded configuration rather than retyped. Cells are pinned to the node that trained the checkpoint, because GPU type changes floating-point reduction order and therefore the sampled tokens; an unpinned sweep would mix hardware into the trade-off it exists to isolate.

The grid has three stages: temperature $T \in \{0.5, 0.7, 1.0, 1.3, 1.6, 2.0\}$ at top- p 1 and $K = 8 \times 4$ draws; budget $K = 32 \times 2$ draws at $T \in \{1.0, 1.3, 1.6\}$; and truncation at top- p .95 for $T \in \{1.0, 1.6\}$. With five domains, five seeds and two arms this is 110 measured checkpoint cells. PCMD pools a prompt’s draws before scoring it and a cell is reported only where at least 30 prompts return two verified responses, exactly as elsewhere in this paper. Every value is recomputed from the retained per-draw records, whose canonical keys were written by the validator that graded them; nothing is resampled or regraded.

Cohort and its limits. These are E72 checkpoints: Qwen2.5-0.5B, seeds 43–47, trained for twelve passes to step 4,609, with a replay arm that predates the Re:Dr objective contract. The benchmark, splits, verifier and evaluation path are the ones used throughout. What the grid identifies is the behaviour of an RLVR-only policy under changed decoding, which is what the objection is about; the replay column is a scale reference from the same cohort and is not a substitute for the matched comparisons of Section 5. One reading to avoid: temperature 2.0 degrades correctness in Countdown and Python, so a PCMD that fails to rise there says the policy has stopped solving the task, not that its breadth was unreachable. The record `paper/results/decoding_objection_e72.json` retains every cell, its seeds, its defined-prompt count and its source path; `ops/build_paper_decoding_objection.py` regenerates it and the displayed table.

J SAMPLING-BUDGET ABLATION

J.1 PROTOCOL AND ESTIMANDS

Large-budget `pass@k` evaluation can reveal differences hidden at small sampling budgets (Yue et al., 2025). We therefore freeze a second follow-up after observing the eight-draw prompt control: within every domain-level cell we take ranks 1–16 of its original outcome-independent SHA-256 selection. The six cells retain both prompt wordings, the same mathematical problems, and the same fixed initial and trained checkpoints. Every arm receives 64 fresh draws; none of the earlier eight-draw responses enters this analysis. The initial Qwen2.5-0.5B-Instruct checkpoint has no training-seed replication. DrGRPO and Re:Dr retain five matched seeds in Python and MathIR and two in Pantry. Trained Level-3 evaluation is transfer from Level-2 training.

For a complete 64-draw pool let c be its number of correct outputs and n_j its counts of distinct verified canonical keys. At $k \in \{1, 2, 4, 8, 16, 32, 64\}$, primary curves average all size- k subsets without replacement from the complete pool:

$$P_k = 1 - \frac{\binom{64-c}{k}}{\binom{64}{k}}, \quad D_k = \sum_j \left[1 - \frac{\binom{64-n_j}{k}}{\binom{64}{k}} \right], \quad B_k = D_k - P_k.$$

An infeasible numerator combination is zero. Every incorrect, empty, refused, or truncated response remains in the pool; only verifier-accepted responses contribute keys. These correlated curve points describe the retained pool, not complete unseen support. Ordered prefixes by preassigned draw index, rather than response arrival time, are a separately labeled sensitivity in the source report. $P_{64} - P_8$, $D_{64} - D_8$, and $B_{64} - B_8$ separate first-success gains from additional observed modes.

Strict verification is primary. The unchanged frozen formatter is a sensitivity, preserving every strict success and canonical key. All Python texts undergo the same serial warmed-verifier recheck; original grades and any corrections are retained. Intervals use 20,000 whole-problem bootstrap replicates (seed 20260911), paired across wordings and stratified by domain and level. Five-seed intervals additionally resample whole matched training seeds with shared prompt indices; Pantry intervals condition on its two checkpoints. Intervals are pointwise and exploratory for 16 problems per cell. Zero-width intervals reflect zero observed resampled variation, not precise evidence of equivalence.

To compare breadth at matched observed correctness, rarefy m correct draws alone:

$$R_m = \sum_j \left[1 - \frac{\binom{c-n_j}{m}}{\binom{c}{m}} \right].$$

The primary paired conditional comparison keeps only prompts with $c \geq m$ under both wordings; eligibility changes with m . Every eligible denominator is retained, and an undefined registered seed is never silently omitted from an equal-seed mean. Full-64 collision pools equal-key correct pairs within each checkpoint before taking equal-seed means.

All support counts here are certified lower bounds $L \leq M$, not exhaustive support sizes: Python uses its two externally certified witnesses, MathIR five certified modes, and Pantry its frozen certificate count. Accordingly $1/L$ is an upper reference for uniform collision over full support; $L[1 - (1 - 1/L)^m]$ is a lower reference for uniform expected breadth, not a lower bound on model breadth. Observed distinct counts may exceed L . The source report also averages this uniform breadth over $J \sim \text{Hypergeom}(64, c, k)$ to match unconditional correctness at each k .

Table 30: Eight-to-64 discovery gains under strict verification. $G_P = P_{64} - P_8$, $G_D = D_{64} - D_8$, and $G_B = B_{64} - B_8$. Each O/N entry gives original/neutral; ΔG_D is neutral minus original with its pointwise 95% interval. Each row represents a fixed hosted deployment; its intervals resample paired prompts and contain no variation across independently trained model seeds.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
GPT-5.4	Python factors L2	0.014/0.000	2.017/3.051	2.003/3.051	+1.033 [-0.044, +2.371]
GPT-5.6 Sol	Python factors L2	0.000/0.405	0.000/0.866	0.000/0.462	+0.866 [+0.545, +1.231]
Grok 4.3	Python factors L2	0.000/0.000	0.667/1.013	0.667/1.013	+0.346 [-0.583, +1.128]
GPT-5.4	Python factors L3	0.025/0.000	2.100/2.340	2.076/2.340	+0.240 [-0.379, +0.930]
GPT-5.6 Sol	Python factors L3	0.000/0.586	0.000/0.759	0.000/0.174	+0.759 [+0.581, +0.952]
Grok 4.3	Python factors L3	0.000/0.000	0.424/1.576	0.424/1.576	+1.152 [+0.640, +1.689]
GPT-5.4	MathIR L2	0.043/0.042	0.043/0.909	0.000/0.868	+0.867 [+0.587, +1.111]
GPT-5.6 Sol	MathIR L2	0.027/0.004	0.027/0.686	0.000/0.682	+0.659 [+0.416, +0.888]
Grok 4.3	MathIR L2	0.000/0.001	0.223/0.322	0.223/0.321	+0.099 [-0.226, +0.438]
GPT-5.4	MathIR L3	0.000/0.000	0.055/0.663	0.055/0.663	+0.609 [+0.342, +0.888]
GPT-5.6 Sol	MathIR L3	0.000/0.000	0.000/0.515	0.000/0.515	+0.515 [+0.318, +0.714]
Grok 4.3	MathIR L3	0.000/0.000	0.167/0.805	0.167/0.805	+0.638 [+0.348, +0.935]
GPT-5.4	Pantry L2	0.001/0.015	1.158/2.583	1.156/2.568	+1.426 [+0.423, +2.641]
GPT-5.6 Sol	Pantry L2	0.086/0.101	0.917/1.789	0.831/1.688	+0.872 [+0.245, +1.575]
Grok 4.3	Pantry L2	0.000/0.000	1.022/3.377	1.022/3.377	+2.354 [+1.592, +3.123]
GPT-5.4	Pantry L3	0.000/0.002	1.666/3.219	1.666/3.218	+1.553 [+0.776, +2.367]
GPT-5.6 Sol	Pantry L3	0.032/0.129	1.625/1.842	1.592/1.713	+0.217 [-0.489, +0.921]
Grok 4.3	Pantry L3	0.000/0.000	1.810/4.466	1.810/4.466	+2.656 [+1.698, +3.640]

Table 31: Eight-to-64 discovery gains under frozen normalization sensitivity. $G_P = P_{64} - P_8$, $G_D = D_{64} - D_8$, and $G_B = B_{64} - B_8$. Each O/N entry gives original/neutral; ΔG_D is neutral minus original with its pointwise 95% interval. Each row represents a fixed hosted deployment; its intervals resample paired prompts and contain no variation across independently trained model seeds.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
GPT-5.4	Python factors L2	0.000/0.000	2.156/3.216	2.156/3.216	+1.060 [-0.071, +2.386]
GPT-5.6 Sol	Python factors L2	0.008/0.010	0.113/0.794	0.105/0.784	+0.681 [+0.351, +1.066]
Grok 4.3	Python factors L2	0.000/0.000	0.667/1.013	0.667/1.013	+0.346 [-0.583, +1.128]
GPT-5.4	Python factors L3	0.000/0.000	2.431/2.604	2.431/2.604	+0.173 [-0.657, +1.075]
GPT-5.6 Sol	Python factors L3	0.011/0.007	0.120/0.490	0.109/0.483	+0.370 [+0.112, +0.674]
Grok 4.3	Python factors L3	0.000/0.000	0.424/1.576	0.424/1.576	+1.152 [+0.640, +1.689]
GPT-5.4	MathIR L2	0.043/0.042	0.043/0.909	0.000/0.868	+0.867 [+0.587, +1.111]
GPT-5.6 Sol	MathIR L2	0.027/0.004	0.027/0.686	0.000/0.682	+0.659 [+0.416, +0.888]
Grok 4.3	MathIR L2	0.000/0.001	0.223/0.322	0.223/0.321	+0.099 [-0.226, +0.438]
GPT-5.4	MathIR L3	0.000/0.000	0.055/0.663	0.055/0.663	+0.609 [+0.342, +0.888]
GPT-5.6 Sol	MathIR L3	0.000/0.000	0.000/0.515	0.000/0.515	+0.515 [+0.318, +0.714]
Grok 4.3	MathIR L3	0.000/0.000	0.167/0.805	0.167/0.805	+0.638 [+0.348, +0.935]
GPT-5.4	Pantry L2	0.001/0.015	1.158/2.583	1.156/2.568	+1.426 [+0.423, +2.641]
GPT-5.6 Sol	Pantry L2	0.004/0.004	0.954/1.834	0.950/1.830	+0.880 [+0.237, +1.547]
Grok 4.3	Pantry L2	0.000/0.000	1.022/3.377	1.022/3.377	+2.354 [+1.592, +3.123]
GPT-5.4	Pantry L3	0.000/0.002	1.666/3.219	1.666/3.218	+1.553 [+0.776, +2.367]
GPT-5.6 Sol	Pantry L3	0.001/0.020	1.586/2.462	1.585/2.443	+0.877 [+0.272, +1.436]
Grok 4.3	Pantry L3	0.000/0.000	1.810/4.466	1.810/4.466	+2.656 [+1.698, +3.640]

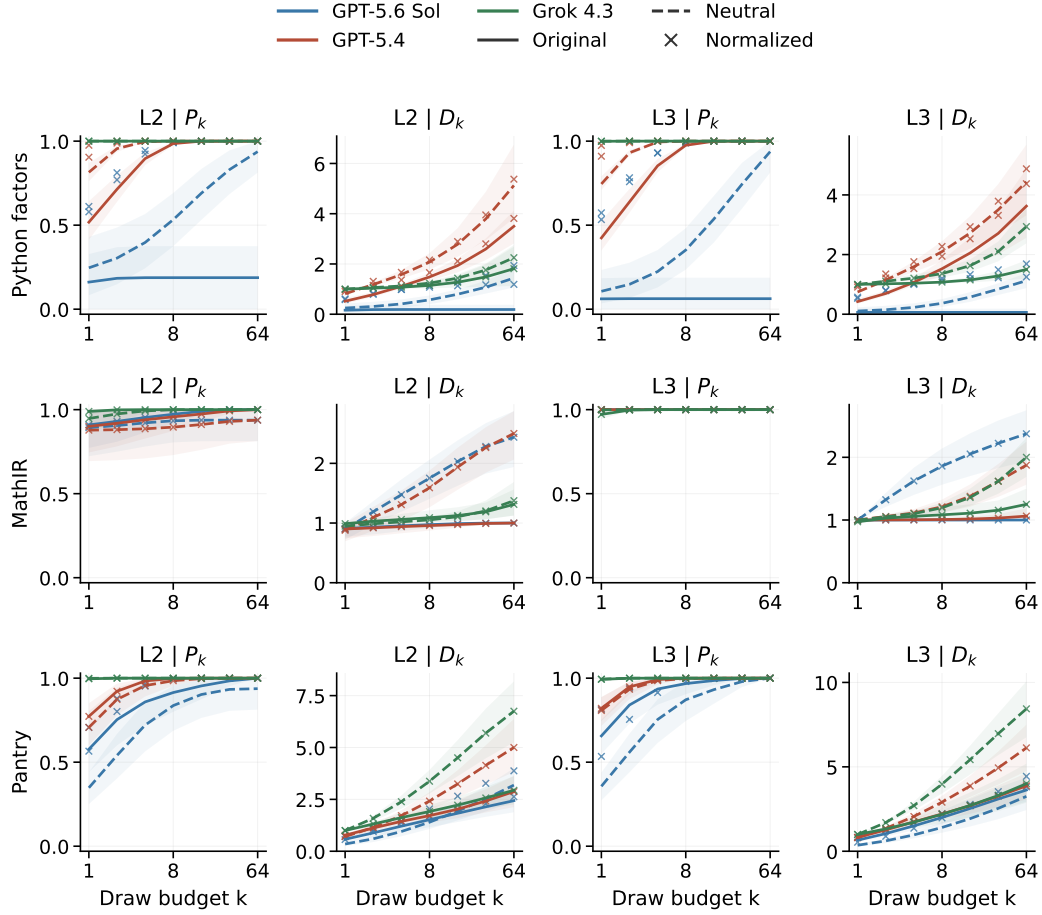


Figure 28: Fresh 64-draw discovery curves. Separate axes show P_k and D_k for each level and domain; pass probability always uses a 0–1 scale. Color identifies the checkpoint method or hosted model, solid/dashed lines original/neutral wording. Lines and shaded pointwise intervals use strict grading; crosses show the frozen-normalization sensitivity. Prefix estimates are separately retained in the report with pointwise intervals for every model and cell.

Table 32: Full-64 strict correct-pair collision. C pools correct pairs within a fixed checkpoint or deployment; trained local rows then average checkpoint rates equally across seeds. U applies the same weighting to the certified-support uniform upper reference. Eligible checkpoint–problem groups E and correct-pair counts Q are summed descriptively across checkpoints. These totals do not define the equal-seed mean; separate pooled rates and per-seed counts are retained in the report. A seed with no eligible pair leaves its all-seed mean undefined, with every individual denominator retained.

Model	Cell	C_O	C_N	U_O	U_N	E_O/E_N	Q_O/Q_N
GPT-5.4	Python factors L2	0.790	0.665	0.500	0.500	16/16	10374/21720
GPT-5.6 Sol	Python factors L2	1.000	0.974	0.500	0.500	3/13	4467/6074
Grok 4.3	Python factors L2	0.963	0.938	0.500	0.500	16/16	32256/32256
GPT-5.4	Python factors L3	0.732	0.620	0.500	0.500	16/16	6747/18043
GPT-5.6 Sol	Python factors L3	1.000	0.966	0.500	0.500	1/10	1953/2097
Grok 4.3	Python factors L3	0.981	0.893	0.500	0.500	16/16	32193/32256
GPT-5.4	MathIR L2	1.000	0.756	0.200	0.200	16/15	28440/28227
GPT-5.6 Sol	MathIR L2	1.000	0.678	0.200	0.200	16/15	28645/28377
Grok 4.3	MathIR L2	0.968	0.985	0.200	0.200	16/16	31600/29677
GPT-5.4	MathIR L3	0.998	0.944	0.200	0.200	16/16	32256/32256
GPT-5.6 Sol	MathIR L3	1.000	0.676	0.200	0.200	16/16	32256/32256
Grok 4.3	MathIR L3	0.966	0.949	0.200	0.200	16/16	30520/32193
GPT-5.4	Pantry L2	0.680	0.499	0.069	0.069	16/16	20097/17436
GPT-5.6 Sol	Pantry L2	0.674	0.624	0.076	0.070	16/15	12941/5001
Grok 4.3	Pantry L2	0.684	0.412	0.066	0.066	16/16	32005/32193
GPT-5.4	Pantry L3	0.546	0.392	0.065	0.064	16/16	22332/21975
GPT-5.6 Sol	Pantry L3	0.566	0.652	0.061	0.071	16/16	15229/4946
Grok 4.3	Pantry L3	0.673	0.294	0.063	0.063	16/16	31830/31881

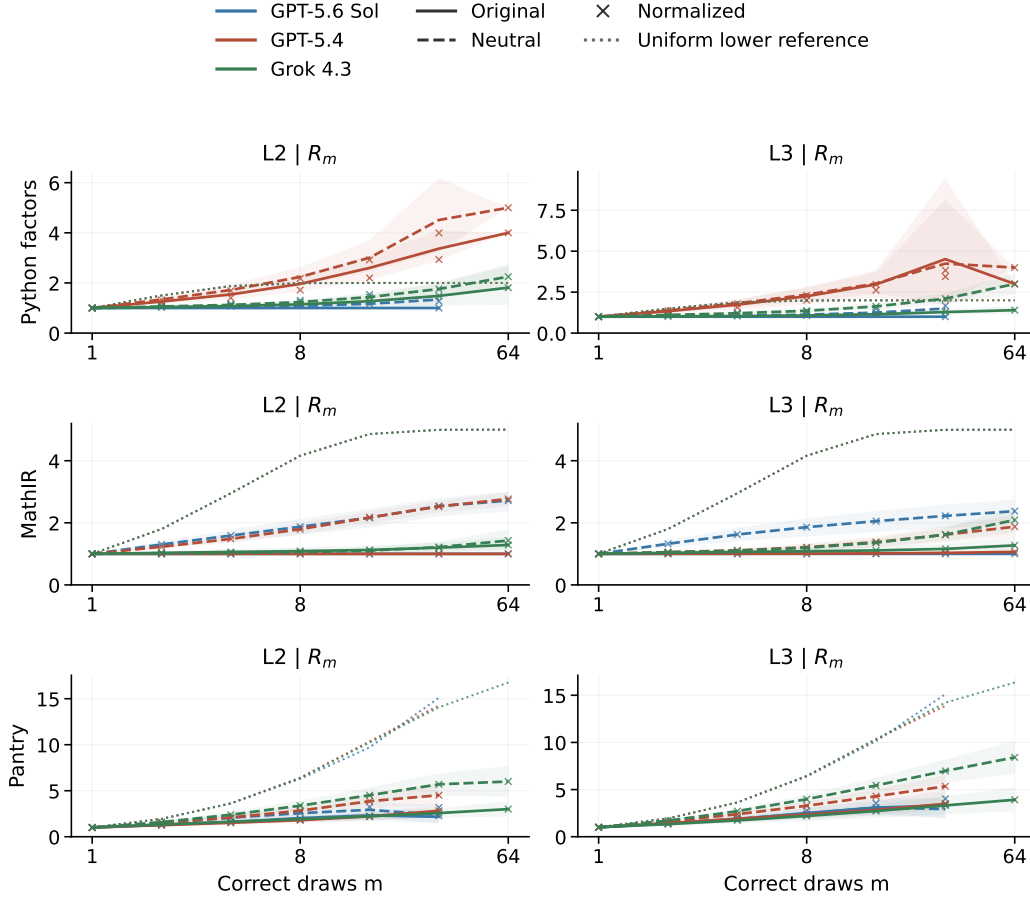


Figure 29: Breadth after exactly m correct draws, on prompts jointly eligible under both wordings. Color and solid/dashed wording styles match the discovery figure; dotted lines give method-specific uniform lower references using the same eligible prompts. Undefined means are gaps. Crosses show frozen normalization. Eligibility varies with m , so curves do not hold the eligible population fixed across the horizontal axis; the following table gives all strict joint counts for every method, level, and correct-draw budget, including groups with zero eligible prompts.

Table 33: Strict joint eligibility for a fixed correct-draw budget: both wordings must have at least m correct draws. Entries count eligible checkpoint–problem groups, summing over displayed seeds; the same 16 problems recur across checkpoints. Arm-specific eligibility at every m is also retained in the report for both prompt wordings and every registered checkpoint.

Model	Cell	$m = 1$	$m = 2$	$m = 4$	$m = 8$	$m = 16$	$m = 32$	$m = 64$
GPT-5.4	Python factors L2	16	16	16	16	16	5	1
GPT-5.6 Sol	Python factors L2	3	3	3	3	3	3	0
Grok 4.3	Python factors L2	16	16	16	16	16	16	16
GPT-5.4	Python factors L3	16	16	16	16	16	3	1
GPT-5.6 Sol	Python factors L3	1	1	1	1	1	1	0
Grok 4.3	Python factors L3	16	16	16	16	16	16	15
GPT-5.4	MathIR L2	15	15	14	14	14	14	13
GPT-5.6 Sol	MathIR L2	15	15	15	15	15	14	14
Grok 4.3	MathIR L2	16	16	16	16	16	15	14
GPT-5.4	MathIR L3	16	16	16	16	16	16	16
GPT-5.6 Sol	MathIR L3	16	16	16	16	16	16	16
Grok 4.3	MathIR L3	16	16	16	16	16	16	11
GPT-5.4	Pantry L2	16	16	16	16	15	13	0
GPT-5.6 Sol	Pantry L2	15	15	15	12	10	5	0
Grok 4.3	Pantry L2	16	16	16	16	16	16	12
GPT-5.4	Pantry L3	16	16	16	16	16	14	0
GPT-5.6 Sol	Pantry L3	16	16	15	13	13	3	0
Grok 4.3	Pantry L3	16	16	16	16	16	16	12

Table 34: Eight-to-64 discovery gains under strict verification. $G_P = P_{64} - P_8$, $G_D = D_{64} - D_8$, and $G_B = B_{64} - B_8$. Each O/N entry gives original/neutral; ΔG_D is neutral minus original with its pointwise 95% interval. $s = 1$ denotes one fixed initial checkpoint, not training-seed replication. Five-seed intervals resample seeds and paired prompts; two-seed Pantry intervals condition on the observed checkpoints and retain all paired prompt outcomes.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
DrGRPO ($s = 5$)	Python factors L2	0.000/0.000	0.000/0.000	0.000/0.000	+0.000 [+0.000, +0.000]
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.147/0.105	0.303/0.165	0.156/0.060	−0.138 [−0.217, −0.061]
Re:Dr ($s = 5$)	Python factors L2	0.000/0.231	0.058/0.463	0.058/0.232	+0.405 [+0.185, +0.620]
DrGRPO ($s = 5$)	Python factors L3	0.000/0.033	0.000/0.033	0.000/0.000	+0.033 [+0.000, +0.120]
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.218/0.096	0.593/0.096	0.376/0.000	−0.497 [−0.766, −0.270]
Re:Dr ($s = 5$)	Python factors L3	0.000/0.239	0.153/0.531	0.153/0.292	+0.379 [+0.111, +0.678]
DrGRPO ($s = 5$)	MathIR L2	0.021/0.017	0.021/0.017	0.000/0.000	−0.004 [−0.068, +0.045]
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.344/0.235	0.344/0.235	0.000/0.000	−0.109 [−0.383, +0.164]
Re:Dr ($s = 5$)	MathIR L2	0.010/0.011	0.010/0.011	0.000/0.000	+0.001 [−0.036, +0.050]
DrGRPO ($s = 5$)	MathIR L3	0.023/0.018	0.026/0.018	0.004/0.000	−0.008 [−0.065, +0.038]
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.254/0.246	0.359/0.360	0.105/0.114	+0.001 [−0.123, +0.095]
Re:Dr ($s = 5$)	MathIR L3	0.064/0.056	0.064/0.056	0.000/0.000	−0.008 [−0.099, +0.059]
DrGRPO ($s = 2$)	Pantry L2	0.332/0.323	0.718/0.736	0.386/0.414	+0.018 [−0.422, +0.433]
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.440/0.419	0.857/0.955	0.416/0.537	+0.099 [−0.201, +0.392]
Re:Dr ($s = 2$)	Pantry L2	0.141/0.147	0.415/0.429	0.274/0.282	+0.014 [−0.123, +0.152]
DrGRPO ($s = 2$)	Pantry L3	0.268/0.258	0.949/1.025	0.681/0.766	+0.076 [−0.237, +0.465]
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.259/0.384	0.927/1.078	0.668/0.694	+0.151 [−0.109, +0.399]
Re:Dr ($s = 2$)	Pantry L3	0.204/0.149	0.568/0.461	0.364/0.312	−0.107 [−0.321, +0.098]

Table 35: Eight-to-64 discovery gains under frozen normalization sensitivity. $G_P = P_{64} - P_8$, $G_D = D_{64} - D_8$, and $G_B = B_{64} - B_8$. Each O/N entry gives original/neutral; ΔG_D is neutral minus original with its pointwise 95% interval. $s = 1$ denotes one fixed initial checkpoint, not training-seed replication. Five-seed intervals resample seeds and paired prompts; two-seed Pantry intervals condition on the observed checkpoints and retain all paired prompt outcomes.

Model	Cell	G_P : O/N	G_D : O/N	G_B : O/N	ΔG_D [95%]
DrGRPO ($s = 5$)	Python factors L2	0.000/0.000	0.000/0.000	0.000/0.000	+0.000 [+0.000, +0.000]
Initial Qwen 0.5B ($s = 1$)	Python factors L2	0.147/0.105	0.303/0.165	0.156/0.060	-0.138 [-0.217, -0.061]
Re:Dr ($s = 5$)	Python factors L2	0.000/0.231	0.058/0.463	0.058/0.232	+0.405 [+0.185, +0.620]
DrGRPO ($s = 5$)	Python factors L3	0.000/0.033	0.000/0.033	0.000/0.000	+0.033 [+0.000, +0.120]
Initial Qwen 0.5B ($s = 1$)	Python factors L3	0.218/0.096	0.593/0.096	0.376/0.000	-0.497 [-0.766, -0.270]
Re:Dr ($s = 5$)	Python factors L3	0.000/0.239	0.153/0.531	0.153/0.292	+0.379 [+0.111, +0.678]
DrGRPO ($s = 5$)	MathIR L2	0.021/0.017	0.021/0.017	0.000/0.000	-0.004 [-0.068, +0.045]
Initial Qwen 0.5B ($s = 1$)	MathIR L2	0.344/0.235	0.344/0.235	0.000/0.000	-0.109 [-0.383, +0.164]
Re:Dr ($s = 5$)	MathIR L2	0.010/0.011	0.010/0.011	0.000/0.000	+0.001 [-0.036, +0.050]
DrGRPO ($s = 5$)	MathIR L3	0.023/0.018	0.026/0.018	0.004/0.000	-0.008 [-0.065, +0.038]
Initial Qwen 0.5B ($s = 1$)	MathIR L3	0.254/0.246	0.359/0.360	0.105/0.114	+0.001 [-0.123, +0.095]
Re:Dr ($s = 5$)	MathIR L3	0.064/0.056	0.064/0.056	0.000/0.000	-0.008 [-0.099, +0.059]
DrGRPO ($s = 2$)	Pantry L2	0.332/0.323	0.718/0.736	0.386/0.414	+0.018 [-0.422, +0.433]
Initial Qwen 0.5B ($s = 1$)	Pantry L2	0.440/0.419	0.857/0.955	0.416/0.537	+0.099 [-0.201, +0.392]
Re:Dr ($s = 2$)	Pantry L2	0.141/0.147	0.415/0.429	0.274/0.282	+0.014 [-0.123, +0.152]
DrGRPO ($s = 2$)	Pantry L3	0.268/0.258	0.949/1.025	0.681/0.766	+0.076 [-0.237, +0.465]
Initial Qwen 0.5B ($s = 1$)	Pantry L3	0.259/0.384	0.927/1.078	0.668/0.694	+0.151 [-0.109, +0.399]
Re:Dr ($s = 2$)	Pantry L3	0.204/0.149	0.568/0.461	0.364/0.312	-0.107 [-0.321, +0.098]

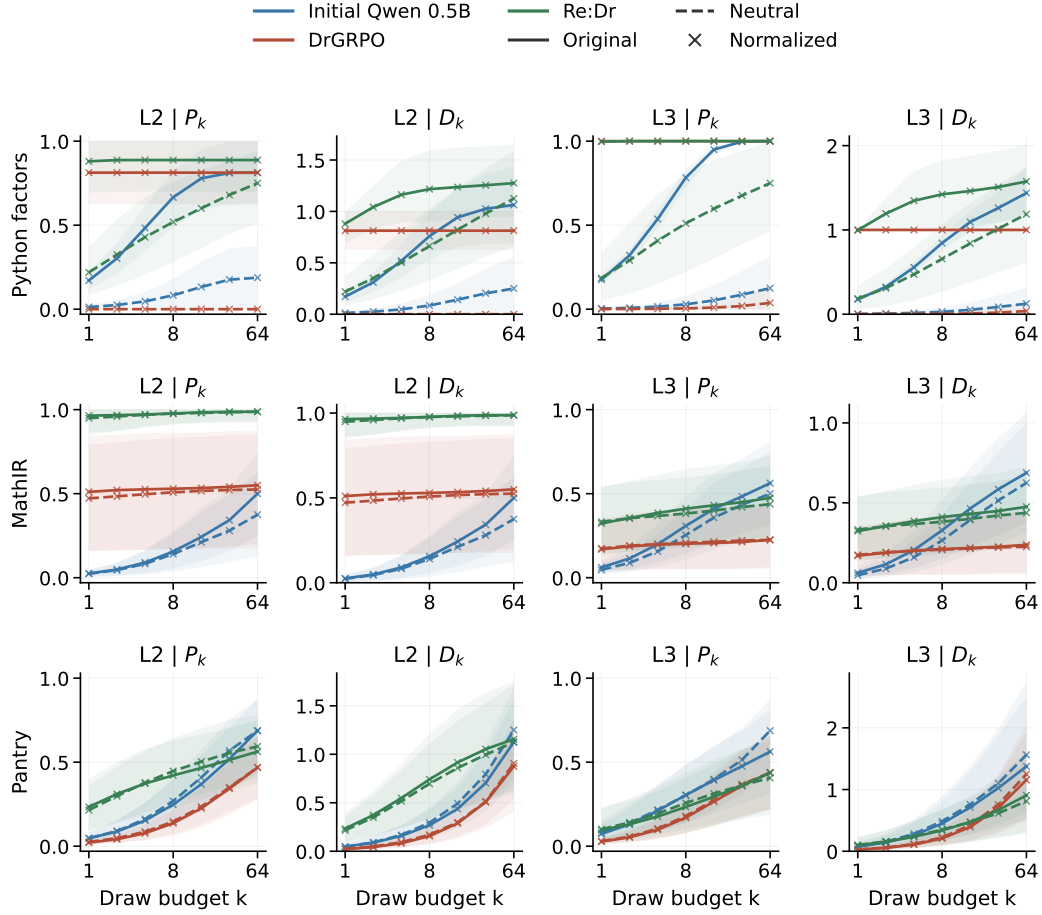


Figure 30: Fresh 64-draw discovery curves. Separate axes show P_k and D_k for each level and domain; pass probability always uses a 0–1 scale. Color identifies the checkpoint method or hosted model, solid/dashed lines original/neutral wording. Lines and shaded pointwise intervals use strict grading; crosses show the frozen-normalization sensitivity. Prefix estimates are separately retained in the report with pointwise intervals for every model and cell.

Table 36: Full-64 strict correct-pair collision. C pools correct pairs within a fixed checkpoint or deployment; trained local rows then average checkpoint rates equally across seeds. U applies the same weighting to the certified-support uniform upper reference. Eligible checkpoint–problem groups E and correct-pair counts Q are summed descriptively across checkpoints. These totals do not define the equal-seed mean; separate pooled rates and per-seed counts are retained in the report. A seed with no eligible pair leaves its all-seed mean undefined, with every individual denominator retained.

Model	Cell	C_O	C_N	U_O	U_N	E_O/E_N	Q_O/Q_N
DrGRPO	Python factors L2	1.000	—	0.500	—	65/0	130977/0
Initial Qwen 0.5B	Python factors L2	0.791	0.875	0.500	0.500	13/3	1270/24
Re:Dr	Python factors L2	0.809	0.800	0.500	0.500	71/53	140722/18289
DrGRPO	Python factors L3	1.000	—	0.500	—	80/0	161280/0
Initial Qwen 0.5B	Python factors L3	0.875	1.000	0.500	0.500	16/1	1010/3
Re:Dr	Python factors L3	0.805	0.797	0.500	0.500	80/52	160275/12893
DrGRPO	MathIR L2	1.000	1.000	0.200	0.200	43/42	80766/74119
Initial Qwen 0.5B	MathIR L2	1.000	1.000	0.200	0.200	3/3	64/64
Re:Dr	MathIR L2	1.000	1.000	0.200	0.200	79/79	155016/151485
DrGRPO	MathIR L3	0.980	1.000	0.200	0.200	16/18	24457/25393
Initial Qwen 0.5B	MathIR L3	0.883	0.912	0.200	0.200	7/6	282/181
Re:Dr	MathIR L3	1.000	1.000	0.200	0.200	34/34	46891/50301
DrGRPO	Pantry L2	0.374	0.276	0.047	0.055	9/10	101/193
Initial Qwen 0.5B	Pantry L2	0.849	0.787	0.052	0.053	7/10	212/178
Re:Dr	Pantry L2	0.600	0.613	0.051	0.052	16/18	10144/8510
DrGRPO	Pantry L3	0.203	0.228	0.047	0.049	10/12	134/165
Initial Qwen 0.5B	Pantry L3	0.375	0.313	0.049	0.049	7/6	531/705
Re:Dr	Pantry L3	0.636	0.666	0.051	0.052	10/11	4006/3946

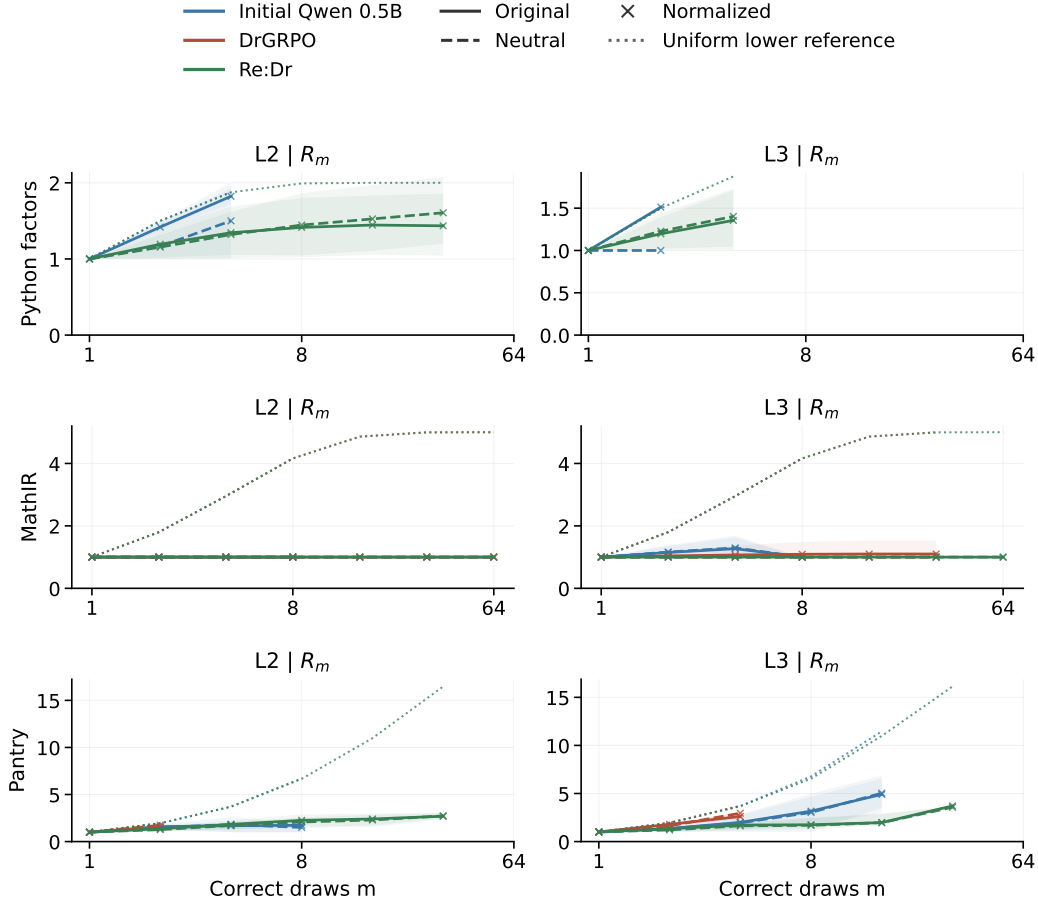


Figure 31: Breadth after exactly m correct draws, on prompts jointly eligible under both wordings. Color and solid/dashed wording styles match the discovery figure; dotted lines give method-specific uniform lower references using the same eligible prompts. Undefined means are gaps. Crosses show frozen normalization. Eligibility varies with m , so curves do not hold the eligible population fixed across the horizontal axis; the following table gives all strict joint counts for every method, level, and correct-draw budget, including groups with zero eligible prompts.

Table 37: Strict joint eligibility for a fixed correct-draw budget: both wordings must have at least m correct draws. Entries count eligible checkpoint–problem groups, summing over displayed seeds; the same 16 problems recur across checkpoints. Arm-specific eligibility at every m is also retained in the report for both prompt wordings and every registered checkpoint.

Model	Cell	$m = 1$	$m = 2$	$m = 4$	$m = 8$	$m = 16$	$m = 32$	$m = 64$
DrGRPO ($s = 5$)	Python factors L2	0	0	0	0	0	0	0
Initial Qwen 0.5B ($s = 1$)	Python factors L2	3	3	2	0	0	0	0
Re:Dr ($s = 5$)	Python factors L2	59	53	44	36	28	13	0
DrGRPO ($s = 5$)	Python factors L3	3	0	0	0	0	0	0
Initial Qwen 0.5B ($s = 1$)	Python factors L3	2	1	0	0	0	0	0
Re:Dr ($s = 5$)	Python factors L3	60	52	44	35	26	7	0
DrGRPO ($s = 5$)	MathIR L2	42	42	40	40	39	37	30
Initial Qwen 0.5B ($s = 1$)	MathIR L2	4	3	3	1	0	0	0
Re:Dr ($s = 5$)	MathIR L2	79	79	78	76	76	75	70
DrGRPO ($s = 5$)	MathIR L3	15	14	14	14	14	14	6
Initial Qwen 0.5B ($s = 1$)	MathIR L3	8	6	6	2	0	0	0
Re:Dr ($s = 5$)	MathIR L3	32	30	28	25	24	23	13
DrGRPO ($s = 2$)	Pantry L2	9	4	1	1	0	0	0
Initial Qwen 0.5B ($s = 1$)	Pantry L2	10	7	4	2	0	0	0
Re:Dr ($s = 2$)	Pantry L2	17	15	12	11	10	5	0
DrGRPO ($s = 2$)	Pantry L3	13	9	4	1	0	0	0
Initial Qwen 0.5B ($s = 1$)	Pantry L3	8	6	5	4	2	0	0
Re:Dr ($s = 2$)	Pantry L3	11	8	6	5	4	2	0

J.2 HOSTED DISCOVERY: GUIDANCE, FORMATTING, AND LATE MODES

The completed hosted panel contains 36,864 fresh responses from GPT-5.4, GPT-5.6 Sol, and Grok 4.3: sixteen fixed problems in each of six Python/MathIR/Pantry level cells, two wordings, and 64 draws per condition. The full report retains both grading conventions, all observed failures, ordered-prefix sensitivities, and every conditional-eligibility denominator. These fixed deployment configurations form a separate experiment from the reasoning-disabled main table. The historical Level-3 problems also precede the newly admitted neutral-Python dataset; this follow-up evaluates no checkpoint trained on that successor dataset.

MathIR guidance changes GPT’s correct-solution breadth. At Level 2, both GPT deployments produce at most one verified mode per problem under original wording even with 64 draws. At matched budgets of eight correct draws, neutral wording increases mean breadth by .794 ([.623, .990]) for GPT-5.4 and .872 ([.606, 1.123]) for GPT-5.6 Sol. The Level-3 effects are .204 ([.106, .320]) and .860 ([.563, 1.141]). Grok’s corresponding intervals include zero at both levels. The GPT results therefore survive conditioning on observed correctness; Grok’s intervals leave its corresponding effect unresolved. These pointwise exploratory intervals resample the sixteen paired problems, without replication across hosted model training seeds.

Pantry continues discovering modes after first success. Under neutral wording, GPT-5.4 gains 2.583 and 3.219 verified modes between eight and 64 draws at Levels 2 and 3; Grok gains 3.377 and 4.466. Grok’s empirical pass probability is already 1.0 at eight draws in both cells, so these gains consist entirely of additional modes. At Level 3 its distinct count rises from 3.972 to 8.438. At eight correct draws, neutral minus original breadth is 1.781 ([1.226, 2.338]) for Grok and .934 ([.670, 1.184]) for GPT-5.4; GPT-5.6 Sol’s Level-3 contrast is inconclusive. These observed counts measure finite-sample discovery rather than exhaustive solution coverage.

Python requires the formatting sensitivity. GPT-5.6 Sol’s original-wording strict pass probability remains .1875 at Level 2 and .0625 at Level 3 from eight through 64 draws. The frozen formatting normalizer instead gives $P_{64} = 1.0$ under both wordings at both levels, with all four P_8 values above .989. Thus its low strict scores cannot be interpreted as an absence of valid mathematical solutions. The normalized neutral-minus-original increase in late mode discovery is .681 ([.351, 1.066]) at Level 2 and .370 ([.112, .674]) at Level 3. Both the strict failure pattern and the normalized sensitivity remain visible; formatting rescue is not a change to the verifier or mode key.

J.3 ALL FIVE DOMAINS AND ALL THREE LEVELS AT 512 DRAWS

This original-wording follow-up retains thirty-two fixed problems in each of five domains at Levels 1, 2, and 3. Every problem has 512 responses: 480 problems and 245,760 responses in total. Every request uses GPT-5.6 Sol with medium reasoning, an 8,192-token output limit, and omitted temperature and top- p . Native receipt headers identify the provider-reported snapshot `gpt-5.6-sol-2026-07-09`. Collection occurred in stages; common settings and this header do not establish identical stochastic behavior across collection times.

The main figure applies the frozen formatting normalizer. The strict frozen verifier remains primary and grades the identical complete pools. In fixed draw-index order, the first 64 responses solve 474 of 480 problems after normalization and 418 under strict grading. At 512 responses these counts are 477 and 436, respectively. These ordered prefixes differ from rarefaction at the same budget inside the full pool. Invalid, refused, and truncated answers remain in every sampling denominator; transport retries do not count as extra samples.

For each complete pool of size $N = 512$, with c correct responses and verified-mode multiplicities n_j , we compute

$$P_N(k) = 1 - \frac{\binom{N-c}{k}}{\binom{N}{k}}, \quad D_N(k) = \sum_j \left[1 - \frac{\binom{N-n_j}{k}}{\binom{N}{k}} \right],$$

then average all thirty-two problems equally within each domain and level. All curves use the same complete 512-response pools and budgets $k \in \{1, 2, 4, 8, 16, 32, 64, 128, 256, 512\}$. An

earlier 64-response endpoint need not equal the rarefied $k = 64$ estimate from this expanded pool. The last-doubling contrast is $D_{512}(512) - D_{512}(256)$. Pointwise 95% intervals use 20,000 whole-problem bootstrap replicates within domain and level (seed 20260911); the same resampled problems form both sides of each contrast. Curves and ordered-prefix sensitivities under both grading conventions remain in the accompanying source record.

Domain	Level	Known support (range)	Normalized D/P	Strict D/P
Graph	1	6.97 (4–12)	2.531/100.0	2.531/100.0
Graph	2	6.03 (4–12)	2.625/100.0	2.625/100.0
Graph	3	6.66 (4–12)	1.719/100.0	1.719/100.0
Countdown	1	≥ 4.53 (2–6)	3.469/100.0	3.375/100.0
Countdown	2	≥ 4.40 (3–7)	3.156/100.0	3.094/100.0
Countdown	3	≥ 4.25 (2–8)	2.812/100.0	2.562/100.0
Python	1	≥ 2.00 (2)	3.750/100.0	2.875/100.0
Python	2	≥ 2.00 (2)	1.438/100.0	0.562/43.8
Python	3	≥ 2.00 (2)	1.344/100.0	0.281/28.1
MathIR	1	≥ 5.00 (5)	1.906/93.8	1.906/93.8
MathIR	2	≥ 5.00 (5)	1.000/100.0	1.000/100.0
MathIR	3	≥ 5.00 (5)	0.969/96.9	0.969/96.9
PantryPlan	1	≥ 16.90 (8–39)	6.906/100.0	6.906/100.0
PantryPlan	2	≥ 16.50 (8–39)	4.875/100.0	4.625/100.0
PantryPlan	3	≥ 16.87 (8–40)	5.562/100.0	5.250/100.0

Table 38: **All fifteen domain–level cells at 512 responses per problem.** D/P gives mean distinct verified modes and the percentage of problems solved at least once in the complete 512-response pool. Support gives each cell’s mean and range across thirty-two problems: exact for Graph, a certified lower bound elsewhere. Both grading conventions retain all 480 problems and 245,760 responses.

Domain	Level	Normalized ΔD [95% CI]	Strict ΔD [95% CI]
Graph	1	0.222 [0.142, 0.305]	0.222 [0.142, 0.305]
Graph	2	0.268 [0.170, 0.373]	0.268 [0.170, 0.373]
Graph	3	0.137 [0.059, 0.228]	0.137 [0.059, 0.228]
Countdown	1	0.148 [0.070, 0.243]	0.164 [0.083, 0.259]
Countdown	2	0.218 [0.123, 0.320]	0.235 [0.136, 0.344]
Countdown	3	0.181 [0.090, 0.289]	0.191 [0.097, 0.297]
Python	1	0.592 [0.438, 0.750]	0.492 [0.334, 0.656]
Python	2	0.073 [0.020, 0.135]	0.148 [0.078, 0.227]
Python	3	0.013 [0.000, 0.032]	0.086 [0.031, 0.156]
MathIR	1	0.071 [0.012, 0.153]	0.071 [0.012, 0.153]
MathIR	2	0.002 [0.000, 0.006]	0.002 [0.000, 0.006]
MathIR	3	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
PantryPlan	1	0.622 [0.458, 0.799]	0.622 [0.458, 0.799]
PantryPlan	2	0.702 [0.476, 0.951]	0.702 [0.472, 0.962]
PantryPlan	3	0.719 [0.527, 0.914]	0.678 [0.489, 0.875]

Table 39: **Additional modes in the final doubling from 256 to 512 draws.** The paired contrast $\Delta D = D_{512}(512) - D_{512}(256)$ compares rarefaction means within the final pool, rather than novel keys in the last chronological block. Intervals are pointwise; they neither provide simultaneous coverage across budgets and cells nor establish asymptotic saturation.

Graph support exhausts all fixed-color-compatible assignments. Countdown references enumerate binary expression trees; the validator also accepts unary negation and retains it in canonical keys, so binary counts are lower bounds. Python, MathIR, and Pantry references retain their certified interpretation. Discovered modes may exceed a lower bound without establishing exhaustive coverage of the available modes.

The expansion from sixteen to thirty-two problems per cell followed earlier observations. It retains all 240 original problems and adds the next sixteen per cell in the same outcome-independent SHA256 ranking (seed 20260911). The 122,880 new responses use 512 fresh draws per added problem; earlier eight-draw evaluations of those problems are excluded. Each launched cell retains its complete fixed cohort and all response slots. These results describe finite-budget discovery: small or zero late gains do not rule out rare unseen modes. Source bindings, full curves, strict grades, support references, and ordered-prefix sensitivities remain in the accompanying JSON.

J.4 OBSERVED SAMPLING-BUDGET EFFECTS

The full experiment is complete: 36,864 hosted and 110,592 local responses, 147,456 in total. Each of the six hosted model-wording cohorts has all 6,144 planned draws; strict and normalized analyses retain every response.

The completed local panel contains 110,592 fresh responses from all 25 registered checkpoints. Each trained checkpoint is evaluated in its own domain at both levels; the initial checkpoint covers all six cells. Serial regrading corrects one original Python grade from failure to success, confirmed once in an independent warmed worker using the unchanged verifier and timeouts. The saved receipt does not identify the cause of the original discrepancy. No response is rescued by formatting normalization, so strict and normalized analyses coincide throughout this panel. The original responses and grades remain available alongside the correction audit and source identities.

MathIR: concentration largely persists through 64 draws. At Level 2, both trained methods have $B_k = 0$ at every measured budget under both wordings. Re:Dr has original $P_8 = .9774$ and $P_{64} = .9875$; neutral wording gives .9767 and .9875. Its D_k equals P_k throughout, so the small late gain consists entirely of first successes. At exactly eight correct draws, both wordings have $R_8 = 1$ on the same 76 eligible checkpoint-problem groups, versus the certified-support uniform lower reference $5[1 - (4/5)^8] = 4.1611$. These groups reuse the same 16 problems across five checkpoints. At Level 3, all neutral trained groups and all original Re:Dr groups also contain at most one observed correct key. There is one exception in original DrGRPO: a single problem at seed 46 contains two keys with counts 28 and 9, giving mean $B_{64} = .0125$ and $B_{64} - B_8 = .0035$. Its equal-seed correct-pair collision is .9803; the other trained MathIR cells have collision 1.0, against the uniform upper reference .2. Thus the larger budget reveals a rare additional mode without supporting an exact single-mode claim for every trained condition.

Python: prompt dependence and late discovery coexist. Original DrGRPO has unchanged $P_k = D_k = .8125$ at Level 2 and 1.0 at Level 3 from eight through 64 draws. Neutral DrGRPO has zero correct responses out of 5,120 at Level 2 and three out of 5,120 at Level 3. Each of those three successes occurs alone in its checkpoint-problem group, so Level 3 mean $P_{64} = D_{64} = .0375$ while correct-pair collision remains undefined. Re:Dr behaves differently. Under original wording its pass probability is already unchanged from eight to 64, but D_k gains .0582 at Level 2 and .1529 at Level 3. Under neutral wording, P_k rises from .5186 to .7500 and from .5109 to .7500, respectively; D_k rises from .6618 to 1.1250 and from .6561 to 1.1875. These final mode counts remain below the original 1.2750 and 1.5750. The corresponding additional-mode gains are .2318 [.0574, .4565] and .2923 [.0659, .5445], with the registered seed-and-prompt intervals. Larger budgets therefore recover both first successes and additional verified alternatives in the neutral replay condition. These unconditional gains do not identify a change in the conditional distribution of correct solutions by themselves.

Pantry: eight draws understate observed discovery. Across the trained Pantry cells and both wordings, $D_{64} - D_8$ ranges from .4151 to 1.0247, with additional-mode gains ranging from .2739 to .7664. The initial checkpoint also gains .8565–1.0778 modes. In the two observed training seeds, Level 3 point estimates even change method ordering: original DrGRPO/Re:Dr have $D_8 = .2075/.3381$ but $D_{64} = 1.1563/.9063$; neutral wording gives .2253/.3515 at eight and 1.2500/.8125 at 64. This is a descriptive ranking of these checkpoints, not evidence of a population-level method advantage. Pantry has only two training seeds, and fixed-correctness comparisons at larger m often have few or no jointly eligible problems. The eligibility table and undefined means remain essential to interpreting the conditional curves.

These observations support persistent concentration in several trained conditions and substantial late discovery in others, which is why no single saturation claim covers all domains: a flat finite-sample curve still cannot exclude rarer unseen modes. The $k = 8$ points rarefy the fresh 64-draw pools on 16 problems per cell rather than restating the previous experiment’s 32-problem scores. The completed hosted panel below adds a separate comparison of fixed deployments on these historical tasks; it is not evidence from newly trained Level-3 checkpoints.

K INFERENCE FOLLOW-UPS: ADAPTATION, MODEL MIXTURES, AND COARSER KEYS

These three analyses require no additional training. Their frozen protocols precede the reported comparisons. Pointwise 95% intervals use 20,000 paired whole-problem bootstrap replicates (seed 20260911); all perturbations, model arms and fixed training seeds move together within a problem. Intervals do not estimate variation over a population of training seeds, and the 21 model pairs are not independent replications. Machine-readable results retain every registered domain, level, pair, wording and grading sensitivity.

K.1 PANTRY PORTFOLIOS UNDER CHANGED REQUIREMENTS

We evaluate the initial Qwen2.5-0.5B checkpoint and Level-2 Dr.GRPO and Re:Dr checkpoints at fixed seeds 43 and 46 on 32 held-out problems. All 192 single-ingredient outages come from task specifications independently of generated answers; exhaustive search of the original quantity grid finds 181 feasible revised tasks. Verified feasibility witnesses are withheld from the model. The five feasible additional dietary restrictions are reported separately because only five problem clusters qualify.

Each portfolio contains eight plans. We revalidate saved plans unchanged, then allow at most eight sequential recovery calls if none survives, stopping at the first valid plan. Unresolved portfolios consume all eight calls. The common recovery interface shows the revised task, initial portfolio, previous recovery outputs and deterministic verifier feedback. Cumulative success includes portfolios that needed no recovery. We average perturbations within each source problem, then weight problems equally.

Controls retain each checkpoint, verifier and token cap. Temperature is chosen from $\{0.7, 1.0, 1.3\}$ by outage survival on 16 separate development problems (384 responses per checkpoint). Diversity prompting makes eight sequential calls, requesting a new support while retaining the complete output history, including failures. These strategies match initial response counts; their context lengths and total token costs differ.

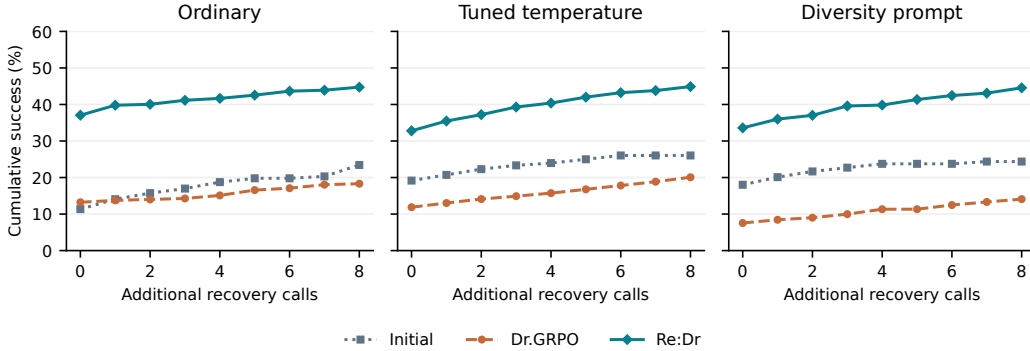


Figure 32: **Recovery after independently chosen feasible Pantry outages.** Zero additional calls measures survival of the eight saved plans. Subsequent points include successful recovery within the displayed call budget. Trained curves average fixed seeds 43 and 46; initial-model curves use one checkpoint. All panels use the same 32 problems; paired uncertainty appears in Table 42.

Table 40: Eight initial plans and up to eight recovery calls under ingredient outages (32 problem clusters). Success columns are percentages. Calls include unresolved cases; token totals charge the initial portfolio once per revised task plus recovery.

Checkpoint	Strategy	Saved	By 8 calls	Calls	Input tokens	Output tokens
Initial	Ordinary	11.35	23.44	6.63	11301.75	376.49
Initial	Tuned temperature	19.17	26.04	6.13	10864.88	370.17
Initial	Diversity prompt	18.02	24.38	6.22	11794.81	352.87
Dr.GRPO 43	Ordinary	9.79	14.01	7.09	12439.22	823.70
Dr.GRPO 43	Tuned temperature	13.44	16.77	6.79	12017.57	775.44
Dr.GRPO 43	Diversity prompt	5.21	8.39	7.44	14103.23	848.09
Dr.GRPO 46	Ordinary	16.67	22.60	6.47	11265.39	637.59
Dr.GRPO 46	Tuned temperature	10.31	23.39	6.75	11531.40	644.94
Dr.GRPO 46	Diversity prompt	9.90	19.79	6.90	13124.10	731.00
Re:Dr 43	Ordinary	39.38	47.97	4.47	9078.92	297.98
Re:Dr 43	Tuned temperature	28.23	40.94	5.19	9718.01	310.50
Re:Dr 43	Diversity prompt	36.93	47.19	4.68	10465.22	344.02
Re:Dr 46	Ordinary	34.74	41.51	4.94	9547.16	308.52
Re:Dr 46	Tuned temperature	37.34	48.85	4.53	9148.77	300.05
Re:Dr 46	Diversity prompt	30.26	41.93	5.06	10584.02	313.50

Table 41: Eight initial plans and up to eight recovery calls under additional dietary restrictions (five problem clusters). Success columns are percentages. Calls include unresolved cases; token totals charge the initial portfolio once per revised task plus recovery.

Checkpoint	Strategy	Saved	By 8 calls	Calls	Input tokens	Output tokens
Initial	Ordinary	0.00	20.00	6.80	11392.80	371.20
Initial	Tuned temperature	20.00	40.00	6.40	11175.20	387.20
Initial	Diversity prompt	0.00	0.00	8.00	13630.20	405.00
Dr.GRPO 43	Ordinary	0.00	0.00	8.00	13310.20	849.20
Dr.GRPO 43	Tuned temperature	0.00	0.00	8.00	13394.60	856.00
Dr.GRPO 43	Diversity prompt	0.00	0.00	8.00	15045.40	939.60
Dr.GRPO 46	Ordinary	0.00	20.00	7.40	12063.00	650.40
Dr.GRPO 46	Tuned temperature	0.00	0.00	8.00	12544.20	615.40
Dr.GRPO 46	Diversity prompt	0.00	0.00	8.00	14132.40	725.60
Re:Dr 43	Ordinary	20.00	60.00	5.20	9732.60	326.00
Re:Dr 43	Tuned temperature	40.00	40.00	4.80	9402.80	307.60
Re:Dr 43	Diversity prompt	40.00	40.00	4.80	10693.60	355.60
Re:Dr 46	Ordinary	40.00	80.00	3.20	7853.20	271.20
Re:Dr 46	Tuned temperature	40.00	80.00	3.60	8127.80	265.20
Re:Dr 46	Diversity prompt	20.00	40.00	5.00	10738.20	341.00

Table 42: Re:Dr minus Dr.GRPO under outages, averaging fixed seeds 43 and 46. Survival and final-success changes are percentage points. Intervals resample 32 paired problems.

Strategy	Δ saved	Δ by 8 calls	Δ calls
Ordinary	23.83 [12.16, 35.89]	26.43 [14.69, 38.18]	-2.08 [-3.03, -1.14]
Tuned temperature	20.91 [9.19, 32.89]	24.82 [14.04, 35.52]	-1.91 [-2.78, -1.05]
Diversity prompt	26.04 [16.93, 35.65]	30.47 [21.51, 39.27]	-2.30 [-3.04, -1.57]

Ordinary sampling improves saved survival by 23.83 percentage points [12.16, 35.89], reduces capped recovery burden by 2.08 calls [1.14, 3.03], and improves success within eight extra calls by 26.43 points [14.69, 38.18]. Mean input and output token totals fall by 2,539.27 [1, 618.01, 3, 468.86] and 427.40 [386.83, 466.30], respectively. However, original correctness also increases by 1.234 correct responses out of eight [.578, 1.984]. Conditioning descriptively on equal observed correct counts leaves a 1.05-point survival gain [0.00, 3.53] on 19 eligible problems. This demonstrates adaptation gains under the tested intervention; it does not isolate diversity as their causal mechanism.

Logical input tokens include reused prefixes and do not measure cached compute. Historical ordinary-sampling generation time is unavailable. Per-perturbation totals represent deploying a portfolio once and recovering from one update; they differ from actual collection totals, which count each new response once. Training cost is outside this inference accounting.

Table 43: Selected temperatures and actual collection cost. Calibration is shown separately and is included in the new-collection totals; saved ordinary portfolios require no new initial generation. Seconds measure generation, not training or hardware-normalized compute.

Checkpoint	Scope	T	Responses	Input tokens	Output tokens	Seconds
Dr.GRPO 43	Calibration	1.3	384	229392	19871	136.3
Dr.GRPO 43	New collection	1.3	4874	4870884	272484	3302.0
Dr.GRPO 46	Calibration	1.3	384	229392	15544	126.5
Dr.GRPO 46	New collection	1.3	4634	4365524	220458	2792.9
Re:Dr 43	Calibration	0.7	384	229392	8552	111.1
Re:Dr 43	New collection	0.7	3559	3160284	92227	1527.7
Re:Dr 46	Calibration	1.0	384	229392	8585	112.5
Re:Dr 46	New collection	1.0	3592	3169337	88300	1510.2
Initial	Calibration	1.3	384	229392	9521	123.5
Initial	New collection	1.3	4416	4035069	116619	1957.4

K.2 COMPLEMENTARITY ACROSS HOSTED MODELS

All seven original-protocol cohorts contribute 107,520 responses on the same 1,920 prompts (five domains, three levels, 128 prompts per cell). This primary analysis uses strict grading and the original Opus 5 Python interface; it is a different condition from the revised-Opus, normalized hosted display. Every one of the 21 model pairs is included. We compare random 4+4 portfolios with eight responses from each constituent, taking exact expectations over subsets of the saved draws. An independent audit matches the formula against all 4,900 subsets for each of 252 deterministic pair/prompt examples.

All pairs improve distinct outcomes over the average constituent by .115–.266. Ten improve over both constituents in point estimates; nine have positive pointwise intervals against both. At a fixed successful-response budget, 2+2 correct draws beat the average four-correct-draw constituent for all pairs (.073–.177 additional outcomes); eligibility varies by pair. These conditional comparisons separate some correctness effects without identifying unobserved support or a causal training mechanism.

Table 44: Every 4+4 mixture versus its average eight-response constituent. Fine and coarse columns use all 1,920 prompts; correctness matching compares 2+2 correct responses with four from each constituent on the eligible subset. Entries are distinct-mode differences with pointwise 95% intervals.

Pair	Fine		Coarse		Correctness matched	n
DeepSeek V4 Pro + Kimi K3	0.148	[0.136, 0.160]	0.142	[0.131, 0.154]	0.107 [0.098, 0.115]	1858
DeepSeek V4 Pro + Opus 4.8	0.198	[0.185, 0.211]	0.183	[0.171, 0.197]	0.153 [0.142, 0.164]	1847
DeepSeek V4 Pro + Opus 5	0.262	[0.249, 0.275]	0.252	[0.240, 0.265]	0.177 [0.165, 0.190]	1510
DeepSeek V4 Pro + GPT-5.4	0.133	[0.122, 0.143]	0.127	[0.116, 0.138]	0.098 [0.090, 0.107]	1695
DeepSeek V4 Pro + GPT-5.6 Sol	0.235	[0.223, 0.247]	0.224	[0.212, 0.237]	0.154 [0.142, 0.166]	1396
DeepSeek V4 Pro + Grok 4.3	0.123	[0.112, 0.134]	0.117	[0.106, 0.127]	0.091 [0.083, 0.099]	1859
Kimi K3 + Opus 4.8	0.187	[0.174, 0.200]	0.172	[0.159, 0.185]	0.138 [0.127, 0.148]	1885
Kimi K3 + Opus 5	0.266	[0.253, 0.279]	0.255	[0.242, 0.268]	0.162 [0.150, 0.175]	1543
Kimi K3 + GPT-5.4	0.127	[0.115, 0.139]	0.112	[0.101, 0.124]	0.083 [0.075, 0.092]	1720
Kimi K3 + GPT-5.6 Sol	0.202	[0.190, 0.214]	0.195	[0.183, 0.207]	0.093 [0.083, 0.104]	1417
Kimi K3 + Grok 4.3	0.115	[0.104, 0.126]	0.110	[0.099, 0.121]	0.073 [0.066, 0.080]	1900
Opus 4.8 + Opus 5	0.187	[0.176, 0.199]	0.182	[0.171, 0.193]	0.098 [0.087, 0.109]	1534
Opus 4.8 + GPT-5.4	0.201	[0.186, 0.215]	0.185	[0.172, 0.199]	0.156 [0.144, 0.168]	1713
Opus 4.8 + GPT-5.6 Sol	0.247	[0.234, 0.261]	0.237	[0.223, 0.250]	0.151 [0.138, 0.164]	1409
Opus 4.8 + Grok 4.3	0.180	[0.167, 0.194]	0.171	[0.158, 0.184]	0.141 [0.130, 0.153]	1894
Opus 5 + GPT-5.4	0.228	[0.215, 0.241]	0.217	[0.204, 0.230]	0.158 [0.145, 0.171]	1493
Opus 5 + GPT-5.6 Sol	0.169	[0.157, 0.182]	0.161	[0.149, 0.174]	0.143 [0.130, 0.156]	1370
Opus 5 + Grok 4.3	0.231	[0.219, 0.244]	0.225	[0.213, 0.237]	0.145 [0.133, 0.157]	1547
GPT-5.4 + GPT-5.6 Sol	0.156	[0.145, 0.167]	0.147	[0.136, 0.158]	0.094 [0.084, 0.104]	1400
GPT-5.4 + Grok 4.3	0.130	[0.119, 0.142]	0.121	[0.111, 0.132]	0.095 [0.086, 0.104]	1722
GPT-5.6 Sol + Grok 4.3	0.211	[0.199, 0.224]	0.204	[0.192, 0.216]	0.118 [0.107, 0.130]	1418

Table 45: Mixtures versus both eight-response constituents, A and B in the displayed order. The final column reports pass@8 differences from the average constituent (percentage points).

Pair	ΔD vs A	ΔD vs B	Δ pass@8 vs mean
DeepSeek V4 Pro + Kimi K3	0.279 [0.254, 0.304]	0.017 [-0.008, 0.043]	0.22 [0.08, 0.39]
DeepSeek V4 Pro + Opus 4.8	0.089 [0.064, 0.114]	0.307 [0.282, 0.332]	0.31 [0.15, 0.49]
DeepSeek V4 Pro + Opus 5	-0.061 [-0.082, -0.039]	0.585 [0.560, 0.610]	7.48 [7.19, 7.78]
DeepSeek V4 Pro + GPT-5.4	0.097 [0.074, 0.121]	0.168 [0.145, 0.192]	0.64 [0.41, 0.90]
DeepSeek V4 Pro + GPT-5.6 Sol	-0.003 [-0.025, 0.019]	0.473 [0.447, 0.500]	7.16 [6.71, 7.62]
DeepSeek V4 Pro + Grok 4.3	0.059 [0.037, 0.082]	0.186 [0.163, 0.210]	0.34 [0.17, 0.52]
Kimi K3 + Opus 4.8	-0.053 [-0.080, -0.027]	0.427 [0.401, 0.452]	0.19 [0.07, 0.32]
Kimi K3 + Opus 5	-0.188 [-0.212, -0.165]	0.719 [0.692, 0.747]	7.34 [7.08, 7.61]
Kimi K3 + GPT-5.4	-0.040 [-0.064, -0.015]	0.293 [0.267, 0.320]	0.73 [0.48, 1.00]
Kimi K3 + GPT-5.6 Sol	-0.167 [-0.191, -0.144]	0.571 [0.543, 0.600]	7.38 [6.92, 7.85]
Kimi K3 + Grok 4.3	-0.080 [-0.104, -0.056]	0.309 [0.284, 0.335]	0.12 [0.03, 0.23]
Opus 4.8 + Opus 5	-0.027 [-0.045, -0.009]	0.401 [0.379, 0.423]	7.42 [7.14, 7.71]
Opus 4.8 + GPT-5.4	0.274 [0.248, 0.300]	0.127 [0.101, 0.153]	0.93 [0.65, 1.24]
Opus 4.8 + GPT-5.6 Sol	0.118 [0.096, 0.141]	0.377 [0.350, 0.403]	7.51 [7.03, 7.99]
Opus 4.8 + Grok 4.3	0.226 [0.202, 0.250]	0.135 [0.110, 0.160]	0.11 [0.02, 0.23]
Opus 5 + GPT-5.4	0.515 [0.489, 0.541]	-0.060 [-0.082, -0.037]	6.30 [5.90, 6.71]
Opus 5 + GPT-5.6 Sol	0.254 [0.232, 0.277]	0.084 [0.063, 0.106]	2.11 [1.66, 2.57]
Opus 5 + Grok 4.3	0.490 [0.467, 0.514]	-0.028 [-0.048, -0.007]	7.41 [7.13, 7.69]
GPT-5.4 + GPT-5.6 Sol	-0.046 [-0.067, -0.025]	0.359 [0.335, 0.383]	5.61 [5.15, 6.08]
GPT-5.4 + Grok 4.3	0.102 [0.078, 0.127]	0.159 [0.135, 0.182]	0.93 [0.65, 1.23]
GPT-5.6 Sol + Grok 4.3	0.386 [0.360, 0.412]	0.037 [0.015, 0.058]	7.55 [7.08, 8.04]

Shared preferences remain substantial: on joint prompts with at least two correct draws per model, cross-model collision is .624–.740, compared with .704–.819 for mean within-model collision. These are equal-prompt averages, not pooled pair counts. The full record stratifies by domain and level and provides exact-support references or bounds from verified support witnesses. Absence of a shared observed key does not establish disjoint true supports. All average-constituent gains remain positive under coarser keys and the frozen normalization sensitivity. Equal response counts do not match provider tokens, compute, monetary cost or reasoning interfaces.

K.3 SENSITIVITY TO TASK-RELEVANT EQUIVALENCES

Graph groups global color renamings that fix every anchored color; vertex identities remain fixed. Python groups opposite members d and n/d of the same unordered proper factor pair, retaining input-vector order. Countdown flattens associative additions and multiplications and sorts their children, preserving operands, multiplicity, and subtraction/division boundaries. MathIR keeps its simplified state trajectory. Pantry keeps ingredient supports, whose differences can matter under outages. Every map is frozen from task semantics and applied consistently to authenticated correct keys. The independent audit verifies all 6,574 unique successful hosted prompt/key combinations. Correctness is unchanged; deterministic merging cannot increase observed support or reduce correct-pair collision.

Uniform fine-key mass generally induces nonuniform coarse-group mass. Graph/Python references are recomputed from exact groups or coarsened verified witnesses; no unsupported numerical Countdown reference is retained. A uniform distribution over coarse groups is a separate hypothetical policy.

Table 46: Hosted coarse-key sensitivity under strict original-protocol grading. Distinct counts include all 1,920 prompts; collision conditions on at least two correct responses. Intervals are pointwise.

Model	Fine D@8	Coarse D@8	Fine collision	Coarse collision
DeepSeek V4 Pro	1.826 [1.792, 1.859]	1.762 [1.729, 1.795]	0.720 [0.710, 0.731]	0.739 [0.729, 0.750]
Kimi K3	2.087 [2.047, 2.129]	1.952 [1.913, 1.992]	0.688 [0.678, 0.699]	0.723 [0.712, 0.733]
Opus 4.8	1.608 [1.575, 1.641]	1.556 [1.525, 1.588]	0.795 [0.785, 0.805]	0.811 [0.801, 0.821]
Opus 5	1.180 [1.155, 1.206]	1.152 [1.128, 1.177]	0.856 [0.845, 0.866]	0.867 [0.857, 0.877]
GPT-5.4	1.755 [1.718, 1.792]	1.679 [1.644, 1.715]	0.729 [0.717, 0.741]	0.750 [0.739, 0.762]
GPT-5.6 Sol	1.349 [1.317, 1.382]	1.312 [1.281, 1.345]	0.778 [0.765, 0.790]	0.794 [0.782, 0.806]
Grok 4.3	1.698 [1.665, 1.732]	1.645 [1.613, 1.679]	0.791 [0.782, 0.800]	0.807 [0.798, 0.816]

Table 47: Local Re:Dr minus Dr.GRPO endpoint probes: 32 problems per domain/level and fixed seed averages. Level-2 trained checkpoints are evaluated at Levels 2 and 3; this is not a reanalysis of every main-paper trajectory. MathIR/Pantry definitions are unchanged.

Domain	Level	Wording	Seeds	Fine $\Delta D@8$	Coarse $\Delta D@8$
Python	2	original	5	0.431 [0.406, 0.456]	0.075 [0.025, 0.138]
MathIR	2	original	5	0.463 [0.387, 0.531]	0.463 [0.387, 0.531]
Pantry	2	original	2	0.469 [0.203, 0.750]	0.469 [0.203, 0.750]
Python	3	original	5	0.419 [0.400, 0.444]	0.000 [0.000, 0.000]
MathIR	3	original	5	0.119 [0.019, 0.225]	0.119 [0.019, 0.225]
Pantry	3	original	2	0.125 [-0.109, 0.375]	0.125 [-0.109, 0.375]
Python	2	neutral	5	0.675 [0.519, 0.838]	0.556 [0.438, 0.675]
MathIR	2	neutral	5	0.469 [0.375, 0.562]	0.469 [0.375, 0.562]
Pantry	2	neutral	2	0.484 [0.234, 0.766]	0.484 [0.234, 0.766]
Python	3	neutral	5	0.656 [0.537, 0.775]	0.500 [0.419, 0.575]
MathIR	3	neutral	5	0.119 [0.025, 0.213]	0.119 [0.025, 0.213]
Pantry	3	neutral	2	0.250 [-0.031, 0.562]	0.250 [-0.031, 0.562]

The original-wording Python Level-3 gain falls from .419 to zero; Level 2 falls from .431 to .075. Neutral-wording coarse gains remain .556 at Level 2 and .500 at Level 3. Thus the original Python effect can reflect opposite members of the same factor pair, and the conclusion depends on wording and the meaning of output identity. Neither divisor vectors nor factor-pair vectors identify different algorithms.

Reproduction. `ops/build_paper_inference_followups.py` checks frozen analysis hashes, reconstructs Pantry means and stopping identities from per-problem records, and renders these tables and curves. The accompanying `results/inference_followups_20260912.json` binds the protocols, complete analysis records, paper values and figure. Full domain/level, per-seed, normalization, overlap and cost details remain in the bound analysis files.

Part V. Fixed deployments

L INFERENCE-ONLY CONCENTRATION IN A HOSTED MODEL

This exploratory evaluation measures the current output distribution of GPT-5.6 Sol under the inherited MODEBENCH interface. It is separate from the matched training interventions: it adds no frontier-model training, replay comparison, or controlled model-size contrast. Only the completed, audited deployment enters these tables.

Protocol and provenance. The September 11, 2026 run uses the Azure deployment `gpt-5.6-sol`; saved provider headers identify snapshot `gpt-5.6-sol-2026-07-09`. Each of the five domains contributes 128 held-out prompts at each of three levels. Eight stateless requests per prompt yield 15,360 responses. Requests use medium reasoning, an 8,192-token output limit, no tools and no conversation state. The service returns temperature 1.0 and top- p .98; neither setting was overridden. These settings and the hosted natural-language interface differ from the local training evaluation’s token-level grammar constraints.

Level 1 uses the native E118 evaluation splits retained in E124. Level 2 uses the frozen matched r5 construction; Level 3 uses the matched v3 construction admitted by the completed canonical v3 confirmation. Level 3 is an exploratory test extension here, not a third level of the paper’s training intervention. All cells use the multi-answer split. The Level-2/Level-3 support histograms match a separate fixed-reference Level-1 confirmation reserve, which differs from the native Level-1 split in Graph, Countdown and Python. Prompts differ across levels and are not paired by row index. Calibration against small-model references does not guarantee increasing difficulty for this hosted model.

Exact prompts, dataset identities and hashes, requests, provider-exposed responses, grades, retry receipts, usage and verifier snapshots are retained in the experiment’s evidence directory. All 15,360 intended samples completed, with distinct response identifiers; 107 failed API attempts were retained and recovered by retries. Provider random-number independence and hidden reasoning contents cannot be audited from the exposed responses.

Table 48: **Hosted correctness and concentration under strict grading.** GPT-5.6 Sol, with medium reasoning. Each level weights five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is the percentage of within-prompt correct pairs sharing a mode, pair weighted within each domain. Brackets give pointwise 95% intervals from whole-prompt resampling.

Level	M	P	D	C (95% interval)
1	83.0	95.8	1.71	74.0 [71.9, 76.1]
2	62.9	77.5	1.16	82.3 [80.3, 84.4]
3	68.7	80.3	1.18	84.5 [82.8, 86.2]

Metrics and uncertainty. Per-response correctness is mean@8 in this paper (called pass@1 in the hosted report). Raw distinct@8 counts verified keys across eight draws, including zero for unsolved prompts. For prompt x , let c_x be its number of correct draws and n_{xc} its count for correct key c . The within-cell correct-pair collision estimate is

$$\hat{C} = \frac{\sum_x \sum_c \binom{n_{xc}}{2}}{\sum_x \binom{c_x}{2}}.$$

It describes concentration conditional on successful draws, with prompts weighted by their number of correct pairs. Level summaries average the five domain estimates equally. When the certified correct-key count m_x is available, a uniform-correct-key reference replaces the numerator by $\sum_x \binom{c_x}{2} / m_x$. This differs from a uniform-action baseline and is a descriptive reference, not a normative sampling requirement. Countdown’s verifier accepts modes beyond its enumerated binary-expression library, including unary negation, so its total support is unknown and this reference is omitted. Intervals use 2,000 whole-prompt bootstrap replicates, seed 20260911, resampling within each domain and level and keeping all eight draws together. They are pointwise descriptive intervals.

Table 49: **GPT-5.6 Sol under strict executable grading.** Every cell contains 128 prompts and 1,024 responses. M and P are mean@8 and pass@8 (%); D is distinct@8. C is correct-pair collision (%), with 95% prompt-bootstrap interval); U is the uniform-correct-key collision reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

Formatting sensitivity. Strict grades remain primary. A secondary typography normalizer was introduced after inspecting the first 15 responses. It fixes a frozen set of presentation conventions, including escaped Python/LaTeX syntax, and reuses the original executable validators without repairing calculations or program logic. It recovers 2,580 additional verified outputs; strict successes and their keys are retained. This is a post hoc diagnostic, not a replacement benchmark. All 3,072 primary Python responses passed an independent serial recheck with no grade changes. One cold-start timeout in the secondary grading cache was reconciled with repeated serial verification; the original cache and correction audit remain preserved.

Table 50: **Post hoc formatting-normalized diagnostic.** Same samples and metric definitions as Table 49, with fixed presentation normalization before executable verification. The strict table remains primary; normalization was designed after the first 15 responses.

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Concentration and level. The strict five-domain collision average is 74.0%, 82.3% and 84.5% at Levels 1–3; the normalized diagnostic gives 74.1%, 81.2% and 83.6%. This aggregate pattern is not a monotonic law within domains. Graph collision is 89.7%, 83.3%, then 95.1%; normalized PantryPlan collision is 46.9%, 64.0%, then 53.0%. Normalized overall correctness also rebounds from 83.2% at Level 2 to 85.7% at Level 3. Thus a higher level label is not equivalent to a measured increase in difficulty for this model, and raw distinct-mode counts also change with correctness and support.

Level-3 Graph separates concentration from failed responses particularly clearly: all 1,024 outputs verify, eight draws produce 1.16 distinct keys on average, and 3,407 of 3,584 cor-

rect pairs share a key (95.1%). The uniform-correct-key reference is 19.0%; the excess is 76.1 percentage points (95% interval [73.5, 78.3]). Graph was selected after the GPT run for this illustration because its correctness is high, support is certified and formatting normalization changes no scores; both tables report all domains.

Interface and claim limits. Level-2/Level-3 MathIR instructions prescribe moving the right-side variable term left, removing the constant and then dividing. Python instructions suggest a nested small-divisor conditional chain, and PantryPlan instructions prefer particular ingredient families. These preferences can concentrate the very keys being measured. In addition, Level-1 PantryPlan requests a support mask whose quantities are supplied by a trusted projection; Levels 2–3 ask the model to provide quantities. MathIR verifies symbolic action compliance, which can reject a numerically valid shortcut that omits a required symbolic step. Neither these failures nor the cross-level differences isolate numerical reasoning ability. Canonical keys describe executed outputs, not hidden reasoning strategies. Eight draws cannot establish that an unseen mode has zero probability. The evidence supports concentration under the specified prompts and deployment configuration; it identifies neither a training cause nor a causal difficulty or model-size effect.

Reproduction record. The source record is `artifacts/frontier_modebench_gpt56sol_20260911/summary.json`. Its hash, complete cell statistics and run settings are copied into `results/frontier_hosted_20260911.json`; adjacent generated table bodies supply the displayed values. The evidence directory also retains `completion_audit.json`, `prompt_contract_audit.json`, the original requests and responses, and `interpretation_notes.md`. Hosted-model results are separate from the training-checkpoint archive and its matched-seed comparisons.

L.1 HOSTED CONCENTRATION ON THE SUCCESS-CONDITIONAL AXIS

Table 51 reports PCMD for the same audited responses. The hosted tables above report `correct_pair_collision`, which is one minus PCMD pooled over pairs; the values here instead average over prompts without weighting by how many verified pairs each contributed, matching the aggregation used everywhere else in this paper. Pooling by pairs lets the prompts a model solves most often dominate the estimate, which is the accuracy coupling PCMD exists to avoid. The two readings agree closely, and the pooled reading recomputed from the per-sample records reproduces the published collision estimate exactly, which checks the two pipelines against each other.

This deployment is highly accurate and, on three of five domains, close to deterministic in which verified answer it gives. MathIR reaches PCMD .000 at Levels 2 and 3: across every prompt with two or more verified draws, no two verified answers ever differed. Level-3 Graph sits at .049 against six available colorings. PantryPlan is the consistent exception. Python at Levels 2 and 3 is left blank because the model clears two verified draws on only 14 and 15 of 128 prompts there, which is too few to estimate breadth.

Table 51: **Hosted verified-mode diversity, GPT-5.6 Sol at medium reasoning.** Each entry is PCMD with the effective number of verified modes $N_{\text{eff}}^{(2)} = 1/(1 - \text{PCMD})$ in parentheses, over 128 held-out prompts per cell and eight stateless requests per prompt. An em dash marks a cell in which fewer than 30 of those prompts return two verified responses, leaving PCMD unestimable there.

Level	Graph	Countdown	Python	MathIR	Pantry
1	.106 (1.12)	.315 (1.46)	.134 (1.15)	.196 (1.24)	.531 (2.13)
2	.166 (1.20)	.366 (1.58)	—	.000 (1.00)	.386 (1.63)
3	.049 (1.05)	.228 (1.30)	—	.000 (1.00)	.455 (1.84)

The Level-4 probe. Fig. 3 carries one Level-4 point per domain for GPT-5.6 Sol, at PCMD .181 over 5 domains. Two things separate it from every other hosted cell and it is read on its own account for both of them. It was served direct through OpenAI rather than the Azure deployment that served the other levels, which may be a different build; the served model string is recorded per response rather than assumed. And Python is absent from it because strict grading rejects the LaTeX-escaped Python this deployment emits: two of its 1,024 draws verify strictly, while the

frozen formatting normalizer recovers 675. That is a presentation artifact rather than a capability measurement, and it affects the Levels 1–3 Python cells in the same direction.

Table 52 extends the measurement to the whole completed cohort. Every one of the seven deployments is concentrated on this axis, and none reaches 1.5 effective verified modes: macro PCMD runs from .142 for Claude Opus 5 to .311 for Kimi K3, or $N_{\text{eff}}^{(2)}$ between 1.17 and 1.45. The ordering does not follow correctness, and it is not stable across domains — GPT-5.4 and Kimi K3 lead on Python while DeepSeek V4 Pro leads on Graph — so this is a property of individual deployments rather than a single frontier-wide constant. What is common to all of them is the level: the most varied deployment measured still repeats its own verified mode on roughly seven of every ten pairs of correct answers. Every model also declines from Level 1 to Level 3.

Table 52: **Verified-mode diversity across seven hosted deployments.** Macro PCMD averages the cells that clear the support bar, weighting the five domains and three levels equally, with $N_{\text{eff}}^{(2)} = 1/(1 - \text{PCMD})$ beside it and the per-level macro to its right. Each deployment contributes 128 held-out prompts per cell and eight stateless requests per prompt under its own default sampling. GPT-5.6 Sol and Claude Opus 5 clear the bar on 13 of 15 cells; the other five clear all 15.

Deployment	Macro PCMD	$N_{\text{eff}}^{(2)}$	Macro PCMD by level		
			L1	L2	L3
Kimi K3	.311	1.45	.374	.307	.253
DeepSeek V4 Pro	.280	1.39	.289	.319	.232
GPT-5.4	.269	1.37	.338	.247	.224
GPT-5.6 Sol	.226	1.29	.256	.229	.183
Grok 4.3	.209	1.26	.275	.179	.171
Claude Opus 4.8	.205	1.26	.217	.229	.168
Claude Opus 5	.142	1.17	.140	.164	.121

M COMPARISON ACROSS HOSTED DEPLOYMENTS

Figure 35 compares verified output breadth across hosted deployments. Its main display applies the frozen formatting normalizer to every deployment and uses the complete revised-wording Opus 5 Python cohort at all three levels. Every response enters its accuracy and mode-count metrics; outputs are not selected by correctness or provider outcome. The other cells use their original prompts. This is a descriptive comparison across the stated task formulations, rather than a common-prompt model ranking.

The tables and Graph figure below preserve every original-protocol result, including failed draws and native refusals. The separate Python comparison reports the original and revised conditions under both grading rules. These records expose the effect of the prompt change without replacing the original evidence. Level 3 remains an exploratory test extension; no hosted-model training or replay intervention is performed. Original deployments enter the comparison only with a passing integrity audit and all 15,360 responses.

Completed hosted comparison. The admitted deployments are GPT-5.6 Sol, Claude Opus 5, GPT-5.4, Grok 4.3, Kimi K3, Claude Opus 4.8, DeepSeek V4 Pro: 7 complete runs, 107,520 responses, and the same 128 held-out prompts per domain and level with eight stateless samples per prompt. Only runs with a passing completion audit and the identical frozen prompt digest enter these tables. No training, model-side tools or conversation history is added.

Provider configurations. GPT-5.6 Sol: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 5: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. GPT-5.4: hosted API, requested reasoning setting medium, native output limit 8192. Grok 4.3: hosted API, requested reasoning setting medium, native output limit 8192. Kimi K3: hosted API, requested reasoning setting medium, native output limit 8192. Claude Opus 4.8: Anthropic Messages, requested reasoning setting medium (requested, provider-specific), native output limit 8192. DeepSeek V4 Pro: hosted API, requested reasoning setting medium, native output limit 8192.

Provider effort labels do not establish equal compute, and native token accounting differs. The saved native request and returned usage determine each configuration; provider defaults remain unspecified whenever native responses omit them.

The appendix retains strict executable grades alongside formatting-normalized scores. The main display uses the same frozen formatting rule for every cell and the complete separately collected revised Opus 5 Python condition; all other displayed cells retain the original request wording. The shared typography diagnostic was developed after the first 15 GPT-5.6 Sol responses and frozen before the prospective comparison runs. It retains its post hoc status for GPT-5.6 Sol. Inherited route preferences and the Level-1 PantryPlan interface change limit cross-level interpretation. Countdown has no certified finite-total-support uniform reference. These data establish neither a training cause nor a causal model-size or difficulty effect. All pair-collision intervals are pointwise; overlapping or nonoverlapping cell intervals are not a paired model-difference test.

Collection interruption: DeepSeek V4 Pro. 13 HTTP-200 bodies had no terminal finish reason or visible answer and were preserved as provider protocol failures. A later storage interruption left 192 logged physical request starts without a saved outcome or usage; their outcome and possible billing remain unknown. The audit registers these attempts separately from the 15,360 terminal draws. Only missing terminal draws were recollected; saved terminal responses, including truncations, were retained. Reported token totals are subtotals of known usage, not zero-cost claims for failed or interrupted attempts. Source-bound recovery sidecars preserve the original receipts and the exact interruption accounting.

M.1 A SEPARATE PYTHON PROMPT SENSITIVITY TEST

After the original Opus 5 Python refusals, synthetic tasks were used to select a direct arithmetic-expression formulation with no system message. We then collected eight independent draws for every original Python test task: all 384 prompts across three levels, with unchanged adaptive thinking, medium effort, 8,192 output tokens, allowed operators, public inputs and frozen graders. The

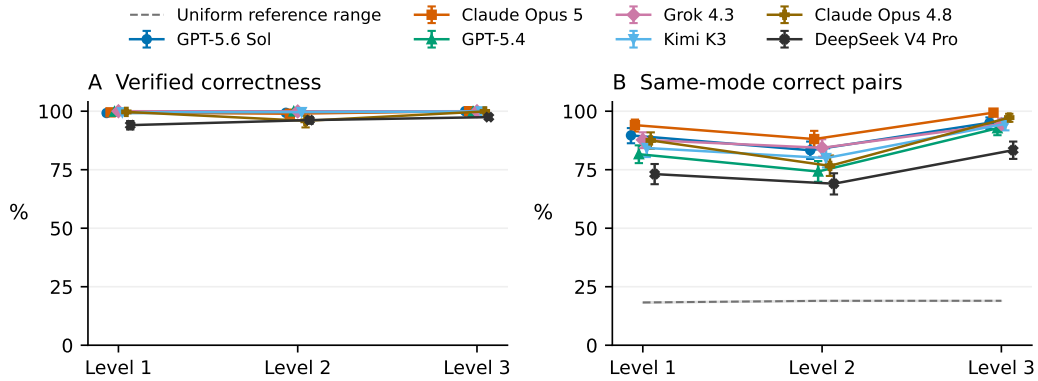


Figure 33: **Verified correctness and concentration in hosted Graph outputs.** Each deployment receives the same 128 prompts per level, with eight stateless samples per prompt. Intervals resample whole prompts. The gray band spans the models’ uniform-correct-key references, conditional on their observed correct draws. Graph was selected after the GPT-5.6 Sol run and before the prospective comparison runs. Level labels do not guarantee increasing difficulty for each model.

revised user instruction asks for a one-line boxed Python lambda returning any proper divisor for each listed input. It does not prescribe a preferred divisor. Both task wording and system-message presence change; collection time also differs. This exploratory follow-up therefore supports a practical prompt repair without isolating its cause. Original refusals and all three residual refusals are retained. The original-protocol appendix tables retain all original Python responses; the main display uses this complete revised condition with an explicit caption note.

Table 53: **Opus 5 Python: original versus plain task wording.** Each row has 1,024 responses. R counts native refusals; A is per-response accuracy (%), D is `distinct@8`, and C is correct-pair collision (%). Strict and normalized results use the same frozen grader and typography rule. Original Level-2/3 collision is undefined because no prompt has two correct draws.

Condition	L	R	Strict			Normalized		
			A	D	C	A	D	C
Original	1	409	38.1	0.91	93.0	60.0	1.12	90.4
	2	1021	0.3	0.02	–	0.3	0.02	–
	3	1024	0.0	0.00	–	0.0	0.00	–
Plain user, no system	1	0	74.9	1.42	80.2	100.0	1.49	79.9
	2	0	71.8	1.31	84.7	100.0	1.38	85.7
	3	3	76.2	1.38	82.7	99.7	1.41	84.1

The new condition has no API failures or token-limit truncations. Strict grading accepts 2,282 responses; the frozen rule rescues 787 more by converting escaped percent signs (\%) to Python modulo (%). Thus every non-refused answer passes after this formatting-only correction. Normalized collision is 79.9/85.7/84.1%, versus conditional uniform references of 1.43/1.15/1.15%. The residual three refusals occur on two prompts whose other identical requests are accepted. No retry-until-accepted selection is used.

The paired analysis resamples whole eight-draw prompt groups across conditions (20,000 replicates). Normalized accuracy gains at Levels 1/2/3 are 40.0 [34.7, 45.5], 99.7 [99.3, 100.0], and 99.7 [99.2, 100.0] percentage points (pointwise 95% intervals). Output draws themselves are independent, and the different correct-pair populations preclude interpreting a raw collision difference as a matched-population prompt effect.

The saved sensitivity record binds the original and revised native receipts, request payloads, strict regrade audit and frozen normalization outputs.

Separate first-valid retry diagnostic. After completing the fixed 3,072-response revised-wording cohort, a separate condition retries only its three remaining refused slots, retaining the first verified-valid response. All three succeed on their first additional request, and all reproduce a mode already seen for that prompt. This uses 3,075 requests to select 3,072 valid responses;

its 100% accepted-set accuracy follows from selection and is not unconditional model accuracy. The selected set's `distinct@8` is unchanged at 1.492, 1.375, 1.414 at Levels 1–3. Level-3 correct-pair collision changes from 84.0909% to 84.1518%; the other levels are unchanged. All initial and retry responses remain preserved. The main figure continues to use the complete fixed eight-draw condition, including its three refusals; these selected-validity results are only a separate diagnostic with variable request effort.

M.2 A MATCHED REQUESTED-TEMPERATURE SENSITIVITY TEST

A separate fixed subset contains eight prompts from each of the five domains at each of three levels: 120 prompts, with eight independent draws per prompt and temperature. Grok 4.3 and Kimi K3 each receive requested temperatures $T = 1.0$ and $T = 1.5$, giving 960 draws per condition and 3,840 in total. The two conditions share task rows and all non-temperature request settings; both temperatures are collected concurrently within each deployment. Every response is retained. This subset is a separate experiment and does not replace any full-cohort result in the main hosted display.

Provider API contracts support the requested controls and the deployments accept the requests, but native responses do not report effective sampling temperature. DeepSeek is excluded from this contrast because its documented thinking-mode interface ignores the temperature setting. These contrasts therefore concern the tested requested settings, not arbitrary temperatures or all deployments.

Tables average all five domains equally within each level; All equally averages all 15 domain-level cells. Confidence intervals use 20,000 paired whole-prompt bootstrap resamples stratified by domain and level, retaining each eight-draw group. They are pointwise and have no multiplicity adjustment. Correct-pair collision remains conditional on correct draws and can change its eligible prompt population; an undefined cell makes the corresponding macro undefined.

Observed sensitivity. For Grok, the observed breadth change is +0.050 [-0.058, +0.158] modes, and the collision change is -0.78 [-3.86, +2.12] percentage points. These intervals do not show a clear broadening at the tested setting. The per-response accuracy change is -2.29 [-3.85, -0.94] percentage points. This does not establish invariance to temperature in general.

Kimi returns 696/960 token-limited outputs at $T = 1.5$, compared with 0 at $T = 1.0$. Strict accuracy falls from 96.56% to 21.25%, while raw breadth falls from 1.933 to 0.983 modes. This is a severe token-limit and accuracy failure, not a clean increase in correct-output concentration. The five-domain collision mean is undefined at $T = 1.5$ because some cells have no eligible correct pairs. Neither model has provider-declared refusals in this experiment.

† marks an omitted interval when fewer than all 20,000 bootstrap replicates define the contrast. For Kimi collision, only 11,815 and 11,616 replicates are defined at Levels 1 and 2; the raw conditional percentile summaries remain in the source record. We do not present them as ordinary 95% intervals. Undefined cells are never replaced by zero or silently omitted.

Full domain-level estimates, counts, native outcomes and collision eligibility remain in the source-bound temperature analysis. Countdown has no finite-total-support uniform reference; finite-support sensitivity summaries use the other four domains. The paper record authenticates every included condition summary, native completion audit, frozen normalizer, paired analysis source and sampling-control review, with the complete fixed response cohort retained throughout.

M.3 GPT-5.6 SOL: TEMPERATURE, PASS@8 AND VERIFIED MODES

Figure 34 plots empirical `pass@8` against `distinct@8` for $T \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$, with reasoning none. Each condition retains the same 480 held-out prompts (32 per domain-level cell), with eight stateless responses per prompt: 3,840 responses per temperature and 19,200 in total. A prompt contributes one to `pass@8` if at least one of its eight answers is verified correct, and contributes its number of distinct correct canonical keys to `distinct@8`. Zero-success groups and all unsuccessful draws remain in the denominator. We compute success directly from the saved groups, not as $1 - (1 - p)^8$ from pooled per-response accuracy; task difficulty varies across prompts.

The expanded cohort retains all 120 original prompts and their 4,800 responses, adding 360 prompts and 14,400 responses. The extension follows the same outcome-independent row-

Table 54: **Requested-temperature sensitivity: Strict executable grades.** A is per-response accuracy (%), D is `distinct@8`, and C is correct-pair collision (%). D differences are mode counts; A/C differences are percentage points. Brackets give paired pointwise 95% intervals for the paired change in requested temperature. Every fixed draw remains in its original denominator.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Grok 4.3	1	A	98.12	95.62	-2.50 [-4.06, -0.94]
		D	1.900	2.000	+0.100 [-0.100, +0.300]
		C	72.89	73.16	+0.27 [-6.31, +6.37]
	2	A	99.06	99.38	+0.31 [-0.94, +1.56]
		D	1.425	1.425	+0.000 [-0.175, +0.200]
		C	86.61	85.20	-1.41 [-6.26, +3.30]
	3	A	98.75	94.06	-4.69 [-9.06, -0.94]
		D	1.500	1.550	+0.050 [-0.125, +0.225]
		C	84.04	82.85	-1.20 [-5.48, +2.79]
	All	A	98.65	96.35	-2.29 [-3.85, -0.94]
		D	1.608	1.658	+0.050 [-0.058, +0.158]
		C	81.18	80.40	-0.78 [-3.86, +2.12]
Kimi K3	1	A	92.50	22.50	-70.00 [-75.62, -63.75]
		D	2.200	1.075	-1.125 [-1.475, -0.775]
		C	66.23	53.12	-13.11 [-] [†]
	2	A	98.12	20.31	-77.81 [-82.50, -72.81]
		D	1.850	0.900	-0.950 [-1.300, -0.625]
		C	72.18	66.55	-5.63 [-] [†]
	3	A	99.06	20.94	-78.12 [-83.75, -72.81]
		D	1.750	0.975	-0.775 [-1.025, -0.525]
		C	78.33	—	—
	All	A	96.56	21.25	-75.31 [-78.44, -72.19]
		D	1.933	0.983	-0.950 [-1.133, -0.767]
		C	72.25	—	—

Table 55: **Requested-temperature sensitivity: Frozen formatting-normalized grades.** A is per-response accuracy (%), D is `distinct@8`, and C is correct-pair collision (%). D differences are mode counts; A/C differences are percentage points. Brackets give paired pointwise 95% intervals for the paired change in requested temperature. Every fixed draw remains in its original denominator.

Deployment	Level	Metric	$T = 1.0$	$T = 1.5$	Difference [95% interval]
Grok 4.3	1	A	98.12	95.62	-2.50 [-4.06, -0.94]
		D	1.900	2.000	+0.100 [-0.100, +0.300]
		C	72.89	73.16	+0.27 [-6.31, +6.37]
	2	A	99.06	99.38	+0.31 [-0.94, +1.56]
		D	1.425	1.425	+0.000 [-0.175, +0.200]
		C	86.61	85.20	-1.41 [-6.26, +3.30]
	3	A	98.75	94.06	-4.69 [-9.06, -0.94]
		D	1.500	1.550	+0.050 [-0.125, +0.225]
		C	84.04	82.85	-1.20 [-5.48, +2.79]
	All	A	98.65	96.35	-2.29 [-3.85, -0.94]
		D	1.608	1.658	+0.050 [-0.058, +0.158]
		C	81.18	80.40	-0.78 [-3.86, +2.12]
Kimi K3	1	A	92.50	22.81	-69.69 [-75.31, -63.44]
		D	2.200	1.075	-1.125 [-1.475, -0.775]
		C	66.23	53.61	-12.62 [-] [†]
	2	A	98.12	20.31	-77.81 [-82.50, -72.81]
		D	1.850	0.900	-0.950 [-1.300, -0.625]
		C	72.18	66.55	-5.63 [-] [†]
	3	A	99.38	20.94	-78.44 [-84.06, -73.12]
		D	1.750	0.975	-0.775 [-1.025, -0.525]
		C	78.18	—	—
	All	A	96.67	21.35	-75.31 [-78.44, -72.19]
		D	1.933	0.983	-0.950 [-1.133, -0.767]
		C	72.20	—	—

hash ranking within each domain-level cell. Requests retain the frozen prompt bytes and verifier; only temperature varies in the sampling controls. New collection interleaves temperatures. Separate old-120 and new-360 analyses below assess cohort sensitivity; prompt composition and collection time differ between these cohorts.

The deployment rejected non-default temperatures with medium reasoning and rejected temperature 2.5. These curves use the supported reasoning `none` profile. Historical medium-reasoning measurements cover only the original 120 prompts and are omitted from the expanded-cohort tables and figure. This does not test temperature robustness of medium reasoning. A zero-temperature request does not establish deterministic outputs.

Each level averages five domains equally; All averages all 15 cells. Pointwise 95 percent intervals use 20,000 paired whole-prompt bootstrap resamples stratified by domain and level, retaining all eight draws. There is no multiplicity adjustment. A zero-width empirical interval reflects constant observed groups, not certain success on unseen prompts.

Table 56: **GPT temperature frontier: Strict executable grades.** P is solved prompts (% pass@8); D is distinct@8; A is per-response accuracy (%). Brackets give pointwise 95% intervals; all temperatures use reasoning `none` and eight draws per prompt.

T	Level	P [95% interval]	D [95% interval]	A [95% interval]
0.0	1	55.0 [48.8, 61.3]	0.569 [0.500, 0.637]	53.9 [47.7, 60.1]
	2	45.0 [39.4, 50.6]	0.456 [0.400, 0.512]	39.5 [34.4, 44.7]
	3	49.4 [42.5, 56.2]	0.519 [0.444, 0.594]	44.5 [38.3, 50.9]
	All	49.8 [46.2, 53.3]	0.515 [0.475, 0.554]	46.0 [42.6, 49.4]
0.5	1	70.6 [64.4, 76.9]	0.956 [0.850, 1.069]	57.5 [51.9, 63.0]
	2	50.0 [44.4, 55.6]	0.525 [0.463, 0.594]	38.6 [33.9, 43.4]
	3	54.4 [48.1, 60.6]	0.581 [0.506, 0.662]	43.0 [37.1, 48.9]
	All	58.3 [54.8, 61.9]	0.688 [0.640, 0.738]	46.4 [43.3, 49.5]
1.0	1	75.6 [69.4, 81.2]	1.181 [1.050, 1.312]	57.3 [52.0, 62.7]
	2	51.9 [45.6, 58.1]	0.581 [0.506, 0.662]	37.6 [33.2, 42.0]
	3	60.6 [54.4, 66.9]	0.713 [0.625, 0.806]	42.7 [37.3, 48.1]
	All	62.7 [59.2, 66.2]	0.825 [0.767, 0.885]	45.9 [42.9, 48.8]
1.5	1	80.6 [75.0, 86.2]	1.413 [1.269, 1.556]	56.6 [51.7, 61.4]
	2	58.1 [51.9, 64.4]	0.650 [0.569, 0.738]	38.4 [34.3, 42.7]
	3	65.6 [59.4, 71.2]	0.838 [0.731, 0.956]	43.0 [38.0, 48.2]
	All	68.1 [64.8, 71.5]	0.967 [0.900, 1.035]	46.0 [43.3, 48.8]
2.0	1	83.8 [78.1, 88.8]	1.519 [1.375, 1.663]	59.7 [54.8, 64.5]
	2	54.4 [48.1, 60.6]	0.600 [0.525, 0.675]	35.3 [31.4, 39.3]
	3	61.9 [56.2, 67.5]	0.819 [0.719, 0.925]	40.5 [35.8, 45.5]
	All	66.7 [63.3, 70.0]	0.979 [0.915, 1.044]	45.2 [42.6, 47.8]

Observed temperature frontier. At $T = 0.0$ and $T = 2.0$, normalized pass@8 is 65.62% and 79.38%, respectively, while distinct@8 is 0.673 and 1.146. Across these 5 measured temperatures, the highest observed aggregate pass@8 occurs at $T = 1.5$ and the highest distinct@8 occurs at $T = 2.0$. This finite exploratory sweep does not establish a population optimum or a continuous Pareto boundary, and level-specific curves differ.

Per-response accuracy changes from 62.11% to 55.94% between the endpoints. That diagnostic is distinct from pass@8: success within eight draws and the reliability of an individual draw need not change together as temperature varies.

The paired $T = 2.0$ minus $T = 0.0$ contrast is 13.8 [10.8, 16.7] pass@8 percentage points and 0.473 [0.417, 0.529] distinct correct modes, with pointwise 95% intervals. All prompt-level records, domain estimates, original per-response and collision analyses, and paired comparisons remain in the retained reports for both strict and normalized grading.

Retained outcomes. At $T = 0.0$, 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. At $T = 0.5$, 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. At $T = 1.0$, 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. At $T = 1.5$, 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. At $T = 2.0$, 0 of 3840 outputs are token-limited and 0 are provider-declared refusals. These outcome counts retain every registered draw and are separate from the verifier-based correctness and canonical-key measurements above.

Table 57: **GPT temperature frontier: Frozen formatting-normalized grades.** P is solved prompts (% pass@8); D is distinct@8; A is per-response accuracy (%). Brackets give pointwise 95% intervals; all temperatures use reasoning *none* and eight draws per prompt.

T	Level	P [95% interval]	D [95% interval]	A [95% interval]
0.0	1	70.6 [65.0, 76.2]	0.725 [0.662, 0.788]	70.5 [64.5, 76.2]
	2	60.6 [55.0, 66.2]	0.613 [0.556, 0.669]	54.5 [49.0, 59.9]
	3	65.6 [59.4, 71.9]	0.681 [0.606, 0.756]	61.4 [55.2, 67.6]
	All	65.6 [62.3, 69.0]	0.673 [0.635, 0.710]	62.1 [58.8, 65.5]
0.5	1	76.9 [71.2, 82.5]	1.038 [0.931, 1.150]	71.7 [66.4, 77.0]
	2	68.1 [62.5, 73.8]	0.706 [0.644, 0.769]	53.7 [48.8, 58.6]
	3	71.9 [65.6, 77.5]	0.775 [0.700, 0.850]	58.1 [52.3, 63.9]
	All	72.3 [69.2, 75.4]	0.840 [0.792, 0.890]	61.2 [58.2, 64.3]
1.0	1	79.4 [73.8, 84.4]	1.238 [1.106, 1.369]	70.4 [65.2, 75.5]
	2	69.4 [63.7, 75.0]	0.756 [0.681, 0.831]	51.6 [47.1, 56.2]
	3	76.9 [71.2, 82.5]	0.894 [0.806, 0.988]	55.2 [49.8, 60.7]
	All	75.2 [72.1, 78.3]	0.963 [0.904, 1.021]	59.1 [56.2, 62.0]
1.5	1	82.5 [77.5, 87.5]	1.494 [1.344, 1.644]	68.5 [63.7, 73.3]
	2	75.0 [69.4, 80.6]	0.844 [0.762, 0.931]	49.8 [45.5, 54.3]
	3	81.9 [76.2, 87.5]	1.038 [0.931, 1.156]	54.7 [49.6, 59.8]
	All	79.8 [76.7, 82.9]	1.125 [1.056, 1.196]	57.7 [54.9, 60.5]
2.0	1	85.0 [80.0, 90.0]	1.606 [1.456, 1.762]	69.6 [64.8, 74.3]
	2	72.5 [66.9, 78.1]	0.800 [0.731, 0.869]	45.7 [41.7, 49.8]
	3	80.6 [75.0, 86.2]	1.031 [0.931, 1.137]	52.5 [47.7, 57.5]
	All	79.4 [76.2, 82.5]	1.146 [1.079, 1.212]	55.9 [53.3, 58.6]

Sensitivity to the prompt expansion. The original and additional prompt cohorts retain separate paired temperature comparisons. Differences between cohorts may reflect prompt composition or collection time; they are not causal time effects.

Across all 480 prompts, the paired $T = 2$ minus $T = 1.5$ contrast is -0.4 [-2.5, 1.7] pass@8 percentage points and 0.021 [-0.029, 0.071] distinct modes (95% intervals). Original 120 prompts: the paired $T = 2$ minus $T = 1.5$ contrast is -1.7 [-5.0, 0.8] pass@8 percentage points and -0.008 [-0.083, 0.067] distinct modes (95% intervals). Additional 360 prompts: the paired $T = 2$ minus $T = 1.5$ contrast is 0.0 [-2.8, 2.8] pass@8 percentage points and 0.031 [-0.033, 0.094] distinct modes (95% intervals).

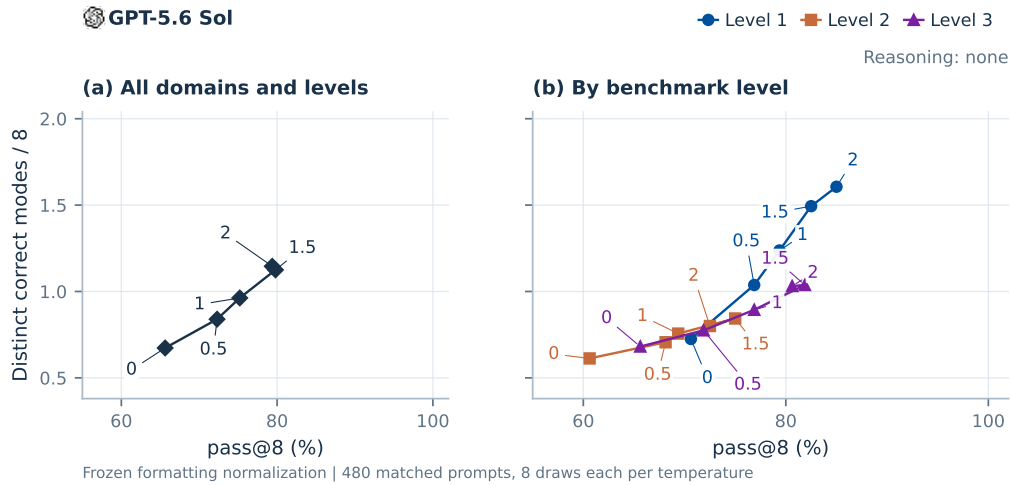


Figure 34: **Temperature traces pass@8 and verified breadth.** GPT-5.6 Sol, with 480 fixed prompts and eight draws each at $T \in \{0, 0.5, 1, 1.5, 2\}$. Left: all domains and levels; right: separate levels, averaging five domains equally. Empirical pass@8 is the fraction with at least one correct draw; the registered distinct@8 averages unique correct modes, including zeroes, and so tracks pass@8 across temperatures by construction. Connected points use reasoning *none* and the frozen formatting normalizer. App. M.3 gives pointwise uncertainty intervals for every benchmark level at each of the five measured temperatures, together with the per-domain values behind each plotted point.

M.4 NATIVE PROVIDER OUTCOMES AND UNDEFINED CONCENTRATION

Provider-declared refusals and filtering are deployment outcomes, not evidence that the underlying model cannot solve the mathematical task. Empty refusals remain failed draws under the frozen verifier; no requests or grades are overwritten. Classification uses native metadata, not a semantic detector of refusals written as ordinary answer text.

Correct-pair collision is undefined when no prompt supplies two correct draws. The five-domain mean is therefore undefined if any domain has no eligible pairs; such entries remain – rather than zero or a mean over fewer domains.

GPT-5.6 Sol native completion states: completed: 15,360. Empty visible answers are reported separately from provider refusals.

Table 58: **Native service outcomes: GPT-5.6 Sol.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Claude Opus 5 native completion states: end_turn: 12,838, refusal: 2,522. Empty visible answers are reported separately from provider refusals.

Claude Opus 5 returns provider-declared refusals for 409, 1021, 1024 of the 1,024 Python requests at Levels 1, 2 and 3, respectively. The native refusal categories on these Python draws are cyber: 2454. These are benign factor-selection tasks; the reported deployment policy must be distinguished from concentration among correct Python outputs.

Table 59: **Native service outcomes: Claude Opus 5.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	409	0	409
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	1021	0	1021
2	MathIR	35	0	35
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	1024	0	1024
3	MathIR	33	0	32
3	PantryPlan	0	0	0

GPT-5.4 native completion states: completed: 15,355, incomplete: 5. Empty visible answers are reported separately from provider refusals.

Table 60: **Native service outcomes: GPT-5.4.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	1
2	Graph	0	0	0
2	Countdown	0	0	1
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	1
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	2

Grok 4.3 native completion states: stop: 15,360. Empty visible answers are reported separately from provider refusals.

Table 61: **Native service outcomes: Grok 4.3.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Kimi K3 native completion states: length: 15, stop: 15,345. Empty visible answers are reported separately from provider refusals.

Claude Opus 4.8 native completion states: end_turn: 15,355, max_tokens: 5. Empty visible answers are reported separately from provider refusals.

Claude Opus 4.8 returns provider-declared refusals for 0, 0, 0 of the 1,024 Python requests at Levels 1, 2 and 3, respectively. The native refusal categories on these Python draws are not supplied. These are benign factor-selection tasks; the reported deployment policy must be distinguished from concentration among correct Python outputs.

DeepSeek V4 Pro native completion states: length: 632, stop: 14,728. Empty visible answers are reported separately from provider refusals.

The result record `results/frontier_comparison_20260911.json` binds every source summary, completion audit, frozen prompt digest, raw-sample digest and primary graded sample digest. Pending deployments are omitted from both the figure and numerical comparisons. The

Table 62: **Native service outcomes: Kimi K3.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	0
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	0

Table 63: **Native service outcomes: Claude Opus 4.8.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	0
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	0
1	PantryPlan	0	0	0
2	Graph	0	0	0
2	Countdown	0	0	0
2	Python	0	0	0
2	MathIR	0	0	0
2	PantryPlan	0	0	0
3	Graph	0	0	0
3	Countdown	0	0	3
3	Python	0	0	0
3	MathIR	0	0	0
3	PantryPlan	0	0	1

figure was selected after the reference GPT run for its high strict correctness, certified support and lack of formatting corrections, and before collecting the prospective comparison runs. All five domains are retained in the tables, including disagreement across models.

Table 64: **Native service outcomes: DeepSeek V4 Pro.** Every row contains 1,024 responses. R counts provider-declared refusals, F content-filtered stops, and E empty visible answers. Counters may overlap; they do not replace the frozen verifier grades.

Level	Domain	R	F	E
1	Graph	0	0	59
1	Countdown	0	0	0
1	Python	0	0	0
1	MathIR	0	0	4
1	PantryPlan	0	0	13
2	Graph	0	0	38
2	Countdown	0	0	1
2	Python	0	0	0
2	MathIR	0	0	59
2	PantryPlan	0	0	80
3	Graph	0	0	24
3	Countdown	0	0	7
3	Python	0	0	0
3	MathIR	0	0	100
3	PantryPlan	0	0	246

Table 65: **Strict level summaries for completed hosted deployments.** Each level weights the five domains equally. M and P are mean@8 and pass@8 (%); D is raw distinct@8. C is correct-pair collision (% with pointwise 95% whole-prompt intervals), pair weighted within each domain. Provider settings differ; these are descriptive outputs.

Deployment	Level	M	P	D	C (95% interval)
GPT-5.6 Sol	1	83.0	95.8	1.71	74.0 [71.9, 76.1]
GPT-5.6 Sol	2	62.9	77.5	1.16	82.3 [80.3, 84.4]
GPT-5.6 Sol	3	68.7	80.3	1.18	84.5 [82.8, 86.2]
Claude Opus 5	1	85.3	94.5	1.32	86.3 [84.5, 88.0]
Claude Opus 5	2	76.6	80.2	1.16	—
Claude Opus 5	3	77.8	79.8	1.07	—
GPT-5.4	1	93.7	98.1	2.09	67.2 [65.1, 69.3]
GPT-5.4	2	80.6	97.7	1.61	74.7 [72.4, 76.8]
GPT-5.4	3	83.3	97.8	1.57	78.8 [76.9, 80.7]
Grok 4.3	1	98.3	99.5	1.97	72.4 [70.7, 74.0]
Grok 4.3	2	99.3	100.0	1.57	82.1 [80.6, 83.7]
Grok 4.3	3	98.8	100.0	1.55	83.0 [81.5, 84.6]
Kimi K3	1	96.9	98.4	2.31	62.5 [60.6, 64.4]
Kimi K3	2	96.8	100.0	2.05	69.4 [67.6, 71.2]
Kimi K3	3	97.8	100.0	1.90	74.6 [73.1, 76.2]
Claude Opus 4.8	1	96.4	98.8	1.64	78.4 [76.6, 80.3]
Claude Opus 4.8	2	98.5	100.0	1.67	77.2 [75.4, 79.1]
Claude Opus 4.8	3	98.9	99.8	1.51	83.4 [81.8, 84.9]
DeepSeek V4 Pro	1	96.4	98.4	2.01	71.0 [69.4, 72.7]
DeepSeek V4 Pro	2	93.7	99.1	1.84	68.7 [67.0, 70.3]
DeepSeek V4 Pro	3	91.0	99.7	1.62	78.8 [76.9, 80.9]

Table 66: **GPT-5.6 Sol: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	86.1	98.4	1.77	71.2 [66.1, 75.9]	–
1	Python	40.4	90.6	1.11	81.8 [76.9, 87.2]	0.8
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	65.4	86.7	1.57	64.7 [59.0, 70.6]	–
2	Python	8.5	11.7	0.12	100.0 [100.0, 100.0]	0.7
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	55.3	96.1	1.72	63.6 [56.8, 70.3]	6.9
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	71.2	89.8	1.38	76.4 [71.1, 81.8]	–
3	Python	9.4	14.1	0.14	100.0 [100.0, 100.0]	0.9
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	65.3	100.0	2.22	51.0 [45.3, 57.3]	6.8

Table 67: **Claude Opus 5: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
1	Python	38.1	81.2	0.91	93.0 [88.7, 96.6]	0.7
1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
2	Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9
2	Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–
2	Python	0.3	2.3	0.02	–	–
2	MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0
2	PantryPlan	94.5	100.0	1.56	80.7 [76.0, 85.3]	6.4
3	Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0
3	Countdown	99.9	100.0	1.66	76.2 [71.7, 80.7]	–
3	Python	0.0	0.0	0.00	–	–
3	MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0
3	PantryPlan	93.6	100.0	1.63	78.8 [74.2, 83.4]	6.5

Table 68: **GPT-5.4: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
1	Countdown	91.8	100.0	1.73	76.6 [71.2, 81.3]	–
1	Python	87.6	100.0	2.52	56.4 [51.3, 61.5]	1.4
1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0
1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5
2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0
2	Countdown	91.1	100.0	1.98	63.0 [58.6, 67.5]	–
2	Python	47.9	96.1	1.47	72.9 [64.6, 80.2]	1.0
2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0
2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8
3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0
3	Countdown	82.2	93.8	1.72	69.1 [64.4, 73.5]	–
3	Python	50.1	97.7	1.58	74.4 [68.6, 79.8]	1.1
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6

Table 69: **Grok 4.3: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
1	Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–
1	Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4
1	MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0
1	PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5
2	Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0
2	Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–
2	Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1
2	MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0
2	PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5
3	Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0
3	Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–
3	Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2
3	MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0
3	PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5

Table 70: **Kimi K3: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
1	Countdown	99.1	100.0	1.91	68.4 [63.8, 73.3]	–
1	Python	97.9	100.0	2.52	57.7 [53.5, 61.9]	1.4
1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0
1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5
2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0
2	Countdown	98.4	100.0	1.98	64.4 [59.9, 68.7]	–
2	Python	98.2	100.0	2.44	61.9 [57.6, 66.2]	1.1
2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0
2	PantryPlan	93.3	100.0	3.20	41.3 [36.6, 45.9]	6.6
3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0
3	Countdown	97.5	100.0	1.84	70.6 [66.1, 74.8]	–
3	Python	98.0	100.0	2.34	64.7 [60.1, 69.0]	1.2
3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0
3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5

Table 71: **Claude Opus 4.8: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
1	Python	98.0	100.0	1.40	85.7 [81.7, 89.4]	1.4
1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–
3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
3	PantryPlan	98.5	100.0	2.33	61.6 [56.7, 66.6]	6.5

Table 72: **DeepSeek V4 Pro: Strict executable grades.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
1	Python	99.3	100.0	1.41	88.8 [86.1, 91.4]	1.4
1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
2	Python	99.6	100.0	1.05	98.4 [97.1, 99.4]	1.2
2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
3	Python	99.6	100.0	1.08	97.8 [96.3, 99.0]	1.2
3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

Table 73: **GPT-5.6 Sol: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.2	100.0	1.30	89.7 [86.3, 92.8]	18.3
1	Countdown	99.9	100.0	1.93	67.9 [63.3, 72.4]	–
1	Python	92.4	100.0	1.41	85.6 [82.3, 88.8]	1.4
1	MathIR	89.3	89.8	1.38	80.3 [75.4, 85.0]	20.0
1	PantryPlan	100.0	100.0	3.00	46.9 [42.2, 51.5]	6.5
2	Graph	99.3	100.0	1.48	83.3 [79.6, 87.0]	18.9
2	Countdown	98.5	100.0	1.99	63.9 [59.5, 68.4]	–
2	Python	60.7	99.2	1.12	94.8 [91.9, 97.2]	1.1
2	MathIR	86.2	93.0	0.93	100.0 [100.0, 100.0]	20.0
2	PantryPlan	71.3	100.0	1.96	64.0 [58.1, 69.6]	6.8
3	Graph	100.0	100.0	1.16	95.1 [92.4, 97.3]	19.0
3	Countdown	99.0	100.0	1.70	75.7 [71.2, 80.1]	–
3	Python	56.5	98.4	1.09	94.3 [90.8, 97.2]	1.0
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	75.2	100.0	2.35	53.0 [47.3, 58.9]	6.7

Table 74: **Claude Opus 5: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8 . C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.15	94.0 [91.3, 96.4]	18.3
1	Countdown	100.0	100.0	1.59	78.8 [74.6, 83.0]	–
1	Python	60.0	94.5	1.12	90.4 [86.2, 94.3]	0.9
1	MathIR	90.5	91.4	1.09	92.2 [88.8, 95.4]	20.0
1	PantryPlan	98.4	100.0	1.86	73.5 [68.5, 78.2]	6.5
2	Graph	98.9	100.0	1.32	88.1 [84.5, 91.6]	18.9
2	Countdown	100.0	100.0	1.81	69.3 [65.2, 73.6]	–
2	Python	0.3	2.3	0.02	–	–
2	MathIR	89.1	98.4	1.06	96.5 [93.9, 98.6]	20.0
2	PantryPlan	99.9	100.0	1.61	79.7 [75.0, 84.3]	6.5
3	Graph	100.0	100.0	1.02	99.4 [98.3, 100.0]	19.0
3	Countdown	100.0	100.0	1.66	76.1 [71.7, 80.7]	–
3	Python	0.0	0.0	0.00	–	–
3	MathIR	95.3	99.2	1.02	98.8 [97.5, 99.8]	20.0
3	PantryPlan	99.4	100.0	1.70	77.3 [72.6, 82.0]	6.5

Table 75: **GPT-5.4: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8 . C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.8	100.0	1.57	81.6 [77.8, 85.4]	18.3
1	Countdown	99.7	100.0	1.80	73.6 [68.5, 78.3]	–
1	Python	98.7	100.0	2.57	56.8 [51.9, 61.6]	1.4
1	MathIR	89.3	90.6	1.45	78.6 [74.0, 83.4]	20.0
1	PantryPlan	99.9	100.0	3.16	42.8 [38.4, 47.4]	6.5
2	Graph	99.9	100.0	1.74	74.2 [69.8, 78.7]	19.0
2	Countdown	98.4	100.0	2.12	61.1 [57.0, 65.1]	–
2	Python	91.3	100.0	1.87	74.0 [69.4, 78.1]	1.1
2	MathIR	87.0	92.2	0.92	100.0 [100.0, 100.0]	20.0
2	PantryPlan	77.0	100.0	1.93	63.2 [58.4, 68.6]	6.8
3	Graph	99.8	100.0	1.23	92.7 [89.7, 95.4]	19.0
3	Countdown	99.0	100.0	1.96	68.4 [64.2, 72.7]	–
3	Python	91.6	100.0	1.91	72.6 [68.3, 77.1]	1.2
3	MathIR	97.7	97.7	0.98	100.0 [100.0, 100.0]	20.0
3	PantryPlan	86.5	100.0	2.34	57.8 [52.8, 63.4]	6.6

Table 76: **Grok 4.3: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote $\text{mean}@8/\text{pass}@8$ (%); D is $\text{distinct}@8$. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	100.0	100.0	1.40	87.8 [84.2, 91.0]	18.3
1	Countdown	99.9	100.0	2.05	64.8 [60.1, 69.5]	–
1	Python	99.8	100.0	1.41	88.0 [84.8, 91.0]	1.4
1	MathIR	92.4	97.7	1.24	90.1 [86.9, 93.1]	20.0
1	PantryPlan	99.4	100.0	3.77	31.4 [27.8, 35.1]	6.5
2	Graph	99.9	100.0	1.52	84.3 [80.5, 88.0]	19.0
2	Countdown	99.8	100.0	1.99	65.8 [61.8, 70.2]	–
2	Python	99.8	100.0	1.05	98.8 [97.8, 99.6]	1.1
2	MathIR	97.5	100.0	1.12	94.9 [92.4, 97.2]	20.0
2	PantryPlan	99.5	100.0	2.16	66.6 [61.2, 71.4]	6.5
3	Graph	100.0	100.0	1.17	94.5 [92.0, 96.8]	19.0
3	Countdown	99.9	100.0	1.98	66.3 [61.7, 71.0]	–
3	Python	99.9	100.0	1.04	98.6 [97.2, 99.6]	1.2
3	MathIR	95.0	100.0	1.05	97.8 [95.9, 99.3]	20.0
3	PantryPlan	99.2	100.0	2.52	58.0 [53.1, 63.5]	6.5

Table 77: **Kimi K3: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote $\text{mean}@8/\text{pass}@8$ (%); D is $\text{distinct}@8$. C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.5	100.0	1.48	84.3 [80.4, 87.8]	18.3
1	Countdown	99.4	100.0	1.91	68.4 [63.9, 73.3]	–
1	Python	98.2	100.0	2.53	57.7 [53.5, 61.9]	1.4
1	MathIR	90.2	92.2	1.47	75.8 [70.7, 80.7]	20.0
1	PantryPlan	97.5	100.0	4.18	26.2 [23.1, 29.2]	6.5
2	Graph	99.6	100.0	1.62	80.0 [76.0, 84.1]	19.0
2	Countdown	98.7	100.0	1.98	64.5 [60.1, 68.9]	–
2	Python	98.3	100.0	2.44	61.9 [57.6, 66.2]	1.1
2	MathIR	94.3	100.0	1.02	99.8 [99.3, 100.0]	20.0
2	PantryPlan	93.4	100.0	3.20	41.4 [36.7, 46.0]	6.6
3	Graph	99.9	100.0	1.19	94.6 [91.8, 96.9]	19.0
3	Countdown	98.1	100.0	1.84	70.3 [65.8, 74.5]	–
3	Python	98.5	100.0	2.34	64.9 [60.3, 69.2]	1.2
3	MathIR	98.1	100.0	1.00	100.0 [100.0, 100.0]	20.0
3	PantryPlan	95.6	100.0	3.13	43.4 [38.6, 48.3]	6.5

Table 78: **Claude Opus 4.8: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8 . C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	99.7	100.0	1.34	87.5 [83.9, 91.0]	18.3
1	Countdown	95.6	99.2	1.78	69.8 [64.9, 74.5]	–
1	Python	98.6	100.0	1.40	85.8 [81.8, 89.5]	1.5
1	MathIR	92.3	94.5	1.10	93.7 [91.1, 96.4]	20.0
1	PantryPlan	96.6	100.0	2.59	55.3 [50.3, 60.1]	6.5
2	Graph	95.9	100.0	1.66	76.6 [72.3, 81.1]	18.8
2	Countdown	99.9	100.0	1.99	65.3 [60.9, 69.9]	–
2	Python	100.0	100.0	1.34	86.0 [82.3, 89.6]	1.2
2	MathIR	97.0	100.0	1.24	90.1 [86.7, 93.4]	20.0
2	PantryPlan	99.5	100.0	2.11	67.9 [62.8, 73.1]	6.5
3	Graph	99.9	100.0	1.08	97.3 [95.5, 98.9]	19.0
3	Countdown	97.9	99.2	1.80	72.5 [68.2, 76.6]	–
3	Python	98.9	100.0	1.32	86.4 [82.5, 90.1]	1.2
3	MathIR	99.1	100.0	1.03	99.1 [98.1, 99.8]	20.0
3	PantryPlan	98.7	100.0	2.33	61.7 [56.7, 66.6]	6.5

Table 79: **DeepSeek V4 Pro: Post hoc formatting diagnostic.** Each cell has 128 prompts and 1,024 responses. M/P denote mean@8/pass@8 (%); D is distinct@8 . C is correct-pair collision (%) with pointwise 95% prompt-bootstrap intervals. U is its conditional uniform reference (%).

Level	Domain	M	P	D	C (95% interval)	U
1	Graph	94.0	100.0	1.80	73.1 [68.8, 77.4]	18.1
1	Countdown	100.0	100.0	2.02	65.8 [61.4, 70.3]	–
1	Python	99.4	100.0	1.41	88.8 [86.1, 91.4]	1.4
1	MathIR	90.1	92.2	1.11	93.9 [91.2, 96.2]	20.0
1	PantryPlan	98.5	100.0	3.73	33.5 [29.8, 37.2]	6.5
2	Graph	96.2	100.0	1.97	69.0 [64.4, 73.5]	19.1
2	Countdown	99.9	100.0	2.04	63.4 [59.2, 67.7]	–
2	Python	99.7	100.0	1.05	98.4 [97.1, 99.4]	1.2
2	MathIR	85.4	95.3	1.82	51.7 [49.8, 53.9]	20.0
2	PantryPlan	87.1	100.0	2.34	61.1 [56.0, 66.2]	6.5
3	Graph	97.5	100.0	1.52	83.4 [79.6, 87.0]	18.9
3	Countdown	99.2	100.0	1.98	67.5 [63.1, 72.2]	–
3	Python	99.7	100.0	1.08	97.8 [96.3, 99.0]	1.2
3	MathIR	88.5	100.0	1.44	83.6 [79.2, 87.6]	20.0
3	PantryPlan	70.3	98.4	2.08	61.9 [55.9, 68.2]	6.5

Model	Level 1		Level 2		Level 3	
	pass@8 (%)	# modes	pass@8 (%)	# modes	pass@8 (%)	# modes
GPT-5.6 Sol	98.0	1.80	98.4	1.50	99.2	1.45
Claude Opus 5	98.3	1.43	99.7	1.44	99.8	1.36
GPT-5.4	98.1	2.11	98.4	1.72	99.5	1.68
Grok 4.3	99.5	1.97	100.0	1.57	100.0	1.55
Kimi K3	98.4	2.31	100.0	2.05	100.0	1.90
Claude Opus 4.8	98.8	1.64	100.0	1.67	99.8	1.51
DeepSeek V4 Pro	98.4	2.01	99.1	1.84	99.7	1.62

Table 80: **Hosted success and output diversity with medium reasoning, averaged by level.** Each level averages five domains equally, using 128 prompts per domain and eight responses per prompt. pass@8 is the fraction with any verified response; # modes is mean distinct@8 over all prompts, with zero modes for every prompt lacking a verified response.

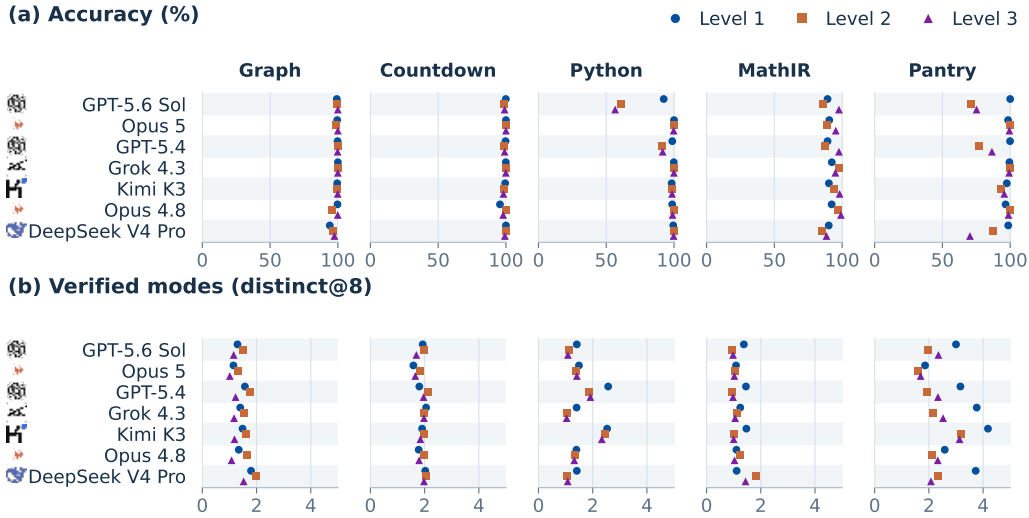


Figure 35: **Accurate hosted outputs can still cover few verified modes.** Columns show domains; markers distinguish Levels 1–3. Each point uses all eight responses to each of 128 prompts. Top: per-response accuracy. Bottom: distinct@8. A fixed formatting normalizer applies to every deployment; Opus 5 Python uses revised task wording without a system message. Table 80 reports the corresponding per-level averages of pass@8 and distinct@8 across all five domains.

Model	Level 1		Level 2		Level 3	
	pass@8 (%)	# modes	pass@8 (%)	# modes	pass@8 (%)	# modes
GPT-5.6 Sol	82.5	1.31	66.9	0.76	76.2	0.88
GPT-5.4	82.5	1.67	71.2	0.92	70.0	0.94
Grok 4.3	82.5	1.64	86.9	1.41	90.0	1.45
Kimi K3	92.5	1.91	92.5	1.37	95.6	1.59
Claude Opus 4.8	91.2	1.46	100.0	1.62	100.0	1.51

Table 81: **Hosted success and output diversity with reasoning disabled.** Each level averages five domains equally: 32 prompts per domain and eight responses per prompt. pass@8 is the fraction with any verified response; # modes is mean distinct@8, including failures. The frozen formatting normalizer applies throughout. Appendix M.5 gives matched medium-reasoning results.

M.5 MATCHED REASONING-CONTROL COMPARISON

We selected original row indices 0–31 in each of five domains and three levels, without consulting outputs. Each model and condition therefore has 480 prompts and 3,840 responses. The medium baseline reuses the original audited responses for these exact prompts and eight sample slots; no new medium responses were collected. Both conditions preserve the original system and user prompt bytes,

including the original Python wording. The 128-prompt medium results in Table 80 and Figure 35 are a separate, larger display; the comparison below uses only the matched 32-prompt subset.

Only the provider reasoning control changes, with an 8,192-token output budget retained in both conditions. GPT-5.6 Sol and GPT-5.4 request `reasoning_effort=none`; Grok 4.3 and Kimi K3 request `reasoning_effort=none`. Claude Opus 4.8 requests `thinking.type=disabled` while retaining `output_config.effort=medium`. These controls denote disabled explicit reasoning at the endpoint, without establishing the absence of hidden internal computation. Native response receipts are checked for control violations. The same frozen formatting normalizer and canonical mode grader apply to both conditions; the accompanying JSON retains strict grades, domain-level counts, controls, and source hashes.

Model	Level	Medium reasoning		Reasoning disabled	
		pass@8 (%)	# modes	pass@8 (%)	# modes
GPT-5.6 Sol	1	96.9	1.81	82.5	1.31
	2	98.8	1.54	66.9	0.76
	3	98.8	1.51	76.2	0.88
GPT-5.4	1	96.9	2.06	82.5	1.67
	2	99.4	1.68	71.2	0.92
	3	99.4	1.82	70.0	0.94
Grok 4.3	1	100.0	1.93	82.5	1.64
	2	100.0	1.62	86.9	1.41
	3	100.0	1.54	90.0	1.45
Kimi K3	1	96.9	2.27	92.5	1.91
	2	100.0	2.02	92.5	1.37
	3	100.0	1.89	95.6	1.59
Claude Opus 4.8	1	98.1	1.61	91.2	1.46
	2	100.0	1.67	100.0	1.62
	3	100.0	1.57	100.0	1.51

Table 82: **Matched medium and disabled reasoning, by level.** Each row averages 160 prompts across five domains. All eight responses and zero-success prompts are retained. # modes denotes mean `distinct@8`; estimates use the frozen formatting normalizer.

Model	Medium reasoning		Reasoning disabled	
	pass@8 (%)	# modes	pass@8 (%)	# modes
GPT-5.6 Sol	98.1	1.62	75.2	0.99
GPT-5.4	98.5	1.85	74.6	1.18
Grok 4.3	100.0	1.70	86.5	1.50
Kimi K3	99.0	2.06	93.5	1.62
Claude Opus 4.8	99.4	1.62	97.1	1.53

Table 83: **Matched reasoning-control averages across levels.** Each model and condition uses the same 480 prompts and 3,840 responses, with equal weight on every domain and level.

Incomplete deployments. Five of seven registered deployments passed complete-cohort admission. Claude Opus 5 is excluded because a native response returned a thinking block and 40 thinking tokens despite an explicit disabled-thinking request. DeepSeek V4 Pro is excluded because only 3,839 of 3,840 sample receipts were recovered, leaving one unresolved request. Neither partial cohort receives a score, and the missing response is not counted as a failure.

Scope of the comparison. Across all five complete deployments, disabling reasoning lowers both overall `pass@8` and observed `distinct@8`. Reduced success also reduces opportunities to observe distinct successful modes, so this does not isolate diversity at fixed accuracy. Medium and disabled outputs were collected at different times; matching sample slots are not shared random seeds. Omitted temperature and `top_p` controls retain provider defaults, which can vary over time. These are descriptive point estimates without uncertainty intervals, not causal claims about training.

Part VI. Idealized analysis

N MODE COLLAPSE AND VERIFIED REPLAY

Under the explicit categorical assumptions below, centered binary-reward estimators amplify a pre-existing imbalance among correct modes, whereas persistent verified replay supplies a loss barrier for banked modes. The complete-response argument then states what would be needed to transfer that protection to a language model. These results concern idealized update rules; they do not certify the implemented AdamW/PPO trajectories. Table 2 lists the symbols used here.

N.1 CATEGORICAL ABSTRACTION AND MEAN UPDATES

Every result in this appendix is stated under one model, fixed here once.

Assumption N.1 (Categorical mean-flow model). Fix one isolated prompt and a finite category set $\mathcal{A} = \mathcal{C} \dot{\cup} \mathcal{N}$ with $m \geq 2$ correct execution modes and $n \geq 1$ incorrect categories, and set $p_a = \text{softmax}(z)_a$, $P = \sum_{c \in \mathcal{C}} p_c$, and $q_c = p_c/P$, so that P measures correctness while q describes how that correct probability is allocated among the individual correct modes.

- (A1) *Independent logits*. The policy carries one independently optimized logit for every category in $\mathcal{A}$, including one separate logit for each canonical correct mode.
- (A2) *Common length normalization*. Every response shares a single length scale, which is absorbed into the positive coefficient of the group estimator, rather than a scale that varies from one response to the next within the same rollout group.
- (A3) *Euclidean mean flow*. Updates are the infinitesimal expected updates in these logits, taken without optimizer preconditioning and without momentum carried between steps.
- (A4) *No competing regularizer*. Explicit reference KL is zero and no entropy term is present.
- (A5) *Nondegenerate start*. Logits are finite initially, $0 < P(0) < 1$, and the group size is $G \geq 2$.

We write “under Assumption N.1” for (A1)–(A5) together. Each limitation of the analysis can be traced to a specific item: AdamW and PPO violate (A3), variable-length GRPO violates (A2), and entropy regularization violates (A4).

A language model induces probabilities over canonical outcomes, but this identification of probabilities does not identify its update geometry. Aggregating several response strings into a mode does not generally commute with gradient flow in response logits. One independent logit per canonical outcome is therefore an explicit abstraction, not a reduction proved for the neural policy or its validator equivalence classes.

Draw a group $Z_1, \dots, Z_G \stackrel{\text{iid}}{\sim} p$ with $G \geq 2$, write $R_i = \mathbf{1}[Z_i \in \mathcal{C}]$, and let $R = \sum_i R_i$. At the on-policy point where the PPO ratio equals one, consider the common-length estimator

$$\hat{g}_G = \frac{1}{G} \sum_{i=1}^G w(R) \left(R_i - \frac{R}{G} \right) \nabla_z \log p_{Z_i}. \quad (14)$$

For the Dr.GRPO abstraction, $w(R) = 1$ after absorbing the shared positive length scale. Reward-standardized GRPO with equal or common response-length normalization takes $w(R)$ to be the reciprocal group reward standard deviation on mixed groups (with a fixed numerical stabilizer if used). Standard variable-length GRPO also weights each response by its own inverse length (Shao et al., 2024; Liu et al., 2025); that factor cannot generally be absorbed into $w(R)$ and is outside the equivalence proved here. In either included case, $0 < w(\ell) < \infty$ for $\ell \in \{1, \dots, G-1\}$. We define the entire update as zero on all-equal groups without evaluating an undefined reciprocal standard deviation; likewise, $w(R)R(G-R)$ below is defined as zero when $R = 0$ or $R = G$. All advantages and normalization weights are detached in the score estimator.

Lemma N.2 (A group baseline does not remove frequency amplification). *For $0 < P < 1$, the expected update in Equation (14) is*

$$\mathbb{E}[\hat{g}_G] = c_G(P) \nabla_z P, \quad c_G(P) = \frac{\mathbb{E}[w(R)R(G-R)]}{G^2P(1-P)} > 0. \quad (15)$$

For the Dr.GRPO abstraction, $c_G(P) = (G-1)/G$. Group centering and reward standardization rescale the expected gradient without changing its direction under these assumptions.

Proof. Condition on the complete binary reward vector, whose sum is R . The score identities give

$$\mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 1] = \nabla_z \log P, \quad \mathbb{E}[\nabla_z \log p_{Z_i} \mid R_i = 0] = \nabla_z \log(1-P).$$

There are R correct rows with centered reward $(G-R)/G$ and $G-R$ incorrect rows with centered reward $-R/G$. Therefore the conditional expectation of the unaveraged sum in (14) is

$$w(R) \frac{R(G-R)}{G} (\nabla_z \log P - \nabla_z \log(1-P)) = w(R) \frac{R(G-R)}{GP(1-P)} \nabla_z P.$$

The outer factor $1/G$ and expectation over $R \sim \text{Binomial}(G, P)$ yield (15). For $w \equiv 1$, $\mathbb{E}[R(G-R)] = G(G-1)P(1-P)$, proving the Dr.GRPO value. $\square$

The binomial identity also gives the denominator-free expression

$$c_G(P) = \frac{G-1}{G} \sum_{j=0}^{G-2} \binom{G-2}{j} w(j+1) P^j (1-P)^{G-2-j}.$$

The binomial weights sum to one, so c_G lies between $(G-1) \min_j w(j+1)/G$ and $(G-1) \max_j w(j+1)/G$. This establishes the smooth, positive extension to $P = 0, 1$ used in the subsequent flow and convergence arguments.

The next lemma specializes the practical-estimator analysis of Tajwar et al. (2026, Section 4.3 and App. D, Theorem 5) to our categorical notation; we include the derivation for completeness.

Lemma N.3 (Binary MaxRL has the same correctness-gradient direction). *Under Assumption N.1, let MaxRL assign $A_i^{\text{MaxRL}} = GR_i/R - 1$ when $R > 0$ and zero when $R = 0$, as in Section 3.3. For $\hat{g}_G^{\text{MaxRL}} = G^{-1} \sum_i A_i^{\text{MaxRL}} \nabla_z \log p_{Z_i}$ at the on-policy point, the expected update is*

$$\mathbb{E}[\hat{g}_G^{\text{MaxRL}}] = c_G^{\text{MaxRL}}(P) \nabla_z P, \quad c_G^{\text{MaxRL}}(P) = \frac{1 - (1-P)^{G-1}}{P} > 0, \quad (16)$$

with continuous endpoint values $c_G^{\text{MaxRL}}(0) = G-1$ and $c_G^{\text{MaxRL}}(1) = 1$.

Proof. Condition on $R = r > 0$. Each successful row has advantage $(G-r)/r$ and each failed row has advantage -1 . Using the same conditional score identities as above, the expected unaveraged sum is

$$(G-r)(\nabla_z \log P - \nabla_z \log(1-P)) = \frac{G-r}{P(1-P)} \nabla_z P.$$

After the outer factor $1/G$ and averaging over $R \sim \text{Binomial}(G, P)$,

$$\mathbb{E}[(G-R)\mathbf{1}\{R > 0\}] = G(1-P) - G\Pr(R=0) = G[(1-P) - (1-P)^G].$$

Substitution gives Equation (16). Its endpoint values follow by continuity (or the finite geometric-series identity), which also gives the stated continuous extension at both boundaries. $\square$

The same finite-series identity gives the potential used below:

$$\Psi_{\text{MaxRL}}(P) := \int_0^P c_G^{\text{MaxRL}}(s) ds = \sum_{k=1}^{G-1} \frac{1 - (1-P)^k}{k}. \quad (17)$$

In particular, $\Psi_{\text{MaxRL}}(1) = \sum_{k=1}^{G-1} 1/k$. For binary rewards, $\text{best}@k$ equals $\text{pass}@k$, so this is a harmonic mixture of sampling objectives for $k = 1, \dots, G-1$. The upper limit is $G-1$, not G : the implemented centered estimator drops the entire update on all-failure groups. Subtracting an unconditional score baseline even on those groups instead gives the order- G estimator. This distinction concerns the exact finite-group expectation at the on-policy point; it does not assert unbiasedness of subsequent clipped or adaptive-optimizer updates.

N.2 WINNER-TAKE-ALL COLLAPSE IN THE CATEGORICAL MEAN FLOW

Consider $\dot{z} = c_G(P)\nabla_z P$, with any of the smooth positive coefficients established above: Dr.GRPO, common-length reward-standardized GRPO, or the specified centered binary MaxRL estimator. This infinitesimal expected-update model removes sampling noise and retains independently trained Euclidean logits. The conditional dynamics reduce to the standard frequency-dependent replicator system with fitness $f_c(q) = q_c$ (Hofbauer & Sigmund, 1998). The result is a specialization to the group estimators, not a new general theorem about replicator dynamics. Related correctness-indifferent collapse analyses include Sinha et al. (2026); Lochab et al. (2026).

Theorem N.4 (Standard replicator specialization for binary-reward RL). *Under Assumption N.1, if the conditional correct-mode distribution $q(0)$ has a unique largest coordinate c^* , then*

$$P(t) \longrightarrow 1, \quad q_{c^*}(t) \longrightarrow 1, \quad q_c(t) \longrightarrow 0 \quad (c \neq c^*).$$

Thus these categorical mean flows converge to a single correct execution mode for generic initial conditions, even though every mode in $\mathcal{C}$ has exactly the same reward.

Remark N.5. Theorem N.4 is asymptotic and internal to (A1)–(A3): it claims neither finite-time support loss nor convergence of the stochastic neural optimizers used in our experiments.

Proof. The coefficient c_G is smooth and bounded on $[0, 1]$, and the softmax reward gradient is bounded. Thus the logit flow exists for all finite times and cannot develop infinite logits at finite time. For a correct mode c and incorrect category a ,

$$\dot{z}_c = c_G(P)p_c(1 - P), \quad \dot{z}_a = -c_G(P)p_a P.$$

Also $\dot{P} = c_G(P)\|\nabla_z P\|_2^2 > 0$. If P converged to an interior value, continuity and positivity of c_G , together with

$$\|\nabla_z P\|_2^2 \geq P^2(1 - P)^2 \left(\frac{1}{m} + \frac{1}{n} \right),$$

would keep $\dot{P}$ bounded away from zero, a contradiction. Hence $P \rightarrow 1$.

Define the effective optimization time

$$\tau(t) = \int_0^t c_G(P(s))P(s)(1 - P(s)) ds.$$

If $\tau(\infty) < \infty$, every correct and incorrect logit would have finite total variation, because the absolute velocity of each is at most $\dot{\tau}$. All logits would then converge to finite values, contradicting $P \rightarrow 1$. Therefore $\tau(t) \rightarrow \infty$, leaving infinite effective time for the correct-simplex dynamics below.

On the correct simplex, changing from t to τ gives the replicator flow

$$\frac{dq_c}{d\tau} = q_c \left(q_c - \sum_{d \in \mathcal{C}} q_d^2 \right), \quad \frac{d}{d\tau} \log \frac{q_c}{q_d} = q_c - q_d. \quad (18)$$

The initially unique maximum remains the unique maximum. For $c \neq c^*$, set $\xi_c = q_c/q_{c^*}$, so $\xi_c(0) < 1$ and ξ_c is nonincreasing. Since $q_{c^*} \geq 1/m$,

$$\frac{d \log \xi_c}{d\tau} = -q_{c^*}(1 - \xi_c) \leq -\frac{1 - \xi_c(0)}{m}.$$

Hence $\xi_c(\tau) \leq \xi_c(0) \exp[-(1 - \xi_c(0))\tau/m] \rightarrow 0$. Every competing mode vanishes relative to c^* ; normalization then yields $q_{c^*} \rightarrow 1$, establishing the claimed single-mode limit. $\square$

If several modes tie for the largest initial probability, symmetry preserves the tie; the same ratio argument eliminates strictly smaller coordinates and leaves the tied maxima uniform. An exactly uniform correct distribution is therefore a special initial condition. The theorem concerns limiting concentration: finite softmax logits already give positive probabilities at every finite time, so finite-time positivity alone is not a retention result.

N.3 A REPLAY SIGNAL WHEN FRESH GROUPS SUPPLY NONE

Lemma N.6 (Fresh-group starvation). *Under Assumption N.1 with correctness probability P , draw G fresh categories independently. The probability of a mixed-reward group is*

$$h_G(P) = 1 - P^G - (1 - P)^G \leq G \min\{P, 1 - P\}.$$

Every all-equal group has zero fresh task component for Equation (14) and for the specified centered MaxRL estimator; so a nonzero fresh task component requires a mixed group, an event whose probability is at most $h_G(P)$ at every correctness level.

Proof. The all-correct and all-incorrect events are disjoint and have probabilities P^G and $(1 - P)^G$. A mixed group requires both a correct and an incorrect sample, so the union bound supplies each of the two stated bounds, GP and $G(1 - P)$, and the minimum of the pair is therefore also a bound. $\square$

Lemma N.7 (Replay ascent persists as fresh signal vanishes). *Under Assumption N.1, let $\mathcal{B} \subseteq \mathcal{C}$ be a nonempty fixed verified bank with normalized positive weights w_b , let $R_w = -\sum_b w_b \log p_b$ with $\rho_w > 0$, and extend w by zero outside the bank to obtain $\bar{w}$. The deterministic replay ascent vector is*

$$v^{\text{replay}} = -\rho_w \nabla R_w = \rho_w (\bar{w} - p),$$

which vanishes only at $p = \bar{w}$. If $p_b \leq w_b/2$ for a banked mode, then $v_b^{\text{replay}} \geq \rho_w w_b/2$, even when b is absent from the fresh group. For a uniform bank of $k \geq 2$ modes and fixed $\rho_w > 0$, as $p \rightarrow \mathbf{e}_b$ at one banked correct mode, $h_G(P) \rightarrow 0$ whereas $\|v^{\text{replay}}\|_2 \rightarrow \rho_w \sqrt{1 - 1/k} > 0$.

Proof. For $R_w = -\sum_b w_b \log p_b$, the softmax score identity gives $\partial R_w / \partial z_a = p_a - \bar{w}_a$, proving the replay formula and coordinate bound. Finally,

$$\|\bar{w} - \mathbf{e}_b\|_2^2 = (1 - 1/k)^2 + (k - 1)/k^2 = 1 - 1/k,$$

which gives the strictly positive limiting norm for $k \geq 2$. $\square$

Remark N.8. Together the two lemmas say only that a banked target can supply a verified signal when the fresh task gradient is unavailable. Lemma N.6 bounds an opportunity, not a gradient magnitude, and says nothing about other loss terms, clipping, or momentum carried across updates. The vector in Lemma N.7 averages the fixed bank, so a stochastic exemplar draw realizes it only in expectation, and a nonzero logit coordinate can still be annihilated by a degenerate neural Jacobian. Replay also need not increase the next mixed-group probability, since $h_G(P)$ is largest at $P = 1/2$.

N.4 FIXED-BANK REPLAY AND QUALITATIVE RETENTION

Let $\mathcal{B} \subseteq \mathcal{C}$ be a nonempty fixed bank of k modes that the policy has produced and the validator has accepted. Uniform verified replay uses one exemplar per banked mode and, in the categorical reduction, minimizes

$$R_{\mathcal{B}}(z) = -\frac{1}{k} \sum_{b \in \mathcal{B}} \log p_b. \quad (19)$$

This is ordinary cross-entropy against the distribution that assigns mass $1/k$ to each banked mode and zero elsewhere. It is not conditioned on the bank: decreasing (19) can raise total bank mass as well as redistribute mass within the bank.

In an averaged continuous-time model, absorb a fixed round-robin replay frequency and loss scale into an effective dose $\rho > 0$. This assumes a fixed eligible-bank set and constant relative weighting; it does not identify finite alternating updates with simultaneous gradient flow. The prompt-isolated categorical mean flow for any of the three fresh binary-reward objectives above plus verified replay is

$$\dot{z} = c_G(P) \nabla_z P - \rho \nabla_z R_{\mathcal{B}}(z). \quad (20)$$

Let $\Psi(P) = \int_0^P c_G(s) ds$. For the finite-group estimators in Lemmas N.2 and N.3, c_G has a finite positive extension to $[0, 1]$; in particular, Dr.GRPO has $c_G = (G - 1)/G$ and MaxRL has $c_G = [1 - (1 - P)^{G-1}]/P$. Define

$$F_{\mathcal{B}}(z) = \rho R_{\mathcal{B}}(z) - \Psi(P(z)). \quad (21)$$

Then (20) is exactly gradient flow $\dot{z} = -\nabla_z F_{\mathcal{B}}$.

Theorem N.9 (Verified replay prevents banked-mode extinction). *Suppose logits are finite at time T and thereafter the nonempty verified bank $\mathcal{B}$ is fixed, fits within replay capacity, and receives a fixed positive effective replay dose ρ . Under (20), define*

$$C_T = R_{\mathcal{B}}(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho}.$$

Then for every $b \in \mathcal{B}$ and every $t \geq T$,

$$p_b(t) \geq \exp(-kC_T) > 0. \quad (22)$$

Thus each banked probability is uniformly bounded away from zero for all $t \geq T$, including asymptotically, even when the bank does not cover every correct mode. This is stronger than finite-time positivity, which already follows from finite softmax logits without replay.

Proof. The smooth vector field has bounded velocity because c_G , $\nabla_z P$, and $\nabla_z R_{\mathcal{B}}$ are bounded, so finite initial logits give a flow for every finite $t \geq T$. Along this flow,

$$\frac{d}{dt} F_{\mathcal{B}}(z(t)) = -\|\nabla_z F_{\mathcal{B}}(z(t))\|_2^2 \leq 0.$$

Since $P(t) \leq 1$ and Ψ is increasing,

$$R_{\mathcal{B}}(z(t)) \leq R_{\mathcal{B}}(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho} = C_T.$$

Every $-\log p_b$ is nonnegative and their sum is $kR_{\mathcal{B}}$. Therefore $-\log p_b(t) \leq kC_T$ for every banked mode, which is exactly Equation (22) for every time after T . $\square$

The floor is qualitative and can be numerically vacuous. For example, at $k = 16$ even a hypothetical $C_T = 160$ gives e^{-2560} , far below any useful evaluation probability. The argument establishes a uniform-in-time lower bound in the stated ideal flow, not practical visibility or a prediction for the experimental replay loss. The effective dose also depends on relative fresh/replay frequencies, not the replay coefficient alone.

A common nonnegative, locally integrable rate $\eta(t)$ multiplying both terms of (20) preserves the energy bound, since $\dot{F}_{\mathcal{B}} = -\eta(t)\|\nabla F_{\mathcal{B}}\|_2^2 \leq 0$. The convergence statements below additionally require $\int_T^\infty \eta(t) dt = \infty$ and then follow by reparameterizing optimization time. Infinitely many replay visits alone establish neither this condition nor a fixed replay-to-fresh weighting. Different time-varying relative weights are outside the fixed-potential argument.

Full coverage and the categorical target.

Corollary N.10 (Conditional full-coverage categorical limit). *Assume the conditions of Theorem N.9 and that by time T the bank covers the full correct support, $\mathcal{B} = \mathcal{C}$. Let $u_c = 1/m$ on $\mathcal{C}$. Then the categorical mean flow converges to*

$$P(t) \longrightarrow 1, \quad q(t) \longrightarrow u.$$

Consequently

$$\min_{c \in \mathcal{C}} q_c(t) \longrightarrow \frac{1}{m}, \quad \mathcal{H}(q(t)) \longrightarrow \log m,$$

and conditional correct-mode collapse is impossible.

Proof. For full coverage,

$$R_{\mathcal{C}} = \log m - \log P + \text{KL}(u||q). \quad (23)$$

The potential is bounded below by $-\Psi(1)$, so its descent gives $\int_T^\infty \|\nabla F_{\mathcal{C}}\|_2^2 dt < \infty$. The finite-group c_G is smooth on $[0, 1]$, and softmax derivatives and the cross-entropy gradient are bounded. Thus the Hessian of $F_{\mathcal{C}}$ and the velocity are bounded; the gradient along the trajectory is uniformly continuous. Square integrability then implies $\nabla F_{\mathcal{C}} \rightarrow 0$: recurring gradients of fixed positive size would otherwise contribute energy over intervals of a uniform minimum length.

Every limiting probability vector therefore satisfies the correct-coordinate equations

$$0 = \rho(p_c - 1/m) - c_G(P)p_c(1 - P).$$

Summing them gives $0 = -(1 - P)(\rho + c_G(P)P)$, hence $P = 1$. Each correct coordinate is then $1/m$ and every incorrect coordinate is zero. Compactness of the probability simplex and uniqueness of this limit establish convergence of the probability vector. $\square$

Positive weights and the implemented length normalization. Uniformity of the target is separate from protection against extinction. For fixed weights $w_b > 0$ with $\sum_{b \in \mathcal{B}} w_b = 1$, define $R_w = -\sum_b w_b \log p_b$ and use replay coefficient $\rho_w > 0$. The same potential argument gives, for all $t \geq T$,

$$p_b(t) \geq \exp(-C_{T,w}/w_b), \quad C_{T,w} = R_w(z(T)) + \frac{\Psi(1) - \Psi(P(T))}{\rho_w}.$$

If $\mathcal{B} = \mathcal{C}$, then $R_w = H(w) - \log P + \text{KL}(w||q)$, and the independent-logit limit is $P \rightarrow 1$, $q \rightarrow w$. The preceding stationarity proof applies with $1/m$ replaced by w_c : summing the correct-coordinate equations forces $P = 1$, then each equation forces $p_c = w_c$.

A partial bank has a stronger limitation in this same ideal flow. For any fixed nonempty $\mathcal{B} \subseteq \mathcal{C}$ and fixed positive weights summing to one, extend w by zero to $\bar{w}$ on all categories. Then $p(t) \rightarrow \bar{w}$, so even correct modes outside the frozen bank vanish asymptotically. For $F = \rho_w R_w - \Psi(P)$, the preceding energy and bounded-Hessian argument still gives $\nabla F \rightarrow 0$; neither argument requires full bank coverage. At every subsequential probability limit, the correct-coordinate equations are

$$0 = \rho_w(p_c - \bar{w}_c) - c_G(P)p_c(1 - P).$$

Summing over correct categories gives $0 = -(1 - P)(\rho_w + c_G(P)P)$, hence $P = 1$; each equation then gives $p_c = \bar{w}_c$, and all incorrect mass is zero. Compactness and uniqueness of this limiting probability vector prove convergence. Thus protecting a fixed set of banked modes need not preserve unbanked correct alternatives. This conclusion concerns the fixed-bank categorical flow, not changing-bank or stochastic neural training.

This distinction matters for the implemented mean-token exemplar loss. Even if each key were represented by exactly one categorical exemplar of fixed length L_b , its loss would be AR_w , where

$$A = \frac{1}{k} \sum_{b \in \mathcal{B}} \frac{1}{L_b}, \quad w_b = \frac{L_b^{-1}}{\sum_{j \in \mathcal{B}} L_j^{-1}}.$$

Its effective categorical replay coefficient would be $\rho_w = \rho_{\text{LM}}A$. The full-coverage limit would therefore weight shorter exemplars more, and would be uniform only for equal lengths. For an actual language model, $\pi_\theta(e_b | x)$ is only one contribution to the total key probability. Equal averaging of per-token exemplar scores does not imply uniform learned key probabilities; the categorical uniformity corollary is not that claim.

Full support is an additional premise, not a consequence of replay. In Python factors, prompts admit 16–3,600 correct execution modes (mean 229.44), while the bank capacity is 16. The full-coverage corollary therefore cannot supply a domain-wide guarantee there. The retention theorem protects only admitted banked modes; it supplies no probability floor for an unbanked key.

N.5 COMPLETE RESPONSES AND FINITE EVALUATION VISIBILITY

The learner scores a particular response string, while evaluation groups strings by execution key. Event containment provides a conditional bridge: one complete verified response is one way of producing its key. This bridge also makes the loss’s length factor explicit. Here $p_\theta(b | x) = \Pr_{Y \sim \pi_\theta(\cdot | x)}[V(Y, s_x) = b]$ is the unconditional probability of producing key b , rather than the probability conditional on correctness.

Lemma N.11 (Retention from a joint exemplar potential). *Fix finitely many verified complete-response exemplars (x_j, e_j) , each realizing key b_j , with lengths $L_j > 0$ and fixed weights $\alpha_j > 0$. Let*

$$F(\theta) = \sum_j \alpha_j \frac{-\log \pi_\theta(e_j | x_j)}{L_j} - J(\theta), \quad J(\theta) \leq J_{\max} < \infty.$$

Suppose $F(\theta(T))$ is finite and $F(\theta(t)) \leq F(\theta(T))$ for every $t \geq T$. With $D = F(\theta(T)) + J_{\max}$, every protected exemplar satisfies

$$p_\theta(b_j | x_j) \geq \pi_\theta(e_j | x_j) \geq \exp\left(-\frac{L_j D}{\alpha_j}\right) > 0 \quad (t \geq T).$$

Proof. The weighted exemplar loss equals $F(\theta(t)) + J(\theta(t)) \leq D$. Each summand is nonnegative, so $-\log \pi_\theta(e_j | x_j) \leq L_j D / \alpha_j$. The complete-response event is a subset of the event producing key b_j , which proves the stated lower bound on every protected key’s probability. $\square$

Exact descent of a smooth joint potential is sufficient for the energy inequality, even with shared parameters and multiple prompts. For example, the ideal categorical model permits $J(\theta) = \sum_x \nu_x \Psi_x(P_x(\theta))$ with fixed nonnegative prompt weights and $J_{\max} = \sum_x \nu_x \Psi_x(1)$. The lemma does not establish this descent condition for stochastic, clipped, alternating AdamW updates, and it neither implies uniform neural key probabilities nor requires them. Its bound applies only to the listed exemplar–prompt pairs. Shared parameters do not extend it to held-out prompts; transfer to those prompts remains an empirical question.

The scored probability must describe the complete response, including its termination event or a fixed deterministic horizon, under the same prompt, temperature, masks, truncation rules, and verifier as the sampling law being bounded. A prefix score or a score under another decoding law does not supply this bound without a separate event or likelihood comparison. For a complete response of length L and mean-token log score s , its probability is e^{Ls} , not e^s . A change in this exemplar’s likelihood need not equal the change in its whole key’s probability.

Retention and finite evaluation visibility. At a fixed checkpoint, suppose k distinct banked keys for one prompt have probability floors $p_b \geq \delta_b > 0$. For K independent draws from that same policy, the banked distinct count $D_{\mathcal{B},K}$ satisfies

$$\mathbb{E}[D_{\mathcal{B},K}] \geq \sum_{b \in \mathcal{B}} [1 - (1 - \delta_b)^K],$$

$$\Pr(\text{some banked key is unseen}) \leq \sum_{b \in \mathcal{B}} (1 - \delta_b)^K \leq k e^{-K \delta_{\min}},$$

where $\delta_{\min} = \min_b \delta_b$. The first statement follows from indicator expectations and the second from the union bound. Thus $K \geq \log(k/\varepsilon)/\delta_{\min}$ suffices to observe all banked keys with probability at least $1 - \varepsilon$, for $0 < \varepsilon < 1$. The energy floors can make this budget impractically large; asymptotic retention does not imply visibility in an eight-draw evaluation.

Conversely, not observing a mode does not establish extinction. At a fixed checkpoint, zero sightings of a prespecified key in N independent draws give a one-sided $(1 - \alpha)$ binomial upper bound $1 - \alpha^{1/N}$ and lower bound zero (Clopper & Pearson, 1934). For $N = 32$ and $\alpha = .05$, the upper bound is approximately 8.94%. These are single-key bounds; simultaneous statements need error allocation. Counts from changing checkpoints cannot be pooled to certify a current policy, and teacher-forced scores are not independent Bernoulli observations.

N.6 SCOPE RELATIVE TO ENTROPY, ADMISSION, AND NEURAL OPTIMIZATION

Dependence on optimization geometry. For a smooth parameterization $z = z(\theta)$ with Jacobian $J_\theta = \partial z / \partial \theta$, Euclidean parameter flow instead induces $\dot{z} = c_G(P) J_\theta J_\theta^\top \nabla_z P$. The independently trained logit theorem uses $J_\theta J_\theta^\top = I$ and does not automatically extend to arbitrary shared parameters or another optimization metric. Equal binary reward alone does not force this winner selection: the categorical natural-gradient reward flow can be represented by $\dot{z}_a = c_G(P)(R_a - P)$, which gives identical velocities to correct logits and preserves their ratios. Here $R_a = \mathbf{1}[a \in \mathcal{C}]$. Parameter count alone does not specify the optimization geometry. The empirical scale comparisons also change pretrained models and learning-rate schedules; they neither measure the geometry nor identify the cause of cross-scale differences. The theorem supplies a mechanism to test in controlled training; it cannot identify the training history or cause of concentration in a hosted model.

A bounded advantage based on fresh samples has a different boundary behavior. The next bound includes group-dependent detached scores and distinguishes sampled updates from gradients of an exactly evaluated entropy functional.

Lemma N.12 (Bounded sampled scores have no probability-independent boundary force). *Under Assumption N.1, condition on the training history, draw $Z_1, \dots, Z_G$ independently from p , and let a_i be any detached, possibly group-dependent advantages with $|a_i| \leq B$. To interpret the bound uniformly as $p_b \rightarrow 0$, assume a finite B independent of that limit. At the on-policy point define*

$$\hat{g} = \frac{1}{G} \sum_{i=1}^G a_i (e_{Z_i} - p).$$

For every category b ,

$$|\mathbb{E}[\hat{g}_b]| \leq \mathbb{E}|\hat{g}_b| \leq 2Bp_b(1 - p_b).$$

On a group with no draw of b , exactly $\hat{g}_b = -p_b G^{-1} \sum_i a_i$; it is zero if the applied advantages sum to zero. By comparison, the verified-replay component for a fixed target w has logit velocity $\rho(w_b - p_b)$, which approaches $\rho w_b > 0$ for a banked category as $p_b \rightarrow 0$.

Proof. Use the triangle inequality and $|a_i| \leq B$. Each marginal draw satisfies $\mathbb{E}|\mathbf{1}\{Z_i = b\} - p_b| = 2p_b(1 - p_b)$, irrespective of the dependence of a_i on the group. On the absence event all indicator terms are zero. The replay formula follows by differentiating $-\sum_b w_b \log p_b$. $\square$

The bound compares the isolated logit-gradient components. It does not prove inevitable collapse, rule out retention by a bounded estimator, or guarantee that replay dominates the full update. An unsampled mode can still change through softmax normalization or shared parameters. Exact entropy is outside the uniformly bounded-score premise because its surprisal is unbounded near zero; the lemma is not an impossibility result for entropy regularization. Nor does the absence of a mode from a finite sample establish zero probability.

Exact entropy is a different comparator. For a known full correct support and uniform u , the elementary identities

$$H(q) = \log m - \text{KL}(q\|u), \quad R_u(p) = \log m - \log P + \text{KL}(u\|q)$$

show that exact conditional entropy and categorical replay both favor a uniform conditional target. With strictly increasing correctness potential Ψ , both $\Psi(P) + \beta H(q)$ and $\Psi(P) - \rho R_u(p)$ have the unique global optimum $P = 1, q = u$ in probability space on the simplex with $P > 0$, for positive coefficients. This is a boundary limit for finite-logit softmax policies. The optimum follows from nonnegativity of KL and increasing Ψ ; it does not assert convergence of either neural update. The KL direction distinguishes their loss level sets (Pereyra et al., 2017; Agarwal et al., 2021): replay loss diverges when a protected coordinate vanishes, whereas conditional entropy can remain $\log(m - 1)$ with one missing mode.

Exact full-output entropy is a third objective and also regularizes incorrect mass. Exact tabular entropy-regularized policy gradient has its own positive probability and convergence guarantees under specified assumptions (Mei et al., 2020); our collapse theorem contains no entropy term. The sampled semantic scores in the experiments are not exact gradients of full-output or conditional verified-key entropy. Neither their empirical comparison nor the bounded-score lemma is an impossibility result for entropy.

Discovery and optimization remain separate requirements. Retention starts after a verified key enters active memory. Actual insertion depends on generation, validation, capacity, and eligibility; a positive sampling probability alone does not establish eventual admission. Once a non-evicting capacity-16 bank fills, an unbanked key can remain ineligible. Changing membership or exemplar weights changes the loss being bounded, so fixed-bank descent cannot simply be carried across those changes. In particular, the fixed-weight argument does not cover a target whose weights continue to change with fresh observation counts.

For a fixed joint potential, a discrete bound $F(\theta_{n+1}) \leq F(\theta_n) + \epsilon_n$ with $\epsilon_n \geq 0$ and $\sum_{n \geq n_0} \epsilon_n = E < \infty$ would extend Lemma N.11, using $D = F(\theta_{n_0}) + J_{\max} + E$ for steps $n \geq n_0$. This follows by telescoping and bounds retention, not convergence. The noisy optimizer would need a separate argument establishing such control of the joint loss’s upward drift. A positive replay coefficient or small nominal learning rate does not establish it. Adam moments, clipping, momentum, prompt scheduling, and bank changes are not covered by the exact-flow proofs. The experiments have not established the required joint-energy or update-error assumptions. The mathematical contribution is the stated categorical mechanism and conditional exemplar argument; empirical support and longitudinal exemplar scores remain separate measurements.

Part VII. Records

O SOURCE INTEGRITY AND REPRODUCIBILITY

O.1 ADMISSIBLE SOURCES AND MISSING CHECKPOINTS

Falcon3-1B Countdown Re:Dr seed 59 contains two complete four-draw terminal evaluation sequences with identical request identities but conflicting metrics. That source is excluded at every checkpoint and from every efficacy endpoint, paired contrast, and aggregate that reuses it; neither sequence is selected or averaged. The independently valid Dr.GRPO and GRPO seed-59 sources remain admissible. The core replay comparison therefore has 74 pairs: 14 complete five-seed blocks and Falcon Countdown at $n = 4$ (seeds 55–58). Primary partial intersections are descriptive and receive no confidence interval.

A second kind of conflict is more common and has a diagnosable cause. The job watchdog can requeue a run; the restart finds no resumable checkpoint, begins from initialization, and evaluates the initial model a second time, appending a second step-0 block to the same record. Separate attempts can land on different GPU models, and GPU model changes floating-point reduction order and therefore the sampled tokens, so the two blocks disagree and the checkpoint is invalidated under the rule below. This is why several MaxRL initial checkpoints are unavailable, leaving the longitudinal contrast with three defined seeds in Falcon3-1B Graph and one in Falcon3-1B PantryPlan. The same hardware sensitivity means that initial evaluations of the identical frozen base model agree within a GPU pool and can differ across pools, which is one reason initial evaluations are measured within each method rather than shared across arms; a before–after contrast compares a method with its own initial checkpoint and is unaffected. Of the step-0 blocks in a run’s records, exactly one is the block its admitted trajectory continues from, and App. O.2 admits that block for the 27 cells where it is unique. That amendment was adopted after the analysis was specified, so it reports every affected block under both admissions.

Trajectory records use registered sources and approved continuations or replacements, with superseded attempts excluded. Identical duplicates are deduplicated; conflicting records invalidate the affected checkpoint rather than being selected by metric value. Missing or ambiguous checkpoints remain gaps; curves do not interpolate or carry values forward across them. An ambiguous nonterminal checkpoint does not invalidate an independently admissible terminal endpoint. Each curve declares its fixed source-admissible seed cohort; paired core curves use the admitted intersection. Accuracy and mode-count views use the same populations and gap rules. Supporting methods retain their explicit method-specific cohorts; available checkpoint counts and gaps are recorded alongside the curves. Machine-readable records identify the sources, exclusions, exact seed sets, checkpoint rule, and source hashes.

O.2 CONFLICTED INITIAL CHECKPOINTS, ADMITTED BY PROVENANCE

A conflicted step-0 endpoint is a requeue duplicate, not a missing evaluation, and the two attempts disagree only because they executed on different GPU models. A fourth amendment to the concentration protocol therefore admits, for each affected cell, the step-0 block that the cell’s admitted trajectory continues from — chosen by file order, step number and the presence of training rows, never by a metric value. The rule resolves 27 of the 36 conflicted cells and leaves nine, where more than one such block exists, unavailable as before.

This amendment was adopted after the analysis was specified, so it reports every affected block under both admissions (Table 84) and is stated as a post-specification change rather than a confirmation. Two checks bound what it can have moved. The twelve Dr.GRPO and GRPO before–after comparisons that carry Section 5.1 are unaffected and reproduce to 10^{-9} , and no terminal replay-effect block changes at all, since those compare two trained checkpoints and use no initial endpoint.

What the re-admission buys is precision, not a new direction: every affected block moves from one to four defined seeds up to the registered five and receives a nominal interval, while its sign is unchanged. It does not license a broader collapse claim. Falcon3-1B Graph under MaxRL moves the other way, and now does so with an interval excluding zero, as does its replay variant; MaxRL concentrates on Countdown, MathIR and PantryPlan at that scale but not on Graph. We therefore continue to state the training-induced concentration result for the RLVR-only objectives it was measured on, and report the MaxRL longitudinal cohort as the mixed result it is.

Table 84: **Every block the provenance amendment touches, under both admissions.** Entries are the change in conditional collision from the initial to the terminal checkpoint on jointly eligible prompts, so positive is more concentrated. n is the defined seed count out of five registered; intervals are nominal paired Student- t and appear only for complete blocks. The preserved columns reproduce the amendment-3 analysis, which this amendment leaves on disk unchanged beside its own result.

Model	Domain	Method	Preserved		Provenance-admitted	
			n	ΔC	n	ΔC
Falcon3-1B	Countdown	MaxRL	2	+0.182	5	+0.182 [+0.182, +0.182]
Falcon3-1B	Countdown	Re:Max	1	+0.135	5	+0.123 [+0.078, +0.167]
Falcon3-1B	Graph	MaxRL	3	-0.113	5	-0.101 [-0.135, -0.067]
Falcon3-1B	Graph	Re:Max	2	-0.062	5	-0.082 [-0.109, -0.054]
Falcon3-1B	MathIR	MaxRL	1	+0.278	5	+0.217 [+0.066, +0.368]
Falcon3-1B	MathIR	Re:Max	2	+0.167	5	+0.133 [+0.031, +0.234]
Falcon3-1B	PantryPlan	MaxRL	1	+0.886	5	+0.886 [+0.886, +0.886]
Falcon3-1B	PantryPlan	Re:Max	2	+0.746	5	+0.749 [+0.648, +0.849]
Qwen2.5-3B	MathIR	Re:Dr	4	-0.012	5	-0.010 [-0.036, +0.017]

O.3 DATA, MODELS, AND SCIENTIFIC RECORDS

The ModeBench dataset provides separate level/domain configurations, fixed split roles, full row identities, and construction and admission records. Its loading guide and evaluation guide identify the reported splits and preserve auxiliary data separately. The model archive links each available model–domain–method–seed export to an immutable upload revision containing original weights, configuration, tokenizer, file hashes, completion provenance, and the applicable base-model license. The experiment index maps the core scale comparisons, MaxRL factorial, Level-2 factorial, replay-weighting ablation, and supporting controls to their model families; the scientific-record index and restore guide identify preserved analysis inputs and SHA-256 verification procedures.

Each empirical figure has a machine-readable record identified in the figure manifest, containing its displayed values and frozen numerical inputs. `paper/FIGURE_MANIFEST.md` maps both paper versions to their builders, records, and reproduction commands. Detailed source and recovery inventories remain in those records and the repository’s archived audit documents. Model availability may extend beyond the frozen analysis population; it does not change a figure’s source-admission or seed-intersection rule. Terminal exports restore inference models, while intermediate trajectories require the recorded evaluation checkpoints.

O.4 EXACT CONTINUATION AND VALIDATION

Exact training continuation requires the atomic optimizer, bank, token, cursor, and request-stream state in App. D.4, in addition to model weights. State mismatch, non-finite values, missing completion evidence, changed dataset identities, and source or aggregate hash drift are hard failures. Placement-only recovery may change hardware or wall time but must preserve the scientific update. Figure reproduction uses frozen records; missing results are never replaced by shallower checkpoints or silently incorporated from a newer archive.

P DISCLOSURES

AI USE STATEMENT

Generative AI assisted manuscript preparation, mathematical analysis, and supporting code. The authors reviewed and verified the resulting content and take responsibility for the final manuscript.

ETHICS STATEMENT

The benchmark and experiment-specific data are synthetic; this study collects no human-subject or personal data. More verified modes are not inherently better: a distinct execution may be risky or costly even when it passes the validator, and `distinct@K` does not measure

downstream value. Systems that evaluate or optimize breadth should report correctness, invalidity, and application-specific costs alongside mode counts.

REPRODUCIBILITY STATEMENT

Every reported number is computed from a versioned result file. App. O records protocols, hashes, validations, and regeneration commands, and links the public dataset and model archive; App. B records split identities. The AI Use Statement above discloses generative-AI assistance.